Fredrik Hammarberg

Optimising Weak Reference Processing in HotSpot's Z Garbage Collector

Reference Pipeline Optimisations and an Exploratory Weak-Field Mechanism for HotSpot

UPPSALA
UNIVERSITET

Abstract

In Java, weak references are a non-trivial source of garbage collection overhead in workloads where objects frequently become weakly reachable. In the Z Garbage Collector (ZGC), all discovered weak references follow the same processing pipeline regardless of whether they are associated with a `ReferenceQueue`. For references that are never registered with a queue this leads to unnecessary pending-list and enqueue-path work. This thesis investigates whether targeted modifications to ZGC's reference-processing pipeline can reduce this overhead, and whether representing weak semantics directly in object fields rather than through separate `WeakReference` objects can reduce overhead further.

Three orthogonal pipeline optimisations for queue-less `WeakReference` processing were implemented in an OpenJDK research fork: a skip-enqueue separation that routes queue-less weak references to a separate discovered list, bypassing the pending list entirely, a dynamic-array representation that replaces the intrusive linked list used during reference discovery to improve traversal locality, and a specialised clear path that removes unnecessary per-reference CAS operations. In addition, an exploratory weak-field mechanism was implemented through a Java field annotation and corresponding ZGC VM support, enabling a representation-level comparison under the same experimental setup.

The implementations were evaluated using synthetic microbenchmarks executed on a supercomputer cluster with 250 repeated runs per variant. Two benchmark designs were used: a single-object stress benchmark that concentrates the entire weak-reference load into one GC cycle, and a multi-object benchmark that releases references gradually across five GC cycles.

The results show that combining the specialised clear path with the dynamic array is the most effective pipeline configuration. In the single-object benchmark, this combination reduces median non-strong reference-processing time by 81 % and median major-collection time by 8 %. In the multi-object benchmark, the non-strong subphase reduction is 57 %, while the end-to-end major-collection gain is 1.1 %. Dynamic-array variants incur substantially higher auxiliary Garbage Collector (GCr) memory than the linked-list baseline. The combination of all three optimisations together, skip-enqueue separation, dynamic array, and specialised clear path, offers the best balance: it matches the same 81 % reduction in non-strong processing time whilst consuming approximately 30 % less auxiliary GCr memory than the clear-path-plus-array pair alone.

The weak-field mechanism achieves substantially lower major-collection times than all `WeakReference` pipeline variants, with reductions of 41 % in the single-object benchmark and 28 % in the multi-object benchmark relative to the baseline. Its advantage is structural: by eliminating `WeakReference` wrapper objects from the heap entirely, it reduces the total volume of GC work across all phases of the collection cycle, not only during reference processing. This comes at a significant implementation cost: the weak-field prototype spans 27 files across the compiler, class-file parser, interpreter, JIT compilers, JVMTI, and GCr subsystems.

Together, the results suggest that, within the evaluation setting of queue-less synthetic workloads on a single hardware platform, the choice of how weak semantics is represented in the language has a larger impact on total GC cost than pipeline-level optimisations. These findings indicate that meaningful improvements in workloads of this kind may require reconsidering how weak semantics is encoded in the language rather than only refining the processing pipeline, though how this conclusion generalises to production workloads with diverse reference patterns and multiple hardware platforms remains an open question.

Sammanfattning
I Java utgör svaga referenser en icke-trivial källa till overhead vid skräpinsamling i system där objekt frekvent blir svagt nåbara. I Z Garbage Collector (ZGC) hanteras alla identifierade svaga referenser i samma pipeline oavsett om de är associerade med en `ReferenceQueue` eller inte. För referenser som aldrig registreras med en kö leder detta till onödigt arbete i en pending list och sedan köläggning. Detta arbete undersöker om riktade modifieringar av ZGC:s referenspipeline kan minska denna overhead, och om representationen av svag semantik direkt i objektfält snarare än genom separata `WeakReference`-objekt kan minska overheaden ytterligare.

Tre ortogonala pipelineoptimiseringar för hantering av köfria `WeakReference` implementerades i en OpenJDK-fork: separata listor som kringgår köläggningen för köfria svaga referenser, en representation baserad på en dynamisk array som ersätter den intrusiva länkade listan som används vid referensidentifiering för att förbättra traverseringslokalitet, och en specialiserad rensningsfunktion som eliminerar onödiga CAS-operationer för varje referens. Dessutom implementerades en explorativ svagfältsmekanism via en Java-fältannotering och tillhörande ZGC VM-stöd, vilket möjliggör en representationsnivåjämförelse under samma experimentella upplägg.

Implementationerna utvärderades med syntetiska mikrobenchmarks körda på ett superdatorkluster med 250 upprepade körningar per variant. Två benchmarkupplägg användes: ett enkel-objekt-stressbenchmark som koncentrerar hela svagreferensbelastningen till en enda GC-cykel, och ett fler-objekt-benchmark som frigör referenser gradvis över fem GC-cykler.

Resultaten visar att kombinationen av den specialiserade rensningsfunktionen med den dynamiska arrayen är den mest effektiva pipelinekonfigurationen. I enkel-objekt-benchmarket minskar denna kombination mediantiden för icke-stark referenshantering med 81 % och median major-insamlingstiden med 8 %. I fler-objekt-benchmarket är minskningen i den icke-starka underfasen 57 %, medan end-to-end-vinsten för major-insamlingstiden är 1.1 %. Varianterna med dynamisk array medför avsevärt högre extra GC-minne än basfallet med länkad lista. Kombinationen av alla tre optimeringarna, *skip-enqueue*-separering, dynamisk array och den specialiserade *clear path*-metoden, ger den bästa kombinationen. Den uppnår samma minskning på 81 % av bearbetningstiden för icke-starka referenser, samtidigt som den använder ungefär 30 % mindre extraminne än kombinationen av enbart *clear path* och dynamisk array.

Svagfälts-mekanismen uppnår avsevärt kortare major-insamlingstider än alla `WeakReference`-pipelinevarianter, med minskningar på 41 % i enkel-objekt-benchmarket och 28 % i fler-objekt-benchmarket relativt basfallet. Fördelen är strukturell: genom att eliminera `WeakReference`-omslagsobjekt från heapen helt minskas den totala volymen GC-arbete över alla faser i insamlingscykeln, inte enbart under referenshantering. Detta medför dock en betydande implementeringskostnad: svagfälts-prototypen spänner över 27 filer i kompilatorn, klassfilsparsern, tolken, JIT-kompilatorer, JVMTI och GC-delsystem.

Sammantaget antyder resultaten att, inom det utvärderade scenariot med köfria syntetiska arbetsbelastningar på en och samma hårdvaruplattform, har valet av hur svag semantik representeras i språket en större inverkan på den totala GC-kostnaden än optimeringar på pipelinenivå. Dessa resultat tyder på att meningsfulla förbättringar i sådana arbetsbelastningar kan kräva en omvärdering av hur svag semantik kodas i språket snarare än enbart en förfining av hanteringspipelinen, men huruvida detta slutsats generaliserar till produktionsarbetsbelastningar med varierande referensmönster och olika hårdvaruplattformar förblir en öppen fråga.

Acknowledgments

The performance evaluations were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725.

I want to express my gratitude to my supervisor Stefan Johansson for his general support throughout this thesis and my subject reviewer Tobias Wrigstad for his academic guidance on the thesis as a whole and feedback on this report.

Additionally, I want to thank my colleagues at Oracle for their insights and feedback on my work throughout the development process.

Uppsala, Sweden, June 2026

Fredrik Hammarberg

Contents

List of Tables

List of Figures

List of Listings

List of Acronyms

API Application Programming Interface
CAS Compare-And-Swap
CPU Central Processing Unit
GC Garbage Collection
GCr Garbage Collector
GenAI Generative Artificial Intelligence
IQR Interquartile Range
JDK Java Development Kit
JFR Java Flight Recorder
JIT Just-In-Time (compilation)
JVM Java Virtual Machine
NAISS National Academic Infrastructure for Supercomputing in Sweden
NUMA Non-Uniform Memory Access
PEP Python Enhancement Proposal
RAII Resource Acquisition Is Initialisation
RSS Resident Set Size
SATB Snapshot-At-The-Beginning
SLURM Simple Linux Utility for Resource Management
STW Stop-The-World
UPPMAX Uppsala Multidisciplinary Center for Advanced Computational Science
ZGC Z Garbage Collector

1. Introduction

In managed runtimes, Garbage Collection (GC) is a central part of application performance: GC pauses affect latency, while concurrent GC work competes with application threads for processor time and memory bandwidth [1, 2]. OpenJDK's Z Garbage Collector (ZGC) is designed for this regime, combining sub-millisecond pauses with a predominantly concurrent architecture intended to keep pause times largely independent of heap size [3, 4, 5]. Since its introduction as an experimental collector, ZGC has matured into a production-ready, generational collector, making it a suitable platform for studying the overhead of individual components in low-latency GC [6, 4]. One such component that has received comparatively little optimisation attention is the processing of *weak references*. In Java, the class `WeakReference` provides weak semantics, allowing applications to retain references to objects without preventing those objects from being collected [7, 8]. When the Garbage Collector (GCr) determines that an object is no longer strongly reachable, being reachable only through a `WeakReference`, it clears the reference and, if the reference is associated with a `ReferenceQueue`, enqueues it for application-level processing [9, 10]. This enqueueing step serves as a GCr callback, notifying the application that the `WeakReference` has been cleared. Weak semantics, both with and without such callbacks, are used in a range of programming patterns and data structures, for example the Java library `WeakHashMap` [11].

The current ZGC reference-processing pipeline treats all weak references uniformly: discovered references are linked into an intrusive linked list, processed, and then handed to a Java-level daemon thread (the `ReferenceHandler`) for enqueueing [12, 9]. This design imposes disproportionate overhead on weak references that are not registered with a `ReferenceQueue`. Such references are used solely for weak semantics and therefore do not require the callback mechanism provided by the queue. Nevertheless, the `WeakReference` object still needs to go through the entire processing pipeline, even though the application has no need for enqueueing.

This inefficiency has previously been identified in the OpenJDK bug tracker [13], which notes the unnecessary traversal of the pending list for references without a `ReferenceQueue`. However, at the time of writing, no implementation addressing this issue has been integrated into the Java Development Kit (JDK). Weak references, unlike regular references, require explicit processing by the Java Virtual Machine (JVM) GCr [12, 14]: each discovered weak reference must be individually evaluated, its referent field cleared if the referent is unreachable, and the reference potentially enqueued. This per-reference processing cost scales linearly with the number of weak references, making it a bottleneck in applications that allocate many weak references.

The problem is amplified by a mismatch between the Application Programming Interface (API) design and its implementation. Java's `WeakReference` API provides an optional `ReferenceQueue` parameter [8], allowing applications to choose whether they want the callback mechanism or not. However, ZGC's reference processor does not take advantage of this distinction, treating all weak references as if they require enqueueing. This results in unnecessary work for references that do not use a `ReferenceQueue`. At the same time, weak semantics are currently implemented through separate `WeakReference` objects to support this callback mechanism, so applications also pay the allocation and processing cost of those objects even when they only need weak semantics. Because ZGC processes references concurrently with the application [3, 12], any reduction in per-reference overhead translates directly into better application performance. Building on that observation, this thesis investigates whether targeted modifications to ZGC's reference-processing pipeline can reduce reference-processing overhead. Two approaches were taken: first, three orthogonal optimisation approaches were explored for `WeakReference`-based processing: a skip-enqueue separation that routes queue-less weak references away from the pending-list machinery, a dynamic array replacing the intrusive linked list used during discovery, and a specialised clear path that eliminates unnecessary operations when clearing referents. Second, a weak-field mechanism was explored as an alternative implementation of weak semantics that avoids `WeakReference` objects entirely.

1.1 Problem Statement

The current ZGC reference processing pipeline does not distinguish between weak references with and without a `ReferenceQueue` during its phases [12]. All cleared references are threaded onto a pending list, transferred to the `ReferenceHandler` thread, and iterated regardless of whether they will actually be enqueued or not [9]. Additionally, the intrusive linked-list data structure used for discovered references exhibits poor cache locality [15]. Beyond these pipeline inefficiencies, representing weak semantics through separate `WeakReference` objects introduces object and heap overhead that may be avoidable if weak semantics is instead expressed directly through annotated object fields.

1.2 Research Questions

This thesis addresses the following research questions:

RQ1 How do three targeted modifications to ZGC's processing of queue-less `WeakReference` objects (a skip-enqueue separation, a dynamic-array, and a specialised clear path) affect the wall-clock time of reference processing, the total GC cycle time, the GCr memory footprint, and Java heap usage?

RQ2 How does expressing weak semantics through annotated object fields affect reference processing time, GC cycle time, GCr memory footprint, and Java heap usage compared to the unmodified baseline and to the most effective combination of the three pipeline optimisations from Research Question 1?

1.3 Contributions

The main contributions of this thesis are:

1. An implementation and evaluation of three orthogonal pipeline optimisations for queue-less `WeakReference` processing within generational ZGC: a skip-enqueue separation, a dynamic-array discovered-reference representation, and a specialised clear path.
2. A combinatorial evaluation of all eight combinations of these three optimisations against a common unmodified baseline, isolating individual and joint effects on GC timing and memory usage.
3. An exploratory prototype implementation of a `@weak` field annotation, enabling weak semantics to be expressed directly on object fields without a `WeakReference` wrapper, evaluated under the same benchmarking framework.
4. Empirical evidence from 250 controlled runs per variant on a supercomputer cluster, quantifying the GC timing and memory trade-offs of each approach under two synthetic benchmarks designed to stress the reference-processing pipeline.

1.4 Scope and Delimitations

This thesis focuses on the reference processing pipeline within ZGC, specifically targeting weak references without a `ReferenceQueue`. The primary implementation work concerns `WeakReference`-based processing, with weak fields treated as an extension evaluated in the same benchmarking framework. Although the optimisation approaches are described in general terms, the implementation and evaluation remain ZGC-specific. Furthermore, some of the optimisations, such as the optimised clear path, may have broader applicability beyond weak references without a `ReferenceQueue`. However, those broader applications fall outside the scope of this thesis and are not evaluated.

The correctness of the optimisations is only verified through the standard OpenJDK test suite and manual inspection. Performance measurements are conducted on a single hardware platform under controlled conditions and therefore may not generalise to all environments.

2. Background

To answer the research questions posed in Section 1.2, it is necessary to understand key concepts of the JVM and its HotSpot implementation, GC mechanisms, the Java reference types, the design and implementation of ZGC, and the reference-processing pipeline within ZGC.

2.1 The JVM and HotSpot

The JVM is a language-independent abstract computing machine that provides a managed execution environment for programs compiled to Java bytecode [2]. Rather than targeting a specific physical processor, Java source code is compiled to bytecode – a compact, platform-neutral instruction set – which the JVM then executes on the host machine. This separation allows the same bytecode to run without modification on any platform for which a JVM implementation exists. The JVM is responsible for loading class files, verifying bytecode, managing memory (including automatic GC), and providing the runtime services (threading, synchronisation, reflection) that Java programs rely on.

There are multiple JVM implementations. The reference implementation, developed and maintained by Oracle and the OpenJDK community, is **HotSpot** [16, 17]. HotSpot is the JVM shipped with Oracle JDK and OpenJDK distributions, and it is the implementation targeted by this thesis. Its name reflects a key design principle: rather than interpreting all bytecode uniformly, HotSpot identifies frequently executed *hot spots* and compiles them to native machine code using Just-In-Time (compilation) (JIT) compilation, while less frequently executed code continues to run in the interpreter. This adaptive strategy allows HotSpot to achieve throughput that approaches that of ahead-of-time compiled languages for hot code paths, whilst retaining fast start-up through interpretation.

HotSpot's internal architecture spans the garbage collectors (Serial, Parallel, G1, and ZGC), the JIT compilers (C1 and C2), the template interpreter, the class loader, and the object memory model [16]. These components interact closely: the GCr depends on the compiler and interpreter to emit barriers at every heap access, the JIT depends on the GCr to provide runtime type and liveness information, and the class loader and memory model together define how objects are laid out in the heap.

All implementation details discussed in this thesis – the execution tiers, the barrier mechanism, the object representation, and the reference-processing pipeline – are specific to HotSpot. Other JVM implementations may differ substantially in their internal organisation, even if they conform to the same JVM specification.

2.2 GC in the JVM

The JVM provides automatic memory management through GC. Rather than requiring the programmer to explicitly allocate and free memory as in languages such as C or C++, the JVM tracks which objects are reachable from the application's root set and reclaims the memory of objects that are no longer reachable. The root set consists of all data that is directly accessible to the application, such as local variables on the stack, static fields, and active threads.

HotSpot ships with several GCr implementations, each designed with different trade-offs between throughput, latency, and memory footprint [1]. The collectors are:

- **Serial GCr** A single-threaded collector that performs all GC work using one thread, avoiding inter-thread communication overhead. It is best suited to single-processor systems and small heaps.
- **Parallel GCr** A throughput-oriented generational collector that uses multiple GC threads to speed up collection on multiprocessor systems. It is intended for medium to large data sets.
- **G1 GCr (Garbage-First)** A mostly concurrent, region-based collector designed to scale from small to large multiprocessor machines. It targets predictable pause times with high probability while maintaining high throughput and is the default collector on most platforms [1].
- **ZGC** A scalable, low-latency collector that performs expensive GC work concurrently with application threads. It targets sub-millisecond pause times (largely independent of heap size) and supports heap sizes from 8 MB to 16 TB [3, 6, 4, 5].

This thesis focuses on ZGC, as its low-latency design and concurrent reference processing pipeline can particularly benefit from optimisations that reduce the amount of work done during reference processing.

2.2.1 GC Phases

GC generally occurs multiple times (cycles) during the lifetime of a Java application. Each cycle consists of several distinct phases, each with specific responsibilities. The exact phases and their order can vary between collectors, but a common structure is as follows:

1. **Mark Phase** The collector identifies all live objects by traversing from the root set and marking all reachable objects. Dead objects are those that are not marked. This phase may be performed concurrently with the application or during a Stop-The-World (STW) pause, depending on the collector design.
2. **Reference Processing Phase** After marking completes, the collector processes *Reference* objects discovered during the mark phase. This phase determines which references have dead referents and prepares them for enqueueing. Like marking, this phase may be concurrent or done during a pause.
3. **Sweep/Reclamation Phase** The collector reclaims memory occupied by dead objects. Some collectors use a sweep algorithm (scanning the heap and freeing unmarked objects) whilst others use evacuation (copying live objects to new memory regions).
4. **Compaction/Relocation Phase** Some collectors compact the heap to eliminate fragmentation, moving live objects to consolidate free space. Others relocate objects incrementally as part of the collection process.

The order and concurrency model of these phases varies significantly between collectors. Serial and Parallel GCr typically perform all phases in STW pauses [1]. G1 performs marking concurrently with the application, which reduces STW pause times, though evacuation (copying) of live objects is still performed during STW pauses [1]. ZGC performs marking, reference processing, and relocation concurrently, using barriers to maintain consistency with concurrent application threads [3, 4].

2.2.2 GC Barriers

In JVM GCrs, a *barrier* is a small piece of code inserted around memory accesses so that the collector can maintain correctness while the application is running [2]. The term is slightly misleading: barriers are not global locks and do not stop threads. Instead, they are local checks or bookkeeping actions attached to individual loads and stores of object references [2, 3, 18].

Barriers are especially important in concurrent collectors because GC and application threads observe and modify heap state at the same time. Rather than pausing the application for long periods, the collector uses barriers to spread consistency work across ordinary execution [3, 2].

Two broad classes are common:

- **Load (read) barriers**, executed when a reference is read. These can validate or transform the loaded pointer before it is used by the application [3, 18].
- **Store (write) barriers**, executed when a reference field is overwritten. These record metadata needed by the collector, for example to preserve marking invariants or to track inter-generational pointers [2, 4].

2.2.3 Generational Collection

In current Oracle JDK releases, all four supported JVM GCrs (Serial, Parallel, G1, and Z) use *generational collection*, based on the *weak generational hypothesis*: most objects die young, and those that survive tend to remain live for a long time [19]. To exploit this hypothesis, the heap is divided into at least two generations:

- The **young generation**, where newly allocated objects reside. Young generation collections are frequent but fast, since most young objects are short-lived.
- The **old generation**, where objects that have survived multiple young collections are promoted to. Old generation collections are less frequent but more expensive since live objects are more plentiful here.

ZGC adopted generational collection starting with JDK 21 [4]. In generational ZGC, the young and old generations can be collected independently and therefore concurrently. This is relevant to reference processing because only old-generation references are discovered and processed through the full pipeline; young-generation references are treated as strongly reachable and are not subject to reference processing, as discussed further in Section 2.6.1.

2.3 Java Reference Types

In addition to ordinary (strong) references, Java provides *reference objects* in the `java.lang.ref` package [7, 9] that allow applications to reference objects without preventing their GC. These reference types are:

1. **SoftReference** The referent field is cleared at the discretion of the GCr, typically when memory is low. Commonly used for memory-sensitive caches.
2. **WeakReference** The referent field is cleared as soon as it becomes weakly reachable (i.e., no strong or soft references exist to it). Widely used in caching frameworks, canonical mappings, and listener registrations.
3. **FinalReference** An internal JVM type (not part of the public API) used to implement the `finalize()` mechanism. Objects with finalisers are wrapped in a `FinalReference` so that the GCr can schedule finalisation before reclaiming the object. This scheduling is done via a `ReferenceQueue`, which means that `FinalReferences` always have an associated `ReferenceQueue`.
4. **PhantomReference** The referent is never accessible through the reference, `get()` always returns `null`. Used for post-mortem cleanup actions, often to avoid the pitfalls of finalisation. Phantom references must be registered with a `ReferenceQueue` to be useful.

All four types extend the abstract class `Reference<T>` (in the `java.lang.ref` package), which defines the key fields of the reference object and the API for retrieving the referent, clearing the referent field, and checking if it is enqueued.

2.3.1 The Reference Object Lifecycle

Each `Reference` object maintains several fields that track its state [9]:

- `referent field` The referenced object. The GCr may clear this field (set it to `null`) when the referent becomes eligible for collection according to the reference's type.

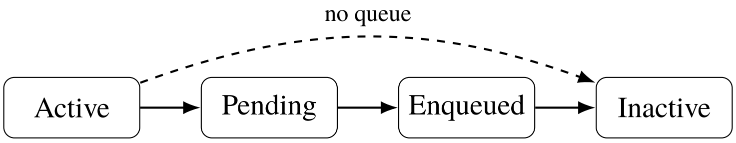

Figure 2.1. Lifecycle of `Reference` objects. References without a queue can bypass Pending and Enqueued.

- `queue field` The `ReferenceQueue` this reference is registered with. If the reference was not created with a queue, this field points to the sentinel object `ReferenceQueue.NULL_QUEUE`.
- `next field` Used as a link pointer when the reference is enqueued in a `ReferenceQueue`.
- `discovered field` A field with dual purpose: during GC marking, it serves as a link in the GCr's internal *discovered list*; after processing, it is reused as a link in the VM's *pending list* that feeds the `ReferenceHandler` thread. Because the field is used as a link in the discovered list, the GCr can detect whether a reference has already been discovered by checking whether this field is `null`.

A reference object transitions through the following states during its lifetime [7, 9]:

1. **Active** The reference has been created and the referent is still reachable (or not yet processed).
2. **Pending** The GCr has determined the referent is eligible for clearing and has added the reference to the VM's pending list. The `ReferenceHandler` thread will process it.
3. **Enqueued** The `ReferenceHandler` thread has moved the reference into its associated `ReferenceQueue`, where the application can retrieve it via `poll()` or `remove()`.
4. **Inactive** The reference has been dequeued by the application, or was created without a queue and has been cleared. It is no longer tracked by the GCr or queuing infrastructure.

An important observation is that references created *without* a `ReferenceQueue` can transition directly from Active to Inactive, bypassing the Pending and Enqueued states entirely. However, as discussed in Section 2.6, the default implementation does not take advantage of this shortcut. The lifecycle and this direct shortcut path are illustrated in Figure 2.1.

2.3.2 WeakReference and ReferenceQueue

A `WeakReference` provides two constructors (with and without a `ReferenceQueue`), as shown in Listing 2.1 and implemented in OpenJDK [8].

```
// Without a queue - queue field set to NULL_QUEUE
public WeakReference(T referent) {
    super(referent),
}

// With a queue
public WeakReference(T referent, ReferenceQueue<? super T> q) {
    super(referent, q),
}
```

Listing 2.1: *WeakReference constructors*

When a `WeakReference` is created with a `ReferenceQueue`, the application can be notified when the referent is collected. When created *without* a queue, the reference serves only as a conditional pointer: the application can call `get()` to check whether the referent is still alive, but receives no notification upon clearing.

The `ReferenceQueue` is implemented as a synchronised singly-linked list using the `next` field of the `Reference` objects. It provides `poll()` for non-blocking retrieval and `remove()`

for blocking retrieval with an optional timeout [10]. This is the mechanism by which applications can react to the collection of referents, for example to clean up associated resources.

A key insight for this thesis is that Java explicitly supports queue-less weak references through the one-argument `WeakReference` constructor [8]. For workloads that only need liveness probing via `get()`, queue registration is unnecessary. In such cases, the VM still performs enqueue-path preparation work in the default pipeline, which motivated JDK-8029205 and the optimisation work in this thesis [13, 9, 12].

2.4 HotSpot Internals

To understand how GC barriers are injected and why ZGC requires different components for each execution tier, it is helpful to examine HotSpot's tiered execution model, which balances start-up latency against sustained throughput. Three execution tiers are relevant to this thesis:

1. **Interpreter.** At start-up, methods are first executed by the template interpreter, in which each bytecode is handled by generated assembly stubs. Interpretation provides the lowest throughput of the three tiers, but it has no JIT compilation overhead and it gathers profiling data for later optimisation.
2. **C1 Compiler.** C1 is a fast JIT compiler that applies lightweight optimisations such as inlining, constant folding, and local simplifications. It compiles quickly, which makes it suitable for methods that are executed often enough to justify compilation but are not yet the hottest parts of the program.
3. **C2 Compiler.** C2 is the JVM's aggressively optimising JIT compiler. It uses profiling data collected in earlier tiers to emit highly optimised native code for hot methods, applying transformations such as aggressive inlining, loop optimisations, and escape analysis.

In tiered compilation, methods move through these tiers as their execution frequency increases. A method typically starts in the interpreter, may then be compiled by C1, and can eventually be recompiled by C2 once sufficient profiling data has been collected.

Relevance to GC barriers.
Because a read barrier must intercept every load from the heap, and such loads occur in all three tiers, each tier requires its own barrier implementation. ZGC therefore ships three separate barrier components: `ZBarrierSetAssembler` generates barrier stubs embedded in the interpreter's dispatch table, `ZBarrierSetC1` inserts barrier nodes during C1 intermediate representation construction, and `ZBarrierSetC2` integrates barriers into the C2 sea-of-nodes graph [18]. All three components obey the same decorator-driven barrier selection logic described in Section 2.5.3.

2.5 The ZGC

ZGC is a concurrent, region-based, compacting GCr included in the JDK since JDK 11 (experimental) [3] and production-ready since JDK 15 [6]. Starting with JDK 21, ZGC supports generational collection, which is now the default mode [4]. The collector targets sub-millisecond pause times that remain bounded regardless of heap size, and supports heaps ranging from a few hundred megabytes to several terabytes [3, 4, 5]. Achieving these goals while performing the bulk of GC work concurrently is partly realised by two closely related design decisions: encoding GCr state directly in heap pointers, and using barriers to act on that state at every pointer access.

2.5.1 Coloured Pointers and Barriers

ZGC embeds metadata directly in heap pointers as *colour bits*. In the original non-generational design, these bits occupied unused high bits of 64-bit pointers [3]. Generational ZGC moved the colour bits to the low-order bits of the pointer to accommodate the richer set of states needed to track generation membership alongside relocation status [4, 20]. In both designs the colour bits encode the state of a pointer relative to the current GC phase – for example, whether the pointed-to object has been remapped after relocation, or which generation it belongs to. Because the state is carried in the pointer itself, the GCr can determine the status of a reference without loading the object's header or consulting external tables.

To act on this encoded state, ZGC inserts barriers at every oop load and store. On a load, the barrier reads the colour bits of the retrieved pointer and checks whether they encode a *good* state for the current GC phase. If the pointer is stale – for example because the object has been relocated since the pointer was written – the barrier's slow path applies the necessary fixup (remapping the pointer to the new address) before returning the address to the caller [3, 18]. Because these fixups happen lazily on access rather than eagerly during relocation, the collector can relocate objects concurrently without stalling the application.

Store barriers serve a complementary role: before overwriting a heap field, the barrier records the field's previous value in a per-thread store-barrier buffer. This record is consumed by the marking phase to satisfy ZGC's Snapshot-At-The-Beginning (SATB) invariant – ensuring that objects that were reachable at the start of a marking cycle are not missed – and by the remembered-set mechanism that tracks cross-generational pointers in the generational case [18].

2.5.2 Concurrent Phases and Parallel Workers

A ZGC collection cycle interleaves short STW pauses, used for root scanning and phase synchronisation, with longer concurrent phases in which marking, reference processing, and relocation proceed alongside running application threads [3, 12]. The load and store barriers described above are the mechanism that keeps the heap consistent during these concurrent phases: application threads that load a stale pointer automatically fix it up, and stores that would break the marking invariant are recorded for GCr consumption.

Within each phase, ZGC distributes its work across multiple GC worker threads. To minimise contention, workers maintain their own thread-local data structures – such as per-worker discovered lists during reference discovery – and merge results into shared state only at well-defined synchronisation points [21, 12]. This design avoids the Compare-And-Swap (CAS) contention that would arise from multiple threads updating a single shared resource.

2.5.3 Access Decorators

Throughout the HotSpot codebase, every load or store is annotated with a *decorator set*: a compile-time or runtime bitmask of type `DecoratorSet` (a 64-bit integer) whose set bits describe the semantics of the access [22]. The active `BarrierSet` implementation (e.g. `ZBarrierSet`) inspects this bitmask to decide which barrier variant to invoke, allowing a single generic `HeapAccess<decorators>::load()` call site to resolve to the correct low-level operation at both compile time (for template-specialised paths) and runtime (for dynamically dispatched paths).

The decorator flags are grouped by concern:

- **Reference strength** bits control how the barrier treats reachability. An access with `ON_WEAK_OOP_REF` tells ZGC that the field being read is the `referent` of a `WeakReference`: the barrier must return `null` if the referent has been collected, rather than reviving it. The default, when no strength decorator is specified, is `ON_STRONG_OOP_REF`.
- **Access location** bits indicate whether the access targets a heap object field or an off-heap native data structure. Some barrier actions are conditioned on `IN_HEAP`.
- **Barrier strength** bits control how strong barriers should be applied. `AS_RAW` bypasses GC barriers entirely and reads the raw memory value, which is necessary in certain internal GCr paths. `AS_NO_KEEPALIVE` performs a correctly remapped access without keeping any objects alive, used in GCr-internal scanning paths. `AS_NORMAL` (the default) invokes the full barrier.

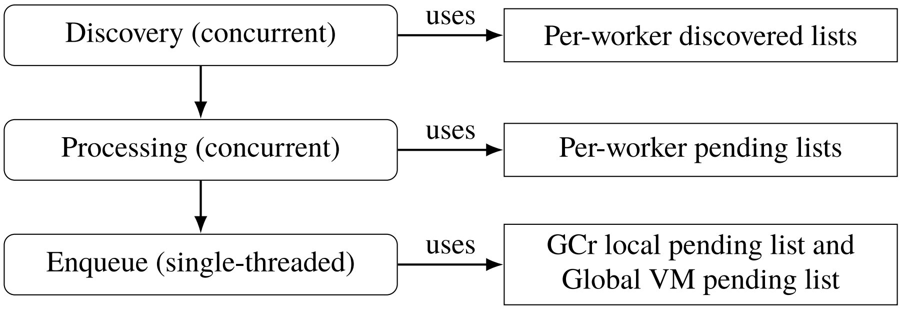

Figure 2.2. ZGC reference-processing pipeline and data flow from discovery to Java-level enqueue handling.

- **Memory ordering** bits map to C++ memory-order semantics for the underlying atomic or plain load or store.

For ZGC, when `WeakReference.get()` is executed the `referent` field load carries `ON_WEAK_OOP_REF`. ZGC's barrier for this decorator checks the colour bits of the loaded pointer and if the pointer is already good (the referent is live and remapped or null), the address is returned directly. If not, the barrier's slow path is invoked. In the slow path, the liveness of the referent is checked and the barrier either returns a coloured null if the referent is no longer live, or a correctly coloured pointer if the referent is live [18, 22].

Store barriers are similarly decorator-driven. A normal store carries the decorator `ON_STRONG_OOP_REF` which triggers ZGC's store barrier, which records the old value in the store-barrier buffer (needed for the SATB marking semantics and for maintaining remembered sets in the generational case).

2.6 Reference Processing Pipeline in ZGC

Reference processing is the mechanism by which the GCr handles `Reference` objects discovered during marking. The pipeline has three stages: *discovery*, *processing*, and *enqueueing*. In ZGC, the first two stages run concurrently with the application during the marking phase, while the enqueue stage involves handing off references to the Java-level `ReferenceHandler` thread [12, 9]. An overview of this data flow is shown in Figure 2.2.

2.6.1 Discovery

During the concurrent marking phase, when ZGC encounters a `Reference` object (an instance of `SoftReference`, `WeakReference`, `FinalReference`, or `PhantomReference`), it evaluates whether the reference should be *discovered*, i.e. added to a list for later processing.

The discovery decision depends on several factors:

1. **Is the reference active?** Only active references are eligible. For non-`FinalReference` types, an inactive reference has a `null` referent field. For `FinalReference`, inactivity is indicated by a non-null `next` field.
2. **Is the referent still strongly reachable?** If the referent is already marked as strongly live, the reference does not need processing. The GCr checks this via its liveness information.
3. **Soft reference policy.** For `SoftReference` objects, the collector applies a policy (based on available memory and the reference's last access time) to decide whether to keep the referent alive or clear the referent field [14, 23].
4. **Generational considerations.** In ZGC, only references residing in the old generation are discovered, young-generation references are always treated as strong [4, 12]. This is because reference processing introduces additional delay and uncertainty to the duration of the young-generation collection. By treating young references as strong, ZGC can complete

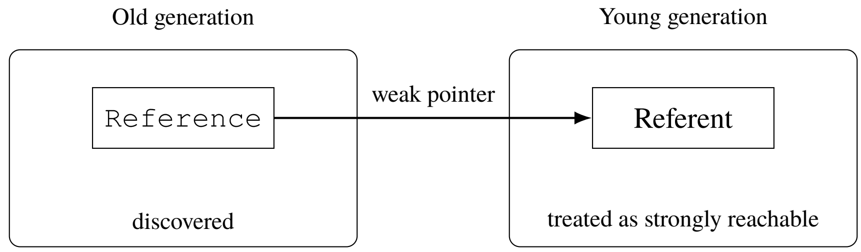

Figure 2.3. If an old-generation `Reference` points to a young referent, the young referent must remain alive during young collection.

young collections more predictably, which decreases the risk of the application running out of memory during a long young collection.

Once a reference is selected for discovery, it is recorded on a *discovered list*. In ZGC, each GC worker thread maintains its own discovered-list head, so discovery is implemented as a cheap, non-atomic prepend to a per-worker list [12]. This avoids the CAS contention that would arise if multiple threads updated the same shared discovered-list head [14].

2.6.2 Processing

After marking completes, the discovered lists are processed. For each discovered reference, the collector determines the final fate of the referent:

1. If the referent field has become `null` (cleared by the application concurrently), the reference is dropped from the list.
2. If the referent is now strongly reachable (e.g. it was marked as alive after the reference's discovery), the reference is dropped.
3. If the referent is still unreachable according to the reference's strength, the referent field is **cleared** (set to `null`), and the reference is prepared for enqueueing.

In ZGC, clearing involves setting the `referent` field to a coloured null pointer via CAS to ensure thread safety since application threads may be concurrently accessing the reference [18, 12].

After clearing, the reference is linked onto the thread-local pending list. This involves writing the reference's address into the `discovered` field of the previously processed reference. Once the thread has processed all references in its discovered list, the thread-local list is atomically prepended to ZGC's internal pending list [12].

Generational interactions are important during processing: when a `Reference` object is old but its referent is young, the referent is treated as strongly reachable and the referent field must not be cleared. Processing must therefore verify generation membership and, if appropriate, ensure young-generation liveness is preserved (by marking the young referent), as illustrated in Figure 2.3 [12, 24].

2.6.3 Enqueue

After a reference's referent field has been cleared, the reference must be made available to the application if it was created with a `ReferenceQueue`. This happens through the *pending list* mechanism which feeds the `ReferenceHandler` thread [12, 9]. The handoff process is as follows:

1. The GCr has built an internal pending list by linking cleared references together via their `discovered` field.
2. Under the heap lock, the GCr atomically prepends its internal pending list onto the global VM pending list.
3. The `ReferenceHandler` daemon thread is notified.
4. The `ReferenceHandler` thread wakes up, atomically retrieves the pending list, and iterates over it. For each reference, it checks whether `queue != NULL_QUEUE`. If the

reference has a real queue, it calls `queue.enqueue(this)`, otherwise it simply advances to the next reference.

This handoff reveals an inefficiency: references created without a `ReferenceQueue` are still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be skipped [9, 12]. For workloads where the majority of weak references lack queues, this results in a considerable amount of extra work. This behaviour has been reported in an OpenJDK enhancement issue [13]. Avoiding enqueue-path work for references with `NULL_QUEUE` is one of the primary optimisation targets explored in this thesis.

2.6.4 Trade-offs of the Three Pipeline Optimisations

The three optimisations investigated in this thesis each address a specific inefficiency in the pipeline described above. This section surveys the expected benefits and limitations of each, providing the conceptual groundwork for the design decisions documented in Chapter 3. Understanding these trade-offs is necessary to interpret the experimental results and to assess whether the optimisations merit integration. Each optimisation is analysed below.

Skip-enqueue separation.
Benefits. Queue-less weak references currently traverse the entire enqueue path (pending-list linking, heap-lock handoff, `ReferenceHandler` thread iteration) only to be skipped at the final check in step 4 above [9, 13]. Routing them to a separate discovered list eliminates all of this work. The pending-list lock acquisition and the `ReferenceHandler` thread wakeup both disappear for the queue-less subset. In mixed workloads containing queue-backed references, the separation also reduces the length of the pending list that the `ReferenceHandler` must traverse, lowering latency for queue-backed notifications [9, 12].

Limitations. The benefit is only observable for references without a `ReferenceQueue`. In workloads dominated by queue-backed references, the separation adds a branch during discovery with no corresponding saving [12]. Furthermore, the enqueue stage itself involves relatively coarse-grained operations (one heap-lock acquisition and one thread notification per GC cycle, not per reference), so the cost amortises over large reference populations and may represent a small fraction of total reference-processing time [9]. A second discovered list also increases code complexity and slightly enlarges the per-worker GC data structures.

Dynamic array.
Benefits. The intrusive linked list used in the baseline stores each reference's successor address in the reference object's `discovered` field. Successive references in the discovered list may be separated by large heap distances, causing each traversal step to generate a cache miss [15]. Storing discovered references in a contiguous dynamic array eliminates this pointer-chasing pattern: sequential iteration accesses memory in a predictable, stride-1 pattern that is friendly to hardware prefetchers and allows multiple references to share a single cache line [15, 25]. The array also enables pre-loading field addresses and values during discovery and storing them inline in each entry, so the processing loop reads only the contiguous array rather than issuing additional heap loads.

Limitations. The intrusive linked list is essentially free in terms of auxiliary GCr memory: it reuses the `discovered` field already allocated as part of each `Reference` object [9]. The dynamic array requires a separately allocated contiguous buffer on the C heap, proportional to the number of discovered references. For variants that pre-load field data, each entry is larger still, increasing the buffer footprint further. This auxiliary GCr memory overhead may represent a significant fraction of GCr-committed memory in workloads with large reference populations (see Section 5.3). The array also introduces management overhead: capacity must be tracked and the array must grow when a new cycle discovers more references than the previous reservation [12]. An additional trade-off exists in the choice of whether to return the array memory to the system after each GC cycle or to retain it for the next: returning the memory reduces peak footprint between cycles but reintroduces allocation costs on the next discovery phase, while retaining it amortises allocation across cycles at the expense of sustained elevated memory usage.

Specialised clear path.
Benefits. The standard ZGC clear path executes three operations per reference: a load barrier to resolve the referent pointer, a virtual call to dispatch on reference type, and a CAS via a ZGC barrier to atomically clear the referent field [12, 18]. For queue-less old-generation `WeakReference` objects, all three can be simplified. The load barrier is unnecessary when the referent was pre-loaded during discovery. The virtual call can be replaced by a direct call since the reference type is statically `REF_WEAK`. Most importantly, the CAS can be replaced by a direct store: the only concurrent operation that can modify the referent field is an application thread explicitly calling `WeakReference.clear()`, which sets it to null [9]. Since no thread can set a null field to a non-null value, overwriting a non-null referent with a coloured null is safe with a plain store. Together these reductions substantially lower instruction count and memory-access overhead per reference [18, 12].

Limitations. The correctness argument for replacing the CAS with a direct store is non-trivial and tightly coupled to the current `WeakReference` API contract [9, 18]. Any future change to the concurrency model of `Reference` objects could invalidate the assumption. The optimisation is therefore limited to old-generation `WeakReference` objects without a `ReferenceQueue`: extending it to `SoftReference` or `PhantomReference` objects, or to `WeakReference` objects with queues, requires re-examining the concurrency argument for each type. Additionally, when used without the dynamic array (the `clear_path_only` variant), the referent must still be loaded through a barrier at processing time, since no pre-loading occurred during discovery, the saving is then limited to eliminating the virtual call and replacing the CAS with a store.

Weak-field mechanism.
Benefits. The fundamental overhead of `WeakReference` objects is structural: each weak-reference relationship requires a separate `WeakReference` heap object, which must be allocated, discovered by the GCr, processed through the clearing pipeline, and eventually collected [8, 12]. In applications that use weak references purely for liveness probing – without a `ReferenceQueue` – all of this object-level machinery exists solely to carry the weak semantics of a single field pointer. Expressing those semantics directly on a field annotation removes the `WeakReference` wrapper entirely: no object is allocated, no object appears in the discovered list, and the GCr processes the field address directly. This should reduce per-GC-cycle cost on two fronts. First, reference-processing overhead is reduced: the GCr processes field addresses rather than `WeakReference` objects, avoiding the object-header indirection and the enqueue path entirely. Second, heap allocation pressure is reduced: fewer live objects mean shorter GC pause times and a lower memory footprint, since GC cost is broadly proportional to the number of live objects [2].

Limitations. The mechanism requires changes in multiple JVM layers – annotation parsing, class-file metadata, compiler and interpreter barrier emission, and the GCr-side marking closure – making it substantially more intrusive than the three pipeline optimisations. The current implementation supports only old-generation fields, weak fields in young-generation objects fall back to strong-reference treatment. There is no `ReferenceQueue` integration: applications that need post-clear notification must continue to use `WeakReference` objects. The annotation is non-standard and not part of any Java specification, which limits its applicability to HotSpot deployments. Finally, the standard `Reference` API (`get()`, `clear()`, `isEnqueued()`) is unavailable for weak fields, callers check for null directly on the field.

3. Design and Implementation

The motivation and trade-offs for each technique are established in Section 2.6.4; this chapter describes their design and implementation. Three optimisations target ZGC's processing of `WeakReference` objects without a `ReferenceQueue`: skip-enqueue separation (Section 3.1), the dynamic array (Section 3.2), and the specialised clear path (Section 3.3). In addition, an exploratory weak-field implementation (Section 3.4) was developed to evaluate an alternative way of expressing weak semantics that avoids the overhead of `WeakReference` objects entirely.

Table 3.1 provides a compact overview of how the four mechanisms differ across the key design dimensions, meanwhile Table 3.2 summarises the expected costs and benefits of each mechanism.

3.1 Skipping Enqueue Path

As established in Section 2.6.4, skip-enqueue separation eliminates pending-list work for queue-less weak references by routing them to a separate discovered list that bypasses the enqueue stage entirely.

3.1.1 Implementation of Skipping Enqueue Path

In order to skip enqueueing weak references without a `ReferenceQueue` the `ZReference-Processor` class was modified to check if the queue field of a weak reference being discovered is equal to the sentinel value `ReferenceQueue.NULL_QUEUE` before adding it to the discovered list. If the queue field is equal to `ReferenceQueue.NULL_QUEUE` (the weak reference was not created with a queue), it is added to a separate discovered list for weak references without a `ReferenceQueue` instead of being added to the regular discovered list. This separate discovered list is processed independently by the GC threads and none of its weak references are enqueued to the pending list for the `ReferenceHandler` thread. Each GC thread maintains its own separate discovered list for weak references without a `ReferenceQueue` to avoid contention between threads. See Listing 3.1 for a pseudocode summary and Appendix E.2 for the actual implementation.

The discovered lists are separated rather than handled through conditional branches in a single processing loop so that two specialised reference-processing functions can be used: one for queue-backed references, which includes enqueue handling, and one for queue-less references, which omits it. This allows the queue-less path to avoid unnecessary per-reference checks during processing and keeps the no-enqueue path structurally simpler.

3.2 Dynamic Array for Discovered References

As established in Section 2.6.4, replacing the intrusive linked list with a contiguous dynamic array improves spatial locality during the processing traversal [15, 25]. The array is used for weak references without a `ReferenceQueue`, and it also enables pre-loading field data during discovery.

3.2.1 Implementation of Dynamic Array for Discovered References

The implemented dynamic array, called `ZWeakRefArray`, stores discovered references in a contiguous layout rather than linking them via the `discovered` field. The array is allocated on the C heap and grows dynamically as needed.

Table 3.1. *Comparison of the four mechanisms along key design dimensions. The representation column describes how weak semantics are expressed. The discovery structure column describes the data structure used during the concurrent mark phase to record discovered references. The enqueue path column describes whether discovered references are enqueued to the pending list for the* `ReferenceHandler` *thread. The clear path column describes the operations performed to clear the referent field of an unreachable reference during processing.*

Mechanism	**Representation**	**Discovery structure**	**Enqueue path**	**Clear path**
Baseline (`none`)	Independent objects	Intrusive linked list	Pending list	Load barrier + virtual call + CAS
Skip-enqueue sep. (`sep`)	Independent objects	Separate linked lists per list type	Queue-less refs bypass pending list	Load barrier + virtual call + CAS
Dynamic array (`dyn`)	Independent objects	Contiguous array	Pending list	Load barrier + virtual call + CAS
Clear path (`clear_path`)	Independent objects	Intrusive linked list	Pending list	Pre-loaded data + direct call + plain store
Weak fields (`weak_fields`)	Annotated object fields	Contiguous array per GC worker	No pending list	Pre-loaded data + direct call + CAS

Each GC worker thread maintains its own `ZWeakRefArray` instance to avoid contention. During discovery, when a reference should be added to a discovery list, its data is appended to the worker's array instead of being appended to a linked list. The reference's `discovered` field is set to a self-referencing pointer to mark the reference as discovered, preventing duplicate discovery.

During processing, the array is iterated sequentially using index-based access. Because the elements are stored contiguously in memory, this iteration improves cache line utilisation compared to following pointers scattered across the heap [15]. After processing, the array is cleared for the next GC cycle but retains enough capacity to at least hold the same number of references that were processed but not made inactive. This avoids unnecessary reallocations in subsequent cycles when the number of discovered references is relatively stable.

See Listing 3.2 for a pseudocode summary and Appendices E.1 and E.2 for the actual implementation.

3.3 Optimised Clear Path

As established in Section 2.6.4, the standard ZGC clear path performs three operations per reference that can be simplified or eliminated for queue-less old-generation `WeakReference` objects [12, 18]:

1. A load barrier to read the referent from the reference object.
2. A virtual call to determine the reference type (soft, weak, final, or phantom).
3. A CAS operation via a ZGC barrier to atomically set the referent field to a coloured null pointer.

For weak references without a `ReferenceQueue`, each of these operations can be simplified or eliminated:

1. When combined with the dynamic array optimisation, the referent field address and its value can be pre-loaded during discovery and stored in the array entry. This avoids redundant memory loads during processing. Pre-loading is safe because the only concurrent application-level operations on a `Reference`'s referent are clearing and enqueueing, both of which set the referent field to null rather than to a new non-null value [9, 12].
2. The reference type is known to be `REF_WEAK` at the call site, eliminating the need for a virtual call to determine the type.

Table 3.2. *Comparison of the expected costs and benefits of the four mechanisms. The discovery structure column describes the data structure used during the concurrent mark phase to record discovered references. "Clear path" describes how the referent field is cleared once the reference is found to be unreachable.*

Mechanism	**Expected benefit**	**Expected cost**
Skip-enqueue sep. (`sep`)	Shorter reference-processing time and less work for the `ReferenceHandler` thread for queue-less weak references	Increased code complexity and slightly larger code objects
Dynamic array (`dyn`)	Shorter reference-processing time due to better cache locality and pre-loaded field data	Higher memory footprint and longer discovery time due to dynamic array management
Clear path (`clear_path`)	Shorter reference-processing time due to fewer and simpler operations for clearing the referent field	Increased code complexity and potential for subtle bugs in future development
Weak fields (`weak_fields`)	Shorter total GC cycle time and lower memory footprint due to avoiding `WeakReference` objects entirely	Immensely increased code complexity and an API-level separation between weak semantics with and without callbacks

3. The only concurrent modification to the referent field is an application thread setting it to null [9, 18, 12]. This means no thread can set the field to a new non-null value, which means that the CAS operation for clearing the referent can be replaced with a simple store of a coloured null pointer.

These simplifications reduce both instruction count and memory-access overhead for each weak reference without a `ReferenceQueue` during processing, which should in turn reduce reference-processing time and GC cycle time.

3.3.1 Implementation of Optimised Clear Path

The implementation introduces a new function for determining whether a queue-less weak reference's referent should be cleared (see Appendix E.2). This function takes a data struct (either from the dynamic array or constructed on the fly from the linked list) containing the pre-loaded field addresses and values. This new function has similar semantics to the standard function and the ZGC barriers it uses, but it is specialised for the queue-less weak reference case and can therefore use more efficient operations. See Section 2.5 for more details on ZGC barriers.

The new function checks whether the referent is strongly live via metadata in ZGC's heap management. If the referent is not strongly live and resides in the old generation, its referent field is cleared using a direct pointer store rather than a CAS. This is safe under the constrained update model described above. If the referent is still live, either because it is strongly marked or because it resides in the young generation, the referent field is recoloured with the correct pointer metadata bits. That recolouring has to use CAS so that a null value is not overwritten with a non-null value. Additionally, if the referent is young and the young generation is currently in its mark phase, the referent is marked to preserve cross-generational liveness [12, 18, 24].

The `discovered` field is also cleared using a direct store to the field address rather than through an atomic access function, since no other thread modifies this field after discovery in this processing path [12].

See Listing 3.3 for a pseudocode summary and Appendix E.2 for the actual implementation.

```
bool discover_reference(oop ref_obj, ReferenceType type) {
    zaddress ref = to_zaddress(ref_obj), // ZGC-coloured pointer

    if (!should_discover(ref, type)) {   // skip if already discovered
        or ineligible
        return false,
    }

    if (type == REF_WEAK && !has_reference_queue(ref)) {
        // Queue-less weak ref: bypasses pending list and
        // ReferenceHandler thread entirely
        weak_no_queue_list_per_worker.append(ref),
    } else {
        // All other types (and queue-backed weak refs) follow
        // the standard discovered-list and enqueue pipeline
        discovered_list_per_worker.append(ref),
    }

    mark_discovered(ref), // set discovered field to prevent re-
        discovery
    return true,
}
```

Listing 3.1: *Pseudocode of the null-queue fast path during discovery. Queue-less weak references are routed to a separate discovered list and processed by a no-enqueue pipeline (see Appendix E.2).*

3.4 Weak Fields

As established in Section 2.6.4, replacing `WeakReference` wrapper objects with annotated fields should reduce both reference-processing overhead and heap allocation pressure. To answer Research Question 2, an exploratory weak-field mechanism was implemented. The mechanism allows developers to annotate object fields as weak, indicating that the GCr should treat those fields with weak semantics. The GCr then applies weak semantic rules directly to the annotated field without requiring a separate `WeakReference` object.

3.4.1 Implementation of Weak Fields

The implementation includes changes in three layers of the JVM: a Java-level annotation, GCr-side field discovery and processing, and compiler/interpreter support for the annotation metadata.

Annotation

A new runtime-retained field annotation, `@java.lang.ref.weak`, was added to the `java.lang.ref` package. The annotation targets only fields and is retained at runtime so that the VM can inspect it during class loading. See Listing 3.4 for the annotation definition. During class-file parsing, the VM recognises the `@weak` annotation and sets a `weak` flag in the field's metadata.

Discovery

Weak-field discovery occurs during ZGC's old-generation marking phase. As the marking closure iterates over each object's reference fields, it checks whether the current field carries the `weak` flag in its metadata. If the field is annotated as weak, the marking closure diverts it to the weak-field discovery path instead of applying the normal strong mark barrier [12].

The discovery function first checks whether the field should be discovered at all. Fields that already point to null or to a strongly live object (identified by their ZGC pointer metadata

```
void discover_reference(oop ref_obj, ReferenceType type) {
    //...
    ZWeakRefData data,
    data.set_from_ref(ref),     // pre-load referent address and value
    worker_array.append(data), // O(1) amortised; array grows if needed
    //...
}

void process_queue_less_weak_refs(ZWeakRefArray& arr) {
    size_t dropped = 0,
    for (size_t i = 0, i < arr.length(), i++) {
        const ZWeakRefData& data = arr.at(i), // sequential: cache-friendly
        data.mark_not_discovered(),            // clear discovered field first

        if (try_make_inactive(data)) { // clear referent if unreachable
            cleared_weak_no_queue_count++,
        } else {
            dropped++, // reference still active; keep slot
        }
    }
    arr.clear_and_reserve(dropped), // retain capacity for next cycle
}
```

Listing 3.2: *Pseudocode for the* `ZWeakRefArray`*-backed queue-less weak-reference path. Discovery appends pre-loaded fields and processing scans the array sequentially (see Appendices E.1 and E.2).*

bits) are skipped, since no clearing action will be needed for them. Fields in young-generation objects are also skipped, because young-generation weak-field processing is not supported by this implementation. If the field passes these checks, its address and current value are recorded as a `ZWeakFieldData` struct and appended to the GC worker thread's per-worker `ZWeakFieldArray`. See Listing 3.5 for a pseudocode summary and Appendix E.3.3 for the actual implementation.

Each GC worker thread maintains its own `ZWeakFieldArray` instance to avoid contention. The array is structurally identical to the `ZWeakRefArray` used for queue-less weak references: it stores entries contiguously on the C heap and grows dynamically (see Appendix E.3.2).

Processing

During the reference-processing phase, each GC worker iterates through its `ZWeakFieldArray` and applies a clearing function to each entry. The function uses the data recorded during discovery. If the field still contains the discovered value and the referent is no longer strongly live, the routine clears the field by storing a coloured null pointer. If the field still contains the discovered value but the referent remains live, it recolours the pointer with the correct metadata bits. If the field value has changed since discovery, no action is required, since only a strongly reachable referent could have been stored in the field after discovery. Apart from this extra check for post-discovery field updates, the function follows the same overall strategy as the optimised clear path described in Section 3.3. After processing, the array is cleared but retains enough capacity to hold the entries that were not cleared, following the same capacity-retention strategy as `ZWeakRefArray`. See Listing 3.6 for a pseudocode summary and Appendix E.3.3 for the actual implementation.

Compiler and Interpreter Support

The `@weak` annotation metadata is also made available to the JIT compilers and interpreter. This is necessary to ensure that any loads or stores to a weak field outside GC processing are handled correctly with respect to weak semantics. The implementation adds a decorator to loads and stores that target a field carrying the `weak` flag in its metadata. ZGC then inserts barriers that apply the required semantics to all accesses with this decorator. Without the load decorator, a load from a weak field could return a non-null reference whose referent will be cleared by the GCr. The store decorator is needed so that the GCr does not record the previous value of the field, unlike in the default strong-reference case used to preserve the SATB marking semantics. See Section 2.5.3 for more details on decorators and Appendix E.3.5 for the implementations.

```
bool try_make_inactive_fast(const ZWeakRefData& data) {
    // Array variant: referent address pre-loaded at discovery (no barrier needed)
    zaddress referent = data.referent_addr,

    // Linked-list variant: must load through a barrier to get the live address
    zaddress referent = load_barrier(data.referent_field_addr),

    // Young referents cannot be cleared here: they are kept alive by
    // generational rules and are only processed in a young-gen collection
    if (is_young(referent)) {
        if (young_mark_is_active()) {
            mark_young_root(referent), // preserve cross-generational liveness
        }
        return false, // keep active
    }

    // Old referent is dead: no thread can store a non-null value back, so
    // a plain store is safe (no CAS required)
    if (!is_strongly_live(referent)) {
        *data.referent_field_addr = to_zpointer(zaddress::null),
        return true, // became inactive
    }

    // Referent is still live: update pointer colour bits for the current
    // GC phase, but avoid overwriting a null written by an application thread
    zpointer observed = data.referent_field_value,
    zpointer recoloured = remap_with_good_metadata(referent),
    cas_referent_if_non_null(data.referent_field_addr, observed, recoloured),
    return false,
}
```

Listing 3.3: *Pseudocode for the specialised queue-less weak-reference inactive check. The old-generation clear path uses direct stores, while recolouring uses CAS to avoid null-to-non-null races (see Appendix E.2).*

```
package java.lang.ref,

@Retention(RetentionPolicy.RUNTIME) // visible to the VM at class-loading time
@Target(ElementType.FIELD)          // applies to fields only, not classes or methods
public @interface weak {}
```

Listing 3.4: *The `@weak` annotation. Applied to a field, it instructs the GCr to treat the field value with weak-reference semantics (see Appendix E.3.1).*

```
// Called by the marking closure for every reference field of every live object
void do_oop(oop* p) {
    if (is_weak_annotated_field(p)) {  // check weak flag in field metadata
        if (discover_weak_field(p)) {
            return, // diverted to weak path, skip strong marking
        }
    }
    // ... normal strong marking ...
}

bool discover_weak_field(oop* field) {
    encountered_weak_fields_count++,

    if (is_young(field)) {
        return false, // young-generation weak fields are not supported
    }

    if (is_null(field) || is_strongly_live(field)) {
        return false, // no clearing needed: already null or strongly reachable
    }

    // Record field address and current pointer value for processing
    discovered_weak_fields_count++,
    ZWeakFieldData data(field),
    worker_weak_field_array.append(data), // per-worker; avoids contention
    return true,
}
```

Listing 3.5: *Pseudocode for weak-field discovery during marking. Annotated fields are diverted from the strong mark barrier and recorded in a per-worker array (see Appendix E.3.3).*

```
void process_worker_discovered_weak_fields(ZWeakFieldArray& arr) {
    size_t dropped = 0,
    for (size_t i = 0, i < arr.length(), i++) {
        const ZWeakFieldData& data = arr.at(i), // sequential scan
        if (try_make_inactive_fast(data)) { // clear dead or recolour live
            cleared_weak_fields_count++,
        } else {
            dropped++, // field still active; keep slot for next cycle
        }
    }
    arr.clear_and_reserve(dropped), // retain capacity for next cycle
}
```

Listing 3.6: *Pseudocode for weak-field processing. Each entry is passed to the clear barrier, which clears dead referents and recolours live ones (see Appendix E.3.3).*

4. Evaluation

The implemented optimisations described in Chapter 3 were evaluated through custom microbenchmarks designed to synthetically target the reference processing pipeline. The benchmarks were executed on a supercomputer cluster under controlled conditions, and the resulting GC timing statistics and memory usage metrics were collected to enable comparison between the baseline and optimised variants.

4.1 Optimisation Variants

Nine optimisation variants were evaluated in total. Eight of them cover all combinations of the three `WeakReference` pipeline optimisations described in Chapter 3: the optimised clear path, skipping enqueue for queue-less references via separate discovered lists, and the dynamic array. Table 4.1 shows which optimisations each variant enables. The `none` variant serves as the unmodified baseline. The remaining variants enable a subset of the three optimisations, allowing their individual and combined effects to be isolated. The `all` variant enables all three simultaneously. The ninth and final variant, `weak_fields`, corresponds to the `@weak` field annotation described in Section 3.4.

Each variant is implemented as a set of patches under `patches/<variant>/`. All variants except for `dyn_only` and `weak_fields` also include a shared base patch with infrastructure changes common to those optimisations. Table 4.2 lists the resulting diff sizes, full per-file breakdowns are available in Appendix C. A build script (`scripts/build_configs.sh`) automates applying the appropriate patches and building each JDK binary.

4.2 Performance Evaluation

To evaluate the performance impact of the optimisation approaches, four microbenchmarks were used under controlled conditions: two `WeakReference`-based benchmarks and two weak-field counterparts with analogous workload structure. The benchmarks and their execution environment are described below.

4.2.1 Benchmark Design

Two benchmark types were used to evaluate the optimisation approaches. The first type isolates reference-processing cost around a single shared referent, and the second is a more realistic multi-object workload that introduces greater allocation and liveness variability. Each type was implemented both with `WeakReference` objects and with weak fields. In order to enable a valid `WeakReference`-to-weak-field comparison, both benchmark types use *holder objects*: plain Java objects that are kept strongly reachable throughout the benchmark and whose sole purpose is to hold a reference to a target object.

In the `WeakReference` variants, each holder contains a `WeakReference<T>` field pointing to the target. In the weak-field variants, each holder contains an `@weak T` field pointing to the same target. In both cases the holder object itself is strongly referenced (kept alive in an array), while the target object is only weakly reachable through the holder. This ensures that the GC workload is structurally identical across the two mechanisms: the same number of holder objects

Table 4.1. *Optimisation variants and their enabled optimisations. A tick indicates the optimisation is active, a dash indicates it is disabled. Throughout the analysis and discussion, variants are referred to by their canonical names as listed in this table (e.g.* `clear_path_dyn`*,* `all`*).*

Variant	Optimised Path Clear	Skip Enqueue	Dynamic Array
`none`	—	—	—
`dyn_only`	—	—	✓
`sep_only`	—	✓	—
`sep_dyn`	—	✓	✓
`clear_path_only`	✓	—	—
`clear_path_dyn`	✓	—	✓
`clear_path_sep`	✓	✓	—
`all`	✓	✓	✓
`weak_fields`	—	—	—

Table 4.2. *Files changed and diff size per variant relative to upstream. Variants marked with † include the shared base patch (8 files, +39/−17 lines) in their totals.*

Variant	Modified	New	Ins.	Del.	Net
`all`†	10	1	+436	−65	+371
`clear_path_dyn`†	10	1	+448	−87	+361
`clear_path_only`†	10	0	+233	−44	+189
`clear_path_sep`†	10	0	+256	−43	+213
`dyn_only`	2	1	+210	−43	+167
`sep_dyn`†	10	1	+398	−58	+340
`sep_only`†	10	0	+213	−42	+171
`weak_fields`	27	2	+940	−257	+683

exists, the same number of target objects is weakly reachable, and the same GC events trigger processing of those weak references.

The key difference is the mechanism used to express the weak reference. In the `WeakReference` variant, each weak relation is represented by a separate `WeakReference` wrapper object that must be allocated, promoted to the old generation, and processed through the standard reference pipeline. In the weak-field variant, the weak pointer is stored directly inside the holder object with no wrapper, so no additional objects are allocated or promoted. Any observed performance difference between the two variants is therefore attributable to the presence or absence of the `WeakReference` wrapper object and its associated pipeline cost, rather than to any difference in workload structure or access pattern.

Single-Object Benchmark

The single-object benchmarks allocate 20 million holder objects, each containing either a `WeakReference` object or an annotated weak field that points to a single shared target object. The holder objects are allocated and stored in a randomised order using a Fisher-Yates shuffle [26] to avoid locality patterns. None of the weak references are created with a `ReferenceQueue`. After a short pause, the single strong reference to the target object is released and a GC cycle is triggered via `System.gc()`. Because all weak references become clearable simultaneously in one GC cycle, this benchmark maximises the per-cycle reference-processing load and isolates the cost of the processing pipeline itself.

Multi-Object Benchmark

The multi-object benchmarks represent a slightly more realistic workload. The benchmarks allocate 2 million objects, each containing a randomly sized byte array payload (between 509

Table 4.3. *Cluster node hardware specification*

Property	**Value**
Central Processing Unit (CPU)	AMD EPYC 9454P (Zen 4), 2.75 GHz
Physical cores	48
Threads	96
Memory	768 GiB

Table 4.4. *JVM options used in all benchmark runs*

Option	**Purpose**
`-Xms100g -Xmx100g`	Fixed 100 GB heap
`-XX:+UseZGC`	Enable ZGC
`-XX:InitialTenuringThreshold=1`	Promote objects to old generation after one cycle
`-XX:MaxTenuringThreshold=1`	Cap tenuring at 1
`-XX:-ZProactive`	Disable proactive GC
`-XX:ZCollectionIntervalMajor=315360000`	Effectively disable timer-driven major collections
`-XX:+ZCollectionIntervalOnly`	Suppress allocation-pressure-driven major collections
`-Xlog:gc+stats,gc+ref`	GC statistics and reference processing logs
`-XX:NativeMemoryTracking=summary`	Native Memory Tracking

and 1097 bytes, two primes close to 512 and 1024). Each object has a strong reference and a corresponding holder object with either a `WeakReference` object or an annotated weak field. The allocation and storing order of both strong references and holder objects are randomised using separate Fisher-Yates shuffles [26] to avoid locality patterns.

Unlike the single-object benchmark, the strong references are not all released at once. Instead, the benchmark proceeds through five rounds of clearing. In each round, 20 % of the strong references are nulled in a randomised order, making the corresponding objects eligible for garbage collection. After each round, `System.gc()` is called explicitly to trigger a collection cycle. This gradual release pattern means that weak references and weak fields are discovered and processed across multiple GC cycles rather than in a single burst. Furthermore, it tests how the pipeline performs when the referents have different sizes and lifetimes, which may affect locality and traversal cost.

4.2.2 Execution Environment

The benchmarks were executed on Uppsala Multidisciplinary Center for Advanced Computational Science (UPPMAX)'s supercomputer cluster Pelle. UPPMAX is part of National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725. Table 4.3 lists the hardware specification of the cluster nodes. Each optimisation variant was run using its pre-built JDK binary (see Section 4.1). To prevent file-system I/O from introducing timing variance, the JDK binaries, output log files, and results data were all stored on a RAM-backed file system. Table 4.4 lists the JVM options used in all benchmark runs.

Table 4.5. *Continuous monitoring metrics used in the measurement pipeline.*

Metric	Units	Description
RSS	kB	Process resident set size, representing total physical memory in use by the JVM process.
GC Committed Memory (auxiliary GCr memory)	kB	GCr-related memory currently committed by the JVM. Referred to as *auxiliary GCr memory*.
Java Heap	kB	Java heap usage.

4.2.3 Measurement Methodology

Each benchmark was run for a total of 250 measurement iterations, yielding 250 measured runs per variant. Jobs were submitted via Simple Linux Utility for Resource Management (SLURM) and allocated one node running in exclusive mode. Iterations were split across 4 parallel instances. Each instance was pinned via `taskset` to a set of 10 JVM cores and 2 auxiliary cores (12 cores per instance), preventing interference between concurrently running instances. Each instance ran its share of the 250 measurement iterations preceded by 1 warm-up iteration (one per parallel instance, four discarded in total) whose results were discarded. The warm-up executed the full benchmark workload for every variant. Within each measurement iteration, all nine optimisation variants were run in sequence with a 10-second cooldown between each variant. This interleaved ordering mitigates systematic drift in system conditions affecting one configuration more than others.

During each run, a memory monitoring script samples memory metrics from the process at high frequency. See Table 4.5 for the memory metrics collected.

In addition to the continuous memory samples, GC statistics are parsed from the log files of each run. For each GC metric, the parser extracts the total average and total maximum values reported by ZGC per run. The single-object benchmark concentrates its entire weak-reference load into a single collection cycle, so the per-run *maximum* is the relevant statistic. The multi-object benchmark spreads work across several cycles, so the per-run *average* is used instead. Table 4.6 lists the GCr log metrics collected. The rest of the GCr log metrics were also collected but are not the focus of this evaluation. They are, however, available in the published dataset alongside two additional measurements, one on differing hardware [27].

Each timing metric corresponds to the elapsed wall-clock duration of a specific ZGC phase as reported in the `gc+stats` log output. ZGC logs one timing entry per phase per GC cycle; these entries are the raw values the parser processes. The single-object benchmark triggers exactly one major GC cycle per run (via a single `System.gc()` call after releasing the target object), so the per-run maximum equals the single cycle's reported duration. The multi-object benchmark triggers five major GC cycles per run (one per 20 % release round), and the per-run average is the arithmetic mean of the five cycle values. Reporting the maximum for the single-object benchmark and the average for the multi-object benchmark is consistent: in the single-object case the maximum isolates the peak cost of processing the full 20-million-reference batch, whilst in the multi-object case the average captures the typical per-cycle cost across the five smaller batches.

4.3 Presentation of Results

Results are presented for all nine variants listed in Table 4.1: the unmodified baseline (`none`), the seven `WeakReference` pipeline variants, and `weak_fields`. This enables two complementary comparisons:

- **Intra-model comparison** among the `WeakReference` variants, isolating the effect of each optimisation and their interactions relative to the baseline.

Table 4.6. *GCr log metrics used in the measurement pipeline. For the single-object benchmark, per-run maximum values are used, for the multi-object benchmark, per-run averages are used.*

Metric	Units	Description
Major Collection	ms	End-to-end duration of major collections.
Old Generation	ms	Total time spent in old-generation collection work.
Old Phase: Concurrent Mark	ms	Time spent in old-generation concurrent marking.
Old Phase: Concurrent Process Non-Strong	ms	Time spent processing non-strong references in the old generation.
Old Phase: Concurrent Relocate	ms	Time spent in old-generation concurrent relocation.
Young Generation (Promote All)	ms	Total time in young-generation collection. Because the benchmarks use `System.gc()`, every young-generation collection is a Promote All, which evacuates all surviving objects directly to the old generation.
Young Phase: Concurrent Mark	ms	Time spent in young-generation concurrent marking.
Young Phase: Concurrent Relocate	ms	Time spent in young-generation concurrent relocation.

- **Inter-model comparison** between the best-performing `WeakReference` variants and the weak-field representation, evaluating whether avoiding `WeakReference` objects altogether offers additional benefit.

Each GC timing figure shows a composite plot: a violin plot (top) with the full per-run distribution and a median percentage-difference histogram (bottom) showing all variants relative to the `none` baseline, including `none` itself at 0 %. The single-object benchmark uses per-run maxima and the multi-object benchmark uses per-run averages, following the rationale in Section 4.2.3. The median is used for comparison rather than the mean, as the non-symmetric distributions tend to skew the mean. Note that the violin plot y-axis does not start at zero, it is range-fitted to the observed values so that the shape of each distribution is easier to see. GC timing results are shown for five metrics for each benchmark: Major Collection (Figures 4.1 and 4.2), Old Generation (Figures 4.3 and 4.4), Old Phase: Concurrent Mark (Figures 4.5 and 4.6), Old Phase: Concurrent Process Non-Strong (Figures 4.7 and 4.8), and Young Generation – Promote All (Figures 4.9 and 4.10). Memory results are shown as median percentage-difference histograms of per-run maxima for three metrics: auxiliary GCr memory (GC Committed Memory), Java heap, and total process RSS, for both benchmarks (Figures 4.11 and 4.12). Figures for remaining GCr metrics from Table 4.6 are shown in Appendix D.

How to read the figures.

Each GC timing figure contains two panels. The **top panel** (violin plot) shows the full empirical distribution of per-run values for each variant: the body width represents the density of observations at that value, the central horizontal line marks the median, and the whiskers reach to the 10th and 90th percentiles. Variants are ordered left to right: `none` (the unmodified baseline), the seven `WeakReference` pipeline variants, and `weak_fields`. When two distributions overlap strongly (the body of one spans the body of the other) median differences between those variants should be treated as directionally indicative. The **bottom panel** (percentage-difference histogram) shows the median of each variant as a percentage change relative to the `none` baseline. Negative bars indicate lower (better) values. The percentage shown is $(\text{median}_{\text{variant}} - \text{median}_{\text{none}}) / \text{median}_{\text{none}}$. The two panels together let the reader assess both the size of the effect (bottom panel) and the certainty of the comparison (top panel): a large bar paired with non-overlapping distributions indicates a clear effect, whilst a small bar paired with heavily overlapping distributions warrants caution in interpretation.

In addition to the figures, Table 4.7 shows the mean, median, and IQR of the fraction of major-collection time attributable to non-strong processing for the `none` baseline and the `all` variant, across both benchmarks. The fraction is computed by dividing the per-run values of the non-strong processing times by the major-collection times. Just as for the timing figures, the single-object benchmark uses per-run maxima and the multi-object benchmark uses per-run averages. In Figures D.9 and D.10 in Appendix D, the full distributions of these ratios are shown as violin plots for both benchmarks.

Table 4.7. *Ratio of Concurrent Process Non-Strong to Major Collection for* `none` *and* `all` *variants.*

	Single			Multi		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	14.7 %	14.4 %	3.2 %	4.5 %	4.5 %	1.1 %
`all`	3.0 %	3.4 %	0.8 %	1.9 %	2.0 %	0.5 %

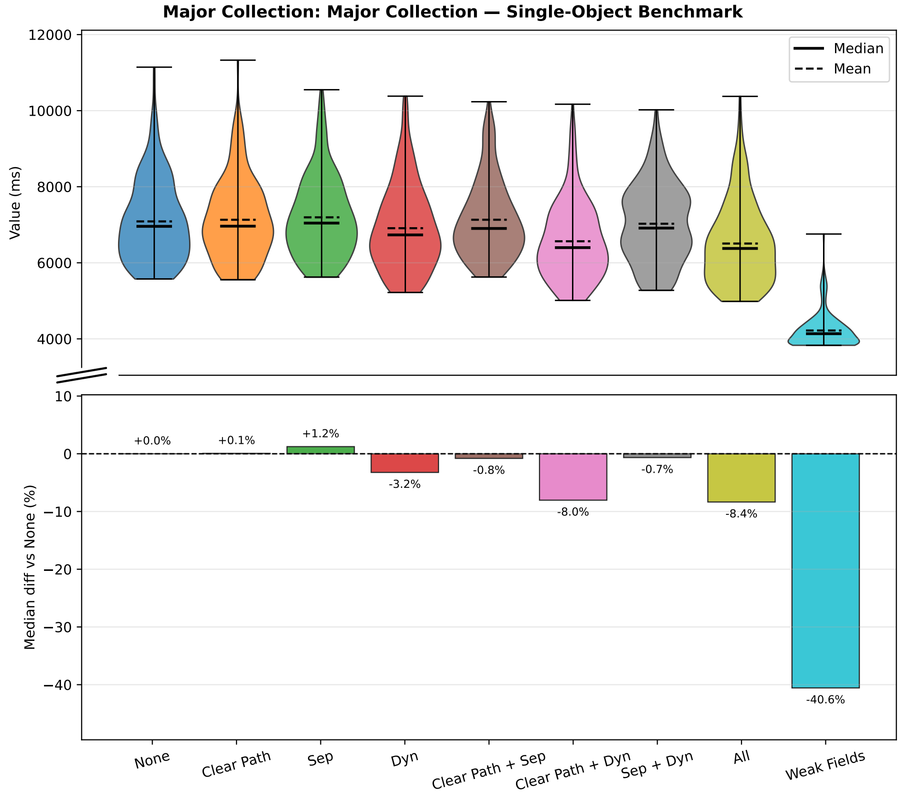

Figure 4.1. Major Collection timing, **single-object benchmark** (per-run maxima). Lower is better. Top: distribution, bottom: median % difference vs baseline.

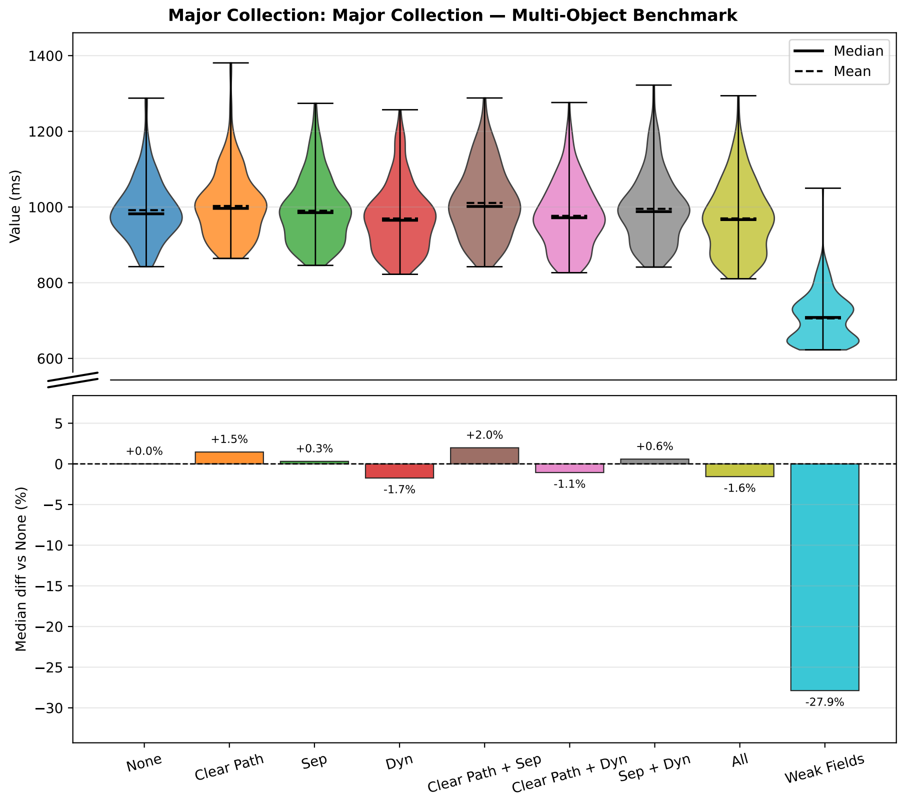

Figure 4.2. Major Collection timing, **multi-object benchmark** (per-run averages). Lower is better. Top: distribution, bottom: median % difference vs baseline.

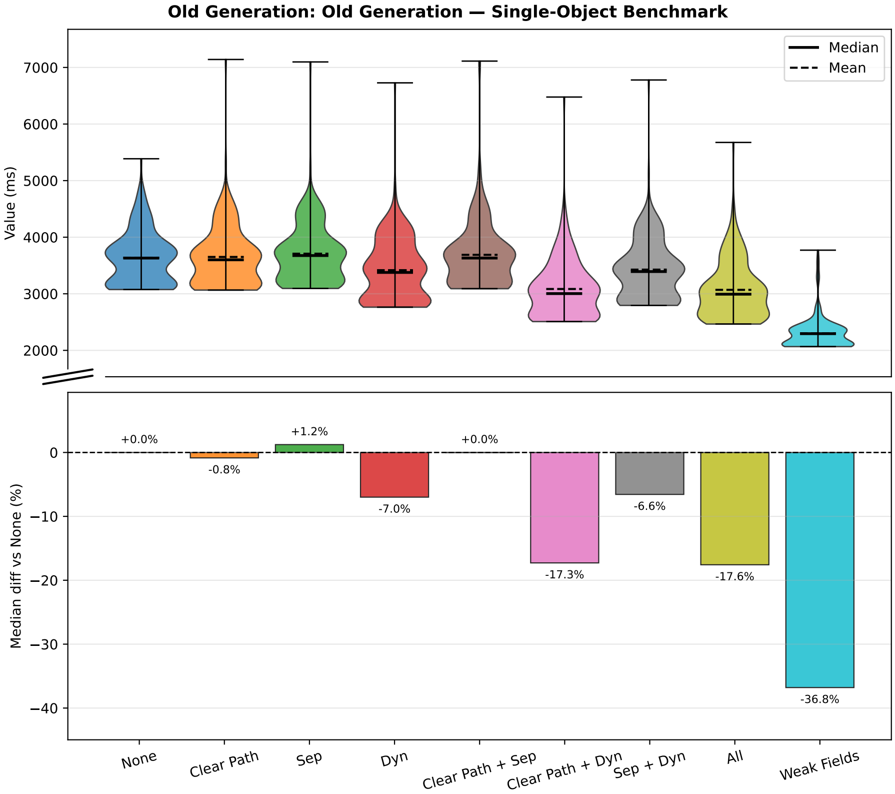

Figure 4.3. Old Generation timing, **single-object benchmark** (per-run maxima). Lower is better. Top: distribution, bottom: median % difference vs baseline.

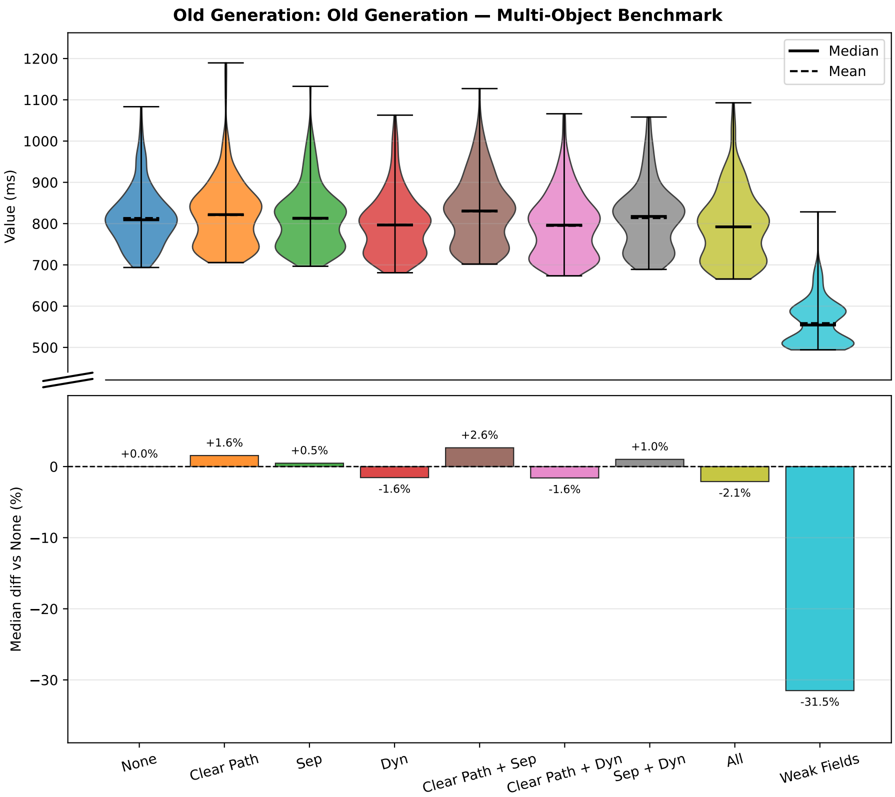

Figure 4.4. Old Generation timing, **multi-object benchmark** (per-run averages). Lower is better. Top: distribution, bottom: median % difference vs baseline.

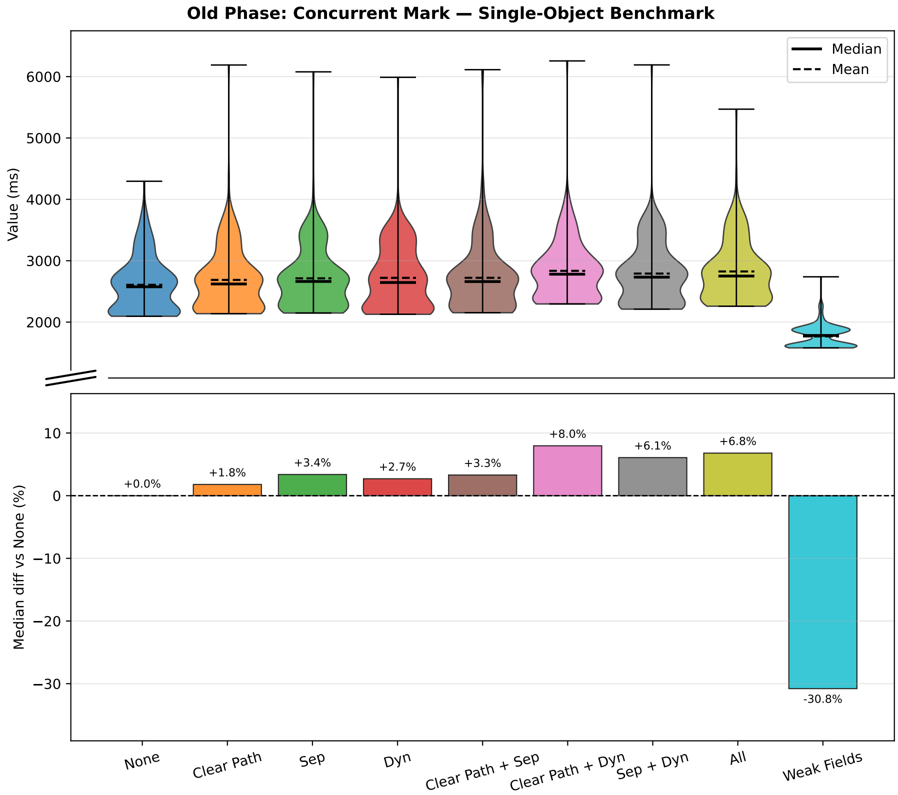

Figure 4.5. Old Phase: Concurrent Mark timing, **single-object benchmark** (per-run maxima). Lower is better. Top: distribution, bottom: median % difference vs baseline.

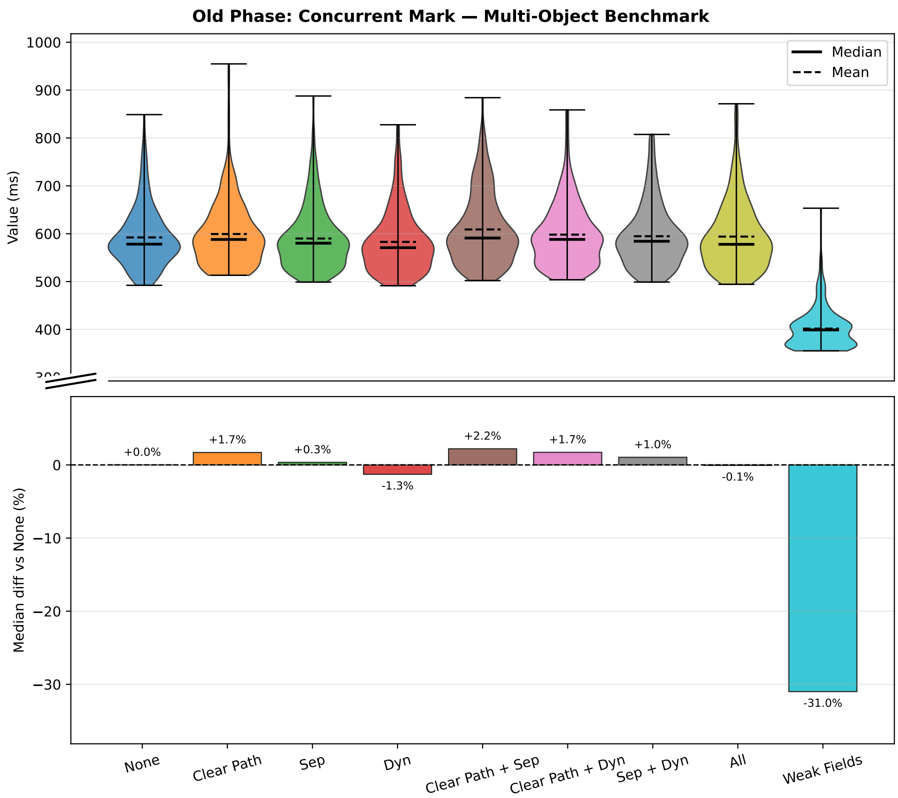

Figure 4.6. Old Phase: Concurrent Mark timing, **multi-object benchmark** (per-run averages). Lower is better. Top: distribution, bottom: median % difference vs baseline.

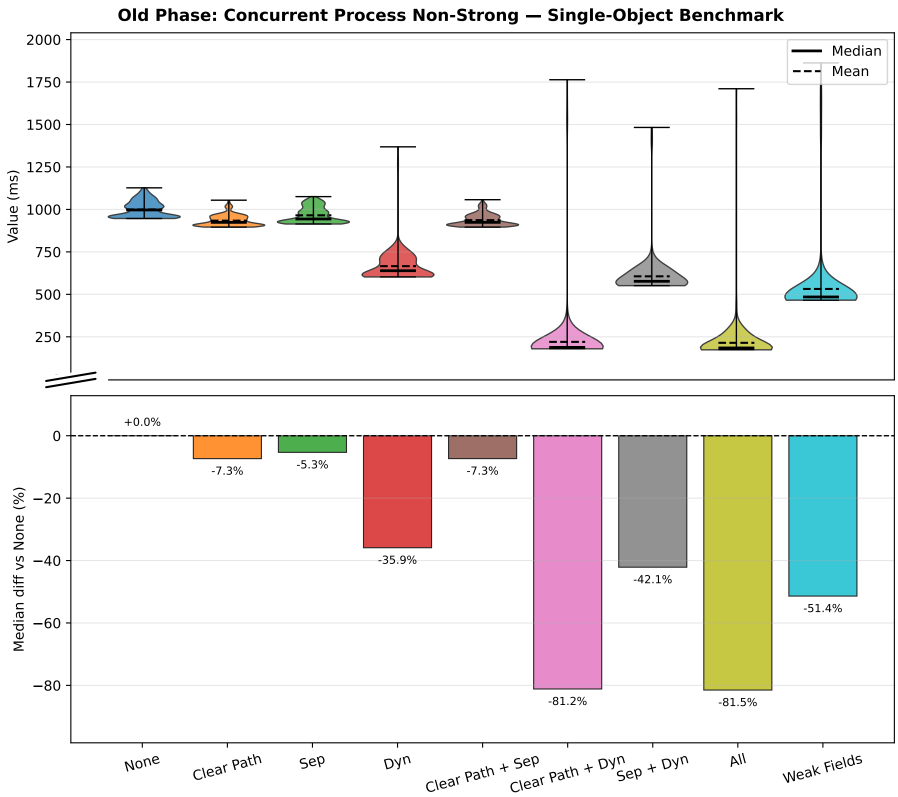

Figure 4.7. Old Phase: Concurrent Process Non-Strong timing, **single-object benchmark** (per-run maxima). Lower is better. Top: distribution, bottom: median % difference vs baseline.

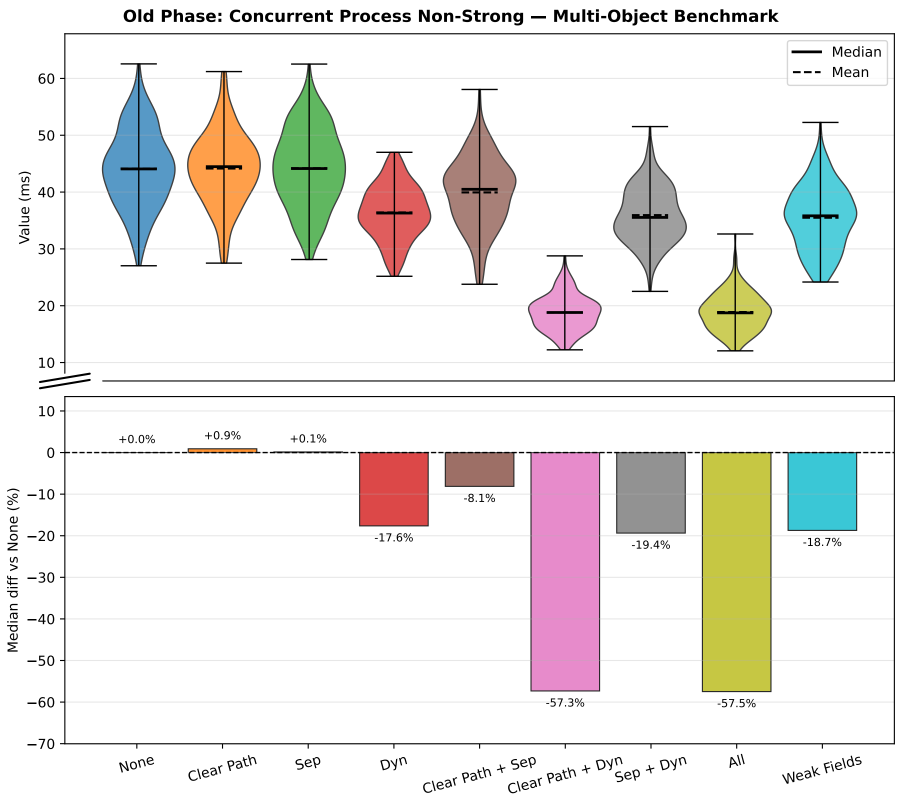

Figure 4.8. Old Phase: Concurrent Process Non-Strong timing, **multi-object benchmark** (per-run averages). Lower is better. Top: distribution, bottom: median % difference vs baseline.

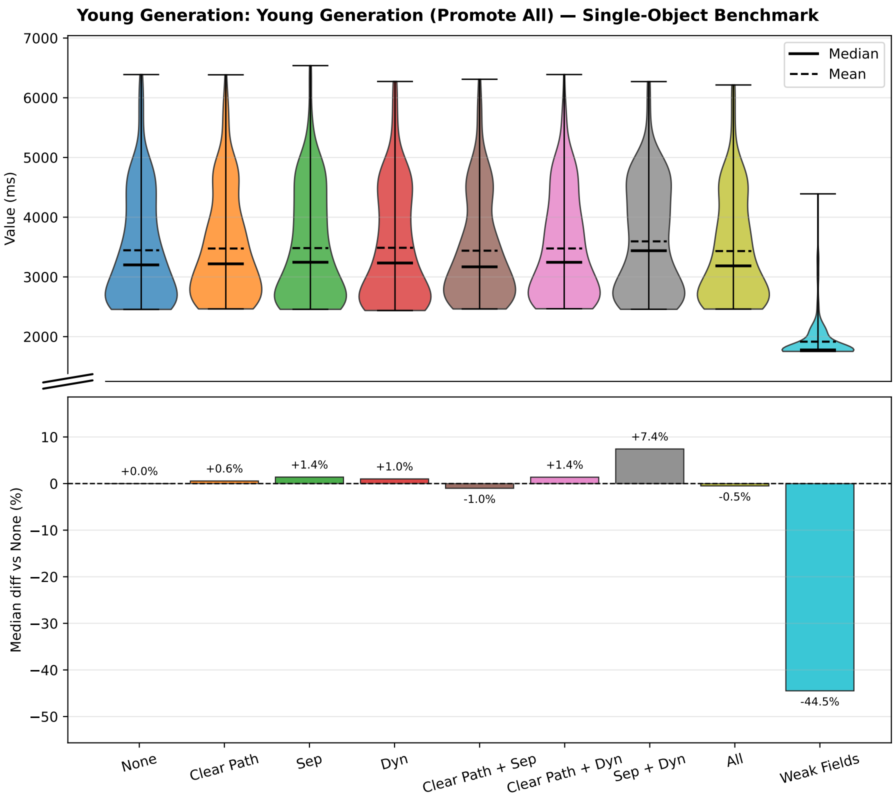

Figure 4.9. Young Generation (Promote All) timing, **single-object benchmark** (per-run maxima). Lower is better. Top: distribution, bottom: median % difference vs baseline.

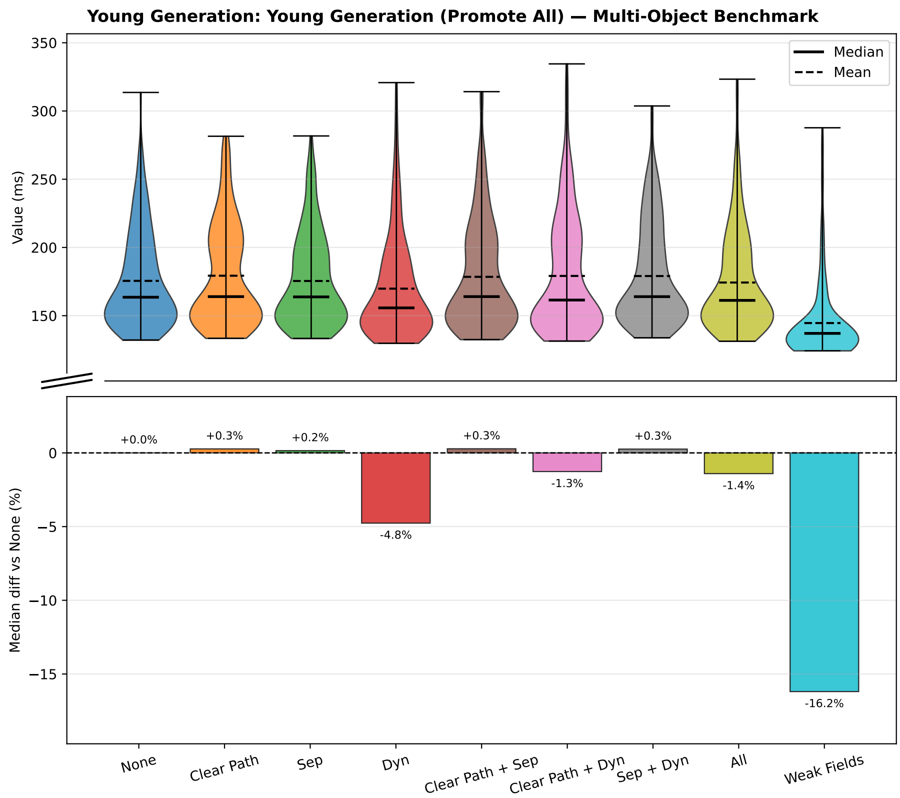

Figure 4.10. Young Generation (Promote All) timing, **multi-object benchmark** (per-run averages). Lower is better. Top: distribution, bottom: median % difference vs baseline.

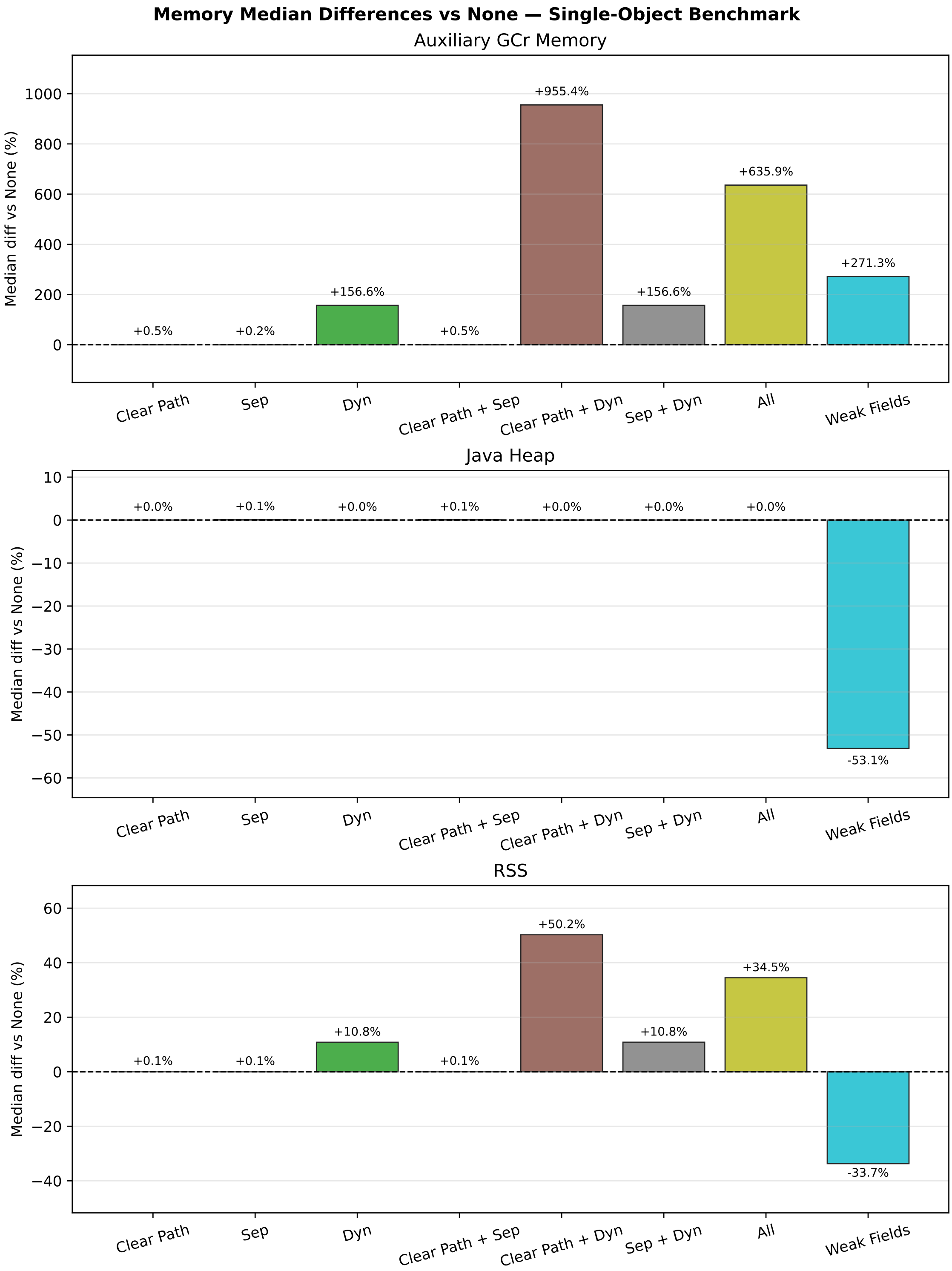

Figure 4.11. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, **single-object benchmark**. Lower is better. Top to bottom: auxiliary GCr memory, Java heap, total process RSS.

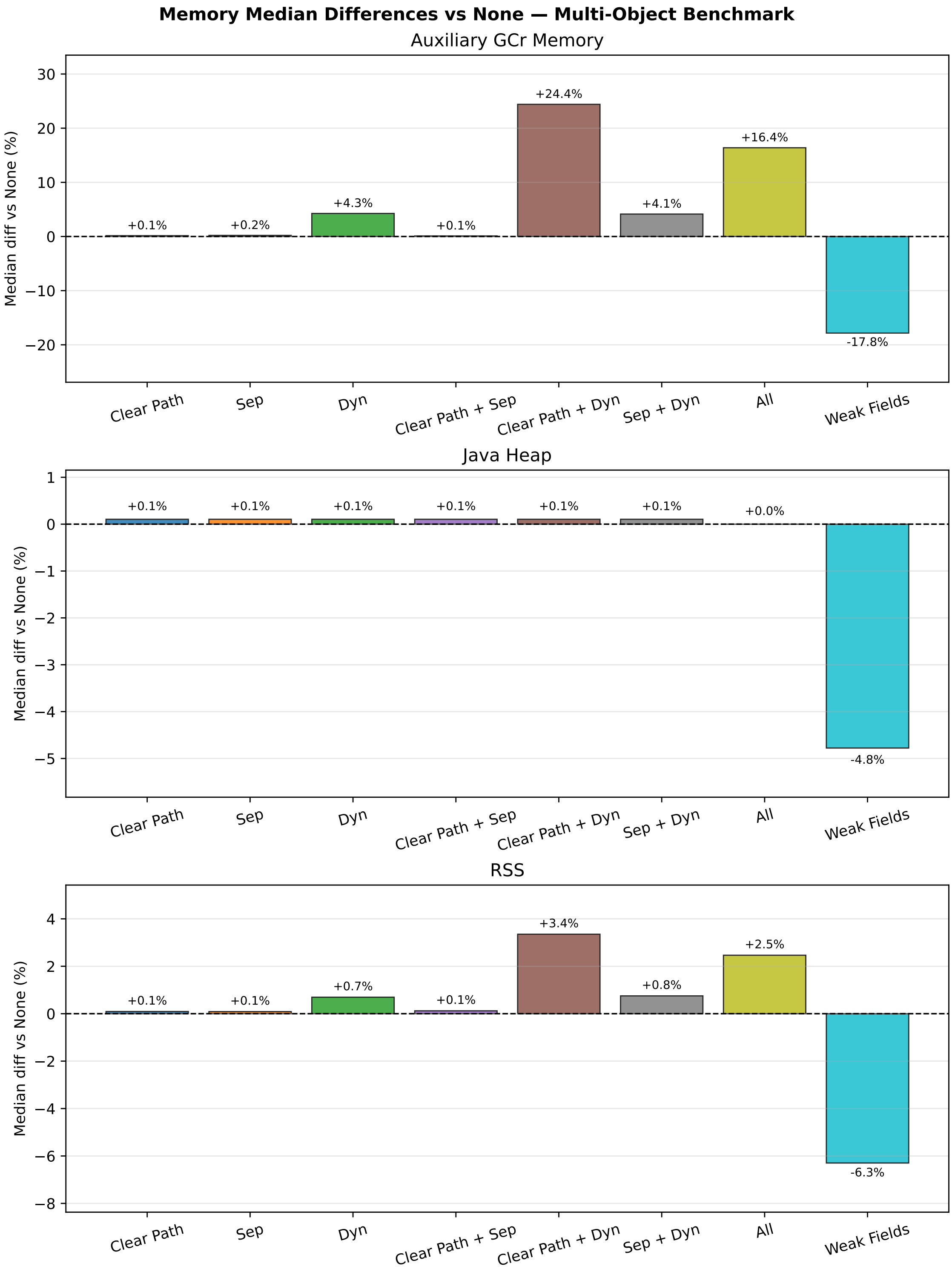

Figure 4.12. Median percentage difference in per-run maximum memory usage relative to the `none` baseline, **multi-object benchmark**. Lower is better. Top to bottom: auxiliary GCr memory, Java heap, total process RSS.

5. Analysis of Results

The measurements presented in Section 4.3 show clear differences between configurations on some metrics and largely overlapping distributions on others. The analysis proceeds in two parts: a comparison among all variants on both the targeted non-strong processing metric and the broader collection-time metrics, followed by an analysis of memory usage and trade-offs. The `weak_fields` variant is included throughout and compared with the `WeakReference` variants at each stage.

5.1 Non-Strong Reference Processing

The metric most directly targeted by the implemented optimisations is `Concurrent Process Non-Strong`, which captures the total wall-clock time spent in ZGC's concurrent reference-processing phase per GC cycle. This phase is the intended site of all three optimisations, and the clearest differences between variants are observed here, making it the natural starting point for the analysis.

Before comparing variants, it is worth establishing what fraction of total major-collection time is attributable to non-strong processing in the baseline. This fraction, summarised for the `none` baseline and the best-performing pipeline variant (`all`) in Table 4.7 in Section 4.3, represents the ceiling on how much the pipeline optimisations can reduce total major-collection time. Even an 81 % reduction in non-strong processing time can at most save the baseline fraction listed in that table. Realised savings in major-collection time will be smaller still because other phases are unaffected. The ceiling is much tighter in the multi-object benchmark: non-strong processing accounts for 14.7 % of major-collection time in the single-object `none` baseline but only 4.5 % in the multi-object `none` baseline, reflecting the more diverse workload in which GC effort is spread across marking, relocation, and other phases. After applying all three optimisations (`all`), these fractions fall to 3 % and 1.9 % respectively.

Figures 4.7 and 4.8 show the distributions of this metric for the single-object and multi-object benchmarks respectively. Full numerical summaries are given in Table B.4.

5.1.1 Effect of the Clear Path and Dynamic Array

The most striking reduction is achieved by the `clear_path_dyn` variant. In the single-object benchmark its median non-strong processing time is 187.9 ms, compared to 996.9 ms for the unmodified `none` baseline, a reduction of approximately 81 %. The `all` variant, which adds the skip-enqueue separation on top of `clear_path_dyn`, reaches a nearly identical median of 184.4 ms (also about an 81 % reduction). In the multi-object benchmark the pattern is consistent: `clear_path_dyn` achieves a median of 18.8 ms versus 44.1 ms for `none` (57 % reduction), and `all` reaches 18.7 ms (also 57 % reduction).

The near-equivalence of `clear_path_dyn` and `all` shows that adding the skip-enqueue separation to a configuration that already has the dynamic array and the specialised clear path yields little extra benefit to reference processing times: `clear_path_dyn` and `all` differ by just 3.5 ms in the single-object benchmark and 0.1 ms in the multi-object benchmark.

The two individual optimisations `clear_path_only` and `dyn_only` show that the dynamic array is the more impactful of the two but that both contribute meaningfully to the overall improvement when combined. The `clear_path_only` variant, which enables only the specialised clear path, reduces the single-object median by 7 % (to 923.7 ms), while `dyn_only`, which enables only the dynamic array, achieves a 36 % reduction (to 639.1 ms). When combined in `clear_path_dyn`, the effect is substantially larger than either isolated contribution. This suggests that the dynamic-array traversal and the specialised clear logic remove different parts of the per-reference cost and therefore interact constructively rather than redundantly.

5.1.2 Effect of the Skip-Enqueue Separation Alone

The `sep_only` variant, which introduces the separate discovered list for queue-less weak references but does not include the dynamic array or the specialised clear path, produces negligible improvement on non-strong processing time. In the single-object benchmark its median is 943.8 ms, only 5 % below the `none` baseline, and its IQR of 73.8 ms is comparable to `none`'s 75.9 ms. In the multi-object benchmark the difference is entirely eliminated: 44.1 ms versus 44.1 ms for `none`.

5.1.3 Comparison with Weak Fields

The `weak_fields` variant achieves a median non-strong processing time of 484.6 ms in the single-object benchmark and 35.8 ms in the multi-object benchmark. These figures correspond to reductions of 51 % and 19 % respectively relative to `none`, showing clear improvement over the `none` baseline while remaining well short of the leading `clear_path_dyn` and `all` configurations. Its non-strong phase therefore improves clearly over the baseline, but not enough to match the best specialised `WeakReference` pipeline variants.

Notably, `weak_fields` is closely comparable to `sep_dyn` on this metric, especially in the multi-object benchmark where their medians are effectively indistinguishable: 35.8 ms versus 35.5 ms, a gap of just 0.3 ms. Their distributions are also closely aligned, with `sep_dyn` showing a slightly tighter Interquartile Range (IQR) of 7.1 ms compared to `weak_fields`'s 8.2 ms. In the single-object benchmark `weak_fields`'s median is somewhat lower (484.6 ms versus 576.9 ms for `sep_dyn`, a difference of about 17 %) and `weak_fields` has a substantially tighter IQR of 8.4 ms compared to `sep_dyn`'s 75.9 ms. This suggests that the weak-field approach may have a slightly better and significantly more consistent performance profile when the reference load is extremely high as it is in the single-object benchmark.

5.2 Overall GCr Collection Times

In contrast to the non-strong processing metric, the overall collection time metrics show broad, heavily overlapping distributions across all `WeakReference` variants, making it difficult to draw firm conclusions from pairwise comparisons within this group. `weak_fields`, however, shows a clear difference in the distribution compared to all `WeakReference` variants across nearly all overall collection metrics.

5.2.1 Major Collection

See Figures 4.1 and 4.2 for the comparisons and Table B.1 for numerical summaries.

For the single-object benchmark, the median major collection time of `none` is 6 958 ms with an IQR of about 1 596 ms. The `all` variant has a median of 6 377 ms (8 % below `none`) and the `clear_path_dyn` variant 6 399 ms (also 8 % below `none`), but with IQRs of 1 419 ms and 1 394 ms respectively, both distributions still overlap substantially with the baseline and with each other. Of the other `WeakReference` variants, `dyn_only` has the lowest median at 6 733 ms, which is only about 3 % below `none`, while the rest of the variants have medians that are within about 1 % of `none` and IQRs that are comparable to `none`'s. In the multi-object benchmark the effect is even smaller in relative terms, with medians ranging from 965 ms (`dyn_only`) to

1 002 ms (clear_path_sep) against a baseline of 982 ms, all distributions remain within a narrow IQR band of roughly 93–136 ms, making fine-grained ranking unreliable.

Among the WeakReference variants, all and clear_path_dyn tend to occupy the lower part of the collection-time distribution in both benchmarks, which is directionally consistent with their superior non-strong processing times. However, given the distribution overlap, this observation should be treated as slightly indicative.

The weak_fields variant stands clearly apart on this metric. In the single-object benchmark its median major collection time is 4 136 ms, compared with 6 377–7 044 ms for all WeakReference variants, a reduction of approximately 35 % against the best WeakReference variant and 41 % against none. Its IQR of 485 ms is also substantially tighter than the 1 354–1 596 ms IQRs of the WeakReference group. In the multi-object benchmark the gap is smaller but still unambiguous: 708 ms versus 965–1 002 ms, a reduction of approximately 28 % versus none. The source of this advantage is structural: by eliminating the WeakReference objects entirely, weak_fields reduces the amount of GC work that must be performed across all phases of the cycle, not only in the non-strong processing phase.

5.2.2 Old Generation

See Figures 4.3 and 4.4 for the comparisons and Table B.2 for full numerical summaries.

The old-generation collection time shows a clearer split among the WeakReference variants in the single-object benchmark than the major collection metric does. Configurations that include the dynamic array stand noticeably apart: all and clear_path_dyn both reach a median of approximately 2 993–3 003 ms, roughly 17 % below the none baseline of 3 631 ms, while dyn_only and sep_dyn achieve intermediate reductions of around 7 % (3 377 ms and 3 393 ms respectively). By contrast, the non-dynamic-array variants (sep_only, clear_path_only, clear_path_sep) remain within 1 % of none at 3 600–3 676 ms. Given that the IQRs are 600–720 ms for the dynamic-array variants and 600–870 ms for the non-dynamic variants, the distributions of the two groups do overlap, nevertheless, the median separation of roughly 600 ms between all and the non-dynamic cluster is comparable to a full IQR, indicating a consistent directional effect. In the multi-object benchmark the picture is completely different: all WeakReference variants fall within 792–831 ms against a baseline of 809 ms.

The weak_fields variant again reduces the median substantially: 2 295 ms in the single-object benchmark (37 % below none's 3 631 ms) and 554 ms in the multi-object benchmark (31 % below none's 809 ms). Its IQR of 282 ms in the single-object benchmark is also considerably tighter than the 600–870 ms range of the WeakReference group. As stated in Section 5.2.1, this advantage is attributable to the reduced GC workload across all phases of the cycle, not only the non-strong processing phase.

5.2.3 Old Phase: Concurrent Mark

See Figures 4.5 and 4.6 for the comparisons and Table B.3 for numerical summaries.

The concurrent mark phase shows where the WeakReference pipeline optimisations suffer. In the single-object benchmark, WeakReference variant medians for the old-phase concurrent mark metric fall in the range 2 575–2 780 ms, with none at 2 575 ms. The combination variants with dynamic arrays show higher medians (clear_path_dyn at 2 780 ms, all at 2 750 ms), the dyn_only variant is slightly lower at 2 644 ms, which is expected given that less information is stored in the dynamic array entries. However, this pattern is not consistent across benchmarks: in the multi-object benchmark, for example, the dyn_only variant has a median of 570 ms, which is lower than the none median of 578 ms. Due to the high IQRs (560–810 ms and 67–99 ms respectively) the differences observed between the WeakReference variants on this metric should also be treated as slightly indicative.

In contrast, weak_fields reduces the concurrent mark median by approximately 31 % in both benchmarks (1 782 ms in single-object, versus 2 575 ms for none; 399 ms versus 578 ms in the multi-object benchmark). This is again consistent with the statements above: by eliminating the `WeakReference` objects, `weak_fields` reduces the amount of GC work that must be performed across most phases of the cycle, including concurrent marking.

5.2.4 Young Generation (Promote All)

See Figures 4.9 and 4.10 for the comparisons and Table B.6 for numerical summaries.

The young-generation (Promote All) phase shows no systematic response to the `WeakReference` pipeline optimisations, as expected: all three optimisations operate exclusively on old-generation reference processing and should have no bearing on young-generation work. The majority of `WeakReference` variants in both benchmarks fall within ±1.5 % of the `none` median (3 201 ms in the single-object benchmark, 164 ms in the multi-object benchmark). Two isolated deviations stand out but are unlikely to reflect genuine causal effects: neither replicates across both benchmarks, and each falls well within the natural spread of its own distribution. In the single-object benchmark, `sep_dyn` has a median of 3 439 ms, some 7.4 % above `none`, yet this deviation is well within its IQR of 1 622 ms and disappears entirely in the multi-object benchmark where `sep_dyn` is within 0.3 % of baseline. In the multi-object benchmark, `dyn_only` has a median of 156 ms, 4.8 % below the `none` median of 164 ms. This too sits within its IQR of 45 ms and has no counterpart in the single-object benchmark where `dyn_only` is within 1 % of baseline.

The `weak_fields` variant again stands apart: a −44 % reduction in the single-object benchmark and −16 % in the multi-object benchmark, attributable to the elimination of `WeakReference` objects that would otherwise be marked and then promoted (moved to the old generation).

5.3 Memory Behaviour

The dynamic-array optimisation improves non-strong processing time at the cost of allocating a contiguous buffer per GC worker thread rather than reusing the intrusive linked-list fields already present in `Reference` objects. This trade-off makes memory usage an important secondary dimension of the evaluation and since the dynamic array is allocated by the GCr, the auxiliary GCr memory is the most direct indicator of the dynamic array's overhead. Java heap occupancy reflects the cost of maintaining the `WeakReference` objects themselves, which `weak_fields` eliminates entirely. Total process RSS captures the combined footprint of heap, GCr metadata, and native memory, providing the broadest view of the memory trade-off. See Figures 4.11 and 4.12 for the comparisons and Tables B.10 to B.12 for numerical summaries.

5.3.1 Auxiliary GCr Memory Usage and the Dynamic Array

All configurations that include the dynamic array (`dyn_only`, `sep_dyn`, `clear_path_dyn`, and `all`) incur substantially higher auxiliary GCr memory than variants that retain the intrusive linked list. In the single-object benchmark, the median per-run maximum for `dyn_only` and `sep_dyn` is 308 MB of auxiliary GCr memory, compared to approximately 120 MB for the linked-list variants, an increase of 157 %. The `clear_path_dyn` variant is the highest of all at 1 268 MB (roughly 10× the non-array baseline), and `all` reaches 884 MB (+637 %). In the multi-object benchmark the relative differences are much smaller: `dyn_only` and `sep_dyn` are only 4 % above the `none` baseline of 292 MB at 305 MB and 304 MB respectively, `all` is 340 MB (+16 %), and `clear_path_dyn` is 363 MB (+24 %).

The `weak_fields` variant occupies an intermediate position. Its median auxiliary GCr memory in the single-object benchmark is 446 MB, 272 % above the 120 MB of the non-array `WeakReference` variants but well below `clear_path_dyn`'s 1 268 MB. In the multi-object benchmark it is the lowest of all configurations at 240 MB, 18 % below the `none` baseline of 292 MB.

5.3.2 Java Heap Usage

The heap usage figures are essentially identical across all `WeakReference` variants (approximately 1 720 MB per-run median in the single-object benchmark and 1 926–1 928 MB in the multi-object benchmark), with all variants within 0.1 % of each other.

The `weak_fields` variant, however, shows markedly lower heap usage: a median of 806 MB in the single-object benchmark, 53 % below the `WeakReference` group, and 1 834 MB in the multi-object benchmark, 5 % below. This is consistent with the fact that `weak_fields` eliminates the `WeakReference` objects entirely, which constitute a significant portion of the heap occupancy in the `WeakReference` variants.

5.3.3 Process Memory Footprint

The RSS results follow the same overall direction as auxiliary GCr memory and heap usage. In the single-object benchmark, `clear_path_dyn` reaches the highest median RSS at 2 856 MB (+50 % versus the `none` baseline of 1 901 MB), followed by `all` at 2 556 MB (+34 %) and `dyn_only` and `sep_dyn` at 2 107 MB each (+11 %). The non-array `WeakReference` variants remain within 0.2 % of `none`. `weak_fields`, by contrast, has the lowest median RSS of all configurations at 1 260 MB, 34 % below `none`. In the multi-object benchmark the spread is much narrower: `clear_path_dyn` is the highest at 2 487 MB (+3 % versus `none`'s 2 407 MB), while `weak_fields` is the lowest at 2 255 MB (−6 %). This strengthens the interpretation that the dynamic-array variants trade runtime memory overhead for faster queue-less weak-reference processing, while weak fields reduce both heap occupancy and total process footprint.

6. Discussion

Across both benchmarks, the pipeline optimisations improve the metrics they directly target while leaving broader collection behaviour relatively unchanged. Against this backdrop, the `weak_fields` variant stands apart: by eliminating `WeakReference` objects from the heap entirely, it propagates improvements through every phase of the collection cycle rather than only the reference processing phase. The following sections explain the mechanisms behind these patterns and reflect on the scope of the evaluation.

6.1 Superadditive Interaction of the Clear Path and Dynamic Array

One clear result in reference processing times is that the `clear_path_dyn` variant, combining the clear-path optimisation with the dynamic array, produces an 81 % reduction in non-strong processing time, far larger than the sum of their individual contributions (7 % and 36 % respectively). This superadditivity arises from the tight coupling of the two mechanisms at the implementation level.

In `dyn_only`, the dynamic array provides improved cache locality during traversal: discovered references are stored contiguously in memory, so iterating through them involves sequential cache-line loads rather than following pointer chains scattered across the heap [15]. However, the per-reference processing logic still uses the standard CAS-based clear path, including a load barrier on the referent field and a CAS operation to atomically set the cleared referent to a coloured null pointer. These operations are not eliminated by the array alone.

In `clear_path_only`, the specialised clear logic eliminates the CAS on the referent field for old-generation referents (replacing it with a direct store) and removes the virtual call to determine the reference type. However, the referent field address and value must still be loaded individually for each reference during processing, and traversal still follows the intrusive linked list with its associated cache misses.

When both optimisations are active in `clear_path_dyn`, the dynamic array enables a further key capability: the referent field address and value are pre-loaded during discovery and stored directly in the array entry (see Listing 3.2). In processing, these values are read from the contiguous array without issuing any additional heap loads. The specialised clear path operates on the pre-loaded data, removing both the load-barrier overhead and the pointer-chasing overhead simultaneously. In summary, each optimisation seems to remove a bottleneck that would persist if the other were applied alone.

6.2 Skip-Enqueue Separation Providing Minimal Benefit

The skip-enqueue optimisation was motivated by the observation that the enqueueing stage performed unnecessary work for queue-less weak references: the references were still linked into the pending list, transferred to the `ReferenceHandler` thread, and iterated over only to be discarded [9, 12, 13]. Despite this, `sep_only` reduces the single-object median by only 5 % and shows no reduction in the multi-object benchmark. When combined with the dynamic array in `sep_dyn`, the improvement over `dyn_only` is only 6 percentage points in the single-object benchmark and 2 percentage points in the multi-object benchmark. The same pattern continues in the `all` variant, which shows only a 0.3 percentage point increase over `clear_path_dyn` in the single-object benchmark and a 0.2 percentage point increase in the multi-object benchmark.

Where `all` offers a meaningful advantage over `clear_path_dyn` is in auxiliary GCr memory: by routing queue-less references to the separate list, the array entries in `all` do not need to store the reference address field that `clear_path_dyn` requires, reducing auxiliary GCr memory by approximately 30 % in the single-object benchmark (from 1 268 MB to 884 MB).

This likely means that the enqueue stage was not the bottleneck in reference processing in these benchmarks. The dominant costs in the pipeline appear to lie in the processing stage.

However, one effect not captured by the evaluation is that the skip-enqueue separation should reduce the load on the `ReferenceHandler` thread, which must process the pending list and perform callbacks for queue-backed references. By routing queue-less references away from this thread, the separation should reduce contention and improve latency for queue-backed references in a mixed workload. This is a plausible benefit of the mechanism, but it remains a hypothesis: neither benchmark in the evaluation contains queue-backed references, so this effect cannot be confirmed from the collected data.

6.3 Concurrent-Mark Impact

In the single-object benchmark, variants with the dynamic array show slightly higher concurrent-mark times than `none`: `dyn_only` is 3 % higher, `clear_path_dyn` is 8 % higher, `sep_dyn` is 6 % higher, and `all` is 7 % higher. This pattern is not observed in the multi-object benchmark, where concurrent-mark times are either unchanged, slightly lower, or slightly higher without a clear pattern.

There are two plausible explanations for the single-object increase. First, the dynamic array variants manage a large auxiliary data structure (the `ZWeakRefArray`) during GC cycles, which increases memory pressure during the concurrent mark phase. Second, the dynamic array variants may have slightly higher average per-reference processing times during marking due to the additional logic for growing and copying the array. However, the fact that the increase is not observed in the multi-object benchmark, where the reference population is smaller and the processing workload is spread across multiple cycles, suggests that the additional per-reference processing time is the primary cause. Since the processing workload is spread over multiple cycles and no extra references are created, the auxiliary array only has to grow in the mark phase of the first cycle. In subsequent cycles, the array can be reused without growth thanks to the capacity reservation strategy (see Section 3.2.1). This means that the usage of the auxiliary array seems to incur a per-reference processing overhead, but only when encountering new references that exceed the previously reserved capacity.

6.4 Memory Overhead of the Pre-Loaded Array Entries

The auxiliary GCr memory results show a notable asymmetry among the dynamic-array variants: `dyn_only` and `sep_dyn` use 308 MB in the single-object benchmark (157 % above `none`'s 120 MB), while `clear_path_dyn` uses 1 268 MB (roughly 10× the non-array baseline) and `all` uses 884 MB. This asymmetry directly reflects the per-entry struct size.

The `ZWeakRefArray` used in `dyn_only` stores one entry per discovered reference containing the reference's address, which is required for processing. The `clear_path_dyn` variant uses an extended entry that additionally stores pre-loaded values enabling the barrier-free clear path described in Section 3.3. The discrepancy between `clear_path_dyn` and `all` is explained by the implementation, where `all` does not require the reference address since that is only needed for other reference types, but it does still require the pre-loaded values for the clear path.

In the multi-object benchmark, the same pattern holds but the absolute differences are smaller since the reference population is smaller.

One interesting observation is that the `weak_fields` variant also follows this pattern in the single-object benchmark, using 446 MB of auxiliary GCr memory (271 % above `none`'s 120 MB) but not in the multi-object benchmark, where it uses 240 MB (18 % below none's 292 MB). This is odd since the weak_fields variant should follow the pattern in both cases since it is also backed by a dynamic array (WeakFieldArray), why it has a below-baseline auxiliary GCr memory usage in the multi-object benchmark is unclear. The most likely explanation is that the weak_fields variant's reduced number of objects on the heap leads to some reduction in the auxiliary GCr memory used for other GCr data structures, which is enough to offset the additional memory used by the WeakFieldArray in the multi-object benchmark but not in the single-object benchmark where the reference population is larger and the auxiliary array is larger.

6.5 Structural Advantage of Weak Fields

The weak_fields variant achieves improvements across all measured GC metrics because its advantage is structural rather than algorithmic: eliminating the WeakReference wrapper objects reduces the number of live objects the GCr must process in every phase of the collection cycle, not just in reference processing.

In the single-object benchmark, 20 million WeakReference objects are heap-allocated Java objects. Each such object must be individually processed during each phase of each GC cycle. In contrast, the weak_fields variant eliminates these objects entirely, so the GCr only processes the holder objects and the referents, which are already present in the heap for the Java application. This leads to a reduction in GC workload across the board, from marking to compaction, and from young generation to old generation.

The heap usage data confirms the claim that there are fewer objects allocated: weak_fields uses 806 MB versus 1 720 MB for all WeakReference variants in the single-object benchmark (53 % lower). In the multi-object benchmark the absolute effect is smaller because the 2 million references represent a smaller fraction of total heap activity, from 1 926 MB to 1 834 MB, a 5 % reduction.

6.6 Processing Mechanics of Weak Fields

On the processing side, weak_fields is mechanically similar to sep_dyn. Entries discovered during old-generation marking are stored in a per-worker ZWeakFieldArray, the direct analogue of the ZWeakRefArray used by the dynamic-array WeakReference variants. Like queue-less WeakReference objects in sep_dyn, weak-field entries carry no ReferenceQueue and therefore bypass the enqueue stage entirely: once the referent is found to be unreachable, the annotated field is zeroed without creating a pending-list entry. Due to the fields being writable, the CAS-based clear path is required to ensure that the field is not changed to an incorrect value. This is analogous to the clear path for queue-less WeakReference objects in both dyn_only and sep_dyn, neither of which employs the optimised clear path.

This structural parallel is reflected in the non-strong processing times. weak_fields achieves a 51 % reduction in the single-object benchmark, compared to sep_dyn's 42 % and dyn_only's 36 %. The somewhat better figure for weak_fields is consistent with the simpler per-entry structure: annotated fields are stored and cleared directly, without the WeakReference object header, queue field, next field, and discovered field.

The result does not reach the 81 % reductions of clear_path_dyn and all for the reason mentioned previously: the clear path for weak fields cannot be optimised in the same way as for queue-less WeakReference objects, so the per-entry processing logic still involves a CAS operation, which is not the case for clear_path_dyn and all.

6.7 Limitations

Several aspects of the evaluation warrant care in interpreting the results.

Benchmark representativeness. Both benchmarks are custom and synthetic microbenchmarks designed to isolate reference processing overhead. The single-object benchmark concentrates its entire weak-reference load into a single GC cycle, and the multi-object benchmark uses a fixed reference population with a regular promotion pattern. Neither benchmark captures the full diversity of real-world weak-reference usage, such as dynamic cache eviction, concurrent insertion and removal, or workloads with a mixture of queue-backed and queue-less references. The strong improvements observed on the non-strong processing metric may therefore represent an upper bound on what would be seen in a more heterogeneous production workload. Evaluating the variants on a real-world application benchmark would require either instrumenting an existing application or identifying a suitable open-source workload with measurable weak-reference usage. Java Flight Recorder (JFR) could assist such an evaluation by providing per-event GC timing data at low overhead, enabling the collection of the same metrics used here without requiring custom benchmark instrumentation. Ensuring measurement validity would also require monitoring system load throughout the run, to detect interference from competing processes on shared infrastructure.

Single hardware platform. All measurements were collected on a single UPPMAX cluster configuration (see Section 4.2.2). The results may not generalise to different CPU micro-architectures, Non-Uniform Memory Access (NUMA) configurations, or cache hierarchy sizes.

ReferenceHandler thread CPU time. Neither benchmark contains queue-backed references, so the `ReferenceHandler` thread had no pending-list entries to process regardless of variant. Its CPU usage was therefore not captured by any of the collected GC metrics. This means the full benefit of the skip-enqueue separation in a workload with a mixture of queue-backed and queue-less references cannot be quantified from the current data.

Correctness coverage. The correctness of the implementation was verified through the standard OpenJDK test suite and manual inspection, not through formal verification or an exhaustive stress suite for the weak-field annotation. In particular, since the OpenJDK test suite does not include any tests for the `@weak` annotation, the correctness of the `weak_fields` variant relies on the soundness of the implementation and the absence of crashes or assertion failures during the benchmark runs.

7. Related Work

The preceding chapters have established that, in the evaluated setting, the structural choice of whether to represent weak references through dedicated wrapper objects or through annotated fields has a larger impact on GC cost than pipeline-level optimisations. This chapter situates that finding within a broader design space by surveying how other language runtimes approach the same problem: providing weak reachability without preventing collection. The survey reveals recurrent design patterns, including a near-universal separation of weak reachability from cleanup notification, and a preference for probe-based semantics over callback-based notification in most mainstream systems. These cross-language comparisons provide context for understanding why Java's coupled `WeakReference` model is comparatively expensive for probe-only workloads, and for assessing where the weak-field mechanism explored in this thesis sits within the space of known alternatives.

Published work directly addressing the optimisation of ZGC's weak-reference pipeline is limited; the closest relevant material is therefore a combination of runtime documentation, implementation sources, and cross-language designs for weak semantics.

7.1 Java and the JVM

Java's reference API explicitly combines weak semantics with optional queue-based notification through the `ReferenceQueue` mechanism [7, 9, 10]. This is a richer contract than a bare weak pointer, because it supports both non-owning reachability and callback-style handoff to application code. The design is flexible, but it also means that the runtime has to maintain queue metadata and pending-list machinery even for workloads that only need liveness probing through `WeakReference.get()`.

The OpenJDK enhancement request JDK-8029205 documents this specific inefficiency by observing unnecessary pending-list traversal for references without a `ReferenceQueue` [13]. That issue does not itself present an implementation or an evaluation, but it establishes that the overhead targeted in this thesis is recognised within the OpenJDK ecosystem. In that sense, the present work contributes by taking a documented inefficiency and following it through implementation, benchmarking, and analysis in generational ZGC.

The broader garbage-collection literature also helps frame the design space. Jones et al. emphasise that GC performance depends not only on asymptotic algorithmic behaviour but also on object representation, locality, and the amount of metadata the runtime must maintain for each managed object [25]. That perspective is directly relevant here: the weak-field exploration is not merely an alternative API shape, but a test of whether moving weak semantics into ordinary object layout changes GC cost in a more fundamental way than improving the existing `WeakReference` pipeline.

7.2 Go

Go exposes weak semantics through the standard-library package `weak`, which provides weak pointers that do not by themselves keep referents alive [28]. This design enables non-owning associations in cache-like or metadata-oriented structures without introducing strong retention edges. Crucially, `weak.Pointer` is callback-less by design: the only way to determine whether a referent is still live is through an explicit call to `.Value()`, which returns `nil` once the referent has been collected [28]. Weak reachability and cleanup notification are entirely separate concerns in Go's model.

For lifecycle callbacks, Go documents finalisation support through runtime facilities such as `SetFinalizer` [29]. Importantly, Go's documentation emphasises that finalisers are not prompt resource-management mechanisms and should not be used as the primary correctness path for releasing resources.

Go therefore presents weak semantics as an explicit, callback-less non-owning reference mechanism and treats finalisation as an auxiliary facility with carefully constrained expectations, a deliberate separation rather than an oversight.

7.3 C++ and .NET

In C++, `std::weak_ptr` represents non-owning reachability within the reference-counted `shared_ptr/weak_ptr` ownership model [30]. Its semantics differ from Java's GCr-integrated weak references, but the design illustrates an important contrast: weak semantics can be expressed without a queue-based callback mechanism. The caller invokes `.lock()` to obtain a temporary strong pointer and checks whether it is null. There is no notification on clearing, and cleanup of associated resources is handled entirely through separate mechanisms such as Resource Acquisition Is Initialisation (RAII) destructors.

.NET exposes a `WeakReference<T>` API [31]. Like C++, `WeakReference<T>` is probe-based: callers use `TryGetTarget()` to check liveness, there is no notification callback when the referent is cleared.

7.4 Lua and Table-Level Weak Semantics

Lua offers weak semantics directly in a table representation: a table can be configured so that its keys, values, or both are weak [32]. This is conceptually close to the weak-field idea explored in this thesis. Rather than allocating a dedicated wrapper object for each weak relation, the language embeds weak semantics into an existing container or field representation. Lua's design therefore demonstrates that weak behaviour can be attached to ordinary object structure instead of being mediated by explicit weak-reference objects. As with Go's `weak.Pointer` and C++'s `std::weak_ptr`, there is no callback when a weak entry is cleared: the entry simply stops being present after the next collection cycle.

7.5 Python

Python's weak-semantics design, standardised in Python Enhancement Proposal (PEP) 205, emphasises two use cases: cache construction and reduction of lifecycle coupling in graph-like object structures [33]. The language provides explicit weak-reference objects, weak mappings, and callback hooks on invalidation. A key contribution of the PEP is its explicit treatment of invalidation strategy and the cost implications of global weak-reference management.

At the API level, Python exposes weak references through the `weakref` module and supports optional callbacks when referents become unreachable [34]. Like Java, Python combines weak semantics and lifecycle notification into a single object. The mechanism differs, however: where Java routes cleared references through a `ReferenceQueue` consumed by a daemon thread, Python invokes a caller-supplied callback function directly at collection time.

7.6 Synthesis

Taken together, these systems expose three recurring design patterns. First, weak semantics is consistently treated as a deliberate programming abstraction rather than as an invisible optimisation.

Second, in most ecosystems, weak reachability and cleanup notification are treated as entirely separate concerns. Go's `weak.Pointer`, C++'s `std::weak_ptr`, .NET's `WeakReference<T>`, and Lua's weak table entries all provide callback-less weak semantics, where the caller probes for liveness rather than registering for a notification. Cleanup callbacks, where needed at all, are provided through separate and explicitly more complex mechanisms. Java's `WeakReference` and Python's `weakref` both couple weak semantics and lifecycle notification into a single object; they differ in delivery mechanism (queue-based versus direct callback), but in both cases the runtime bears notification overhead for every weak reference regardless of whether a callback or queue is actually registered. Third, runtimes differ in whether weak reachability is encoded through wrapper objects, integrated container or field semantics, or ownership metadata – a choice with significant consequences for GC pressure.

Against that background, this thesis occupies a specific niche. The queue-less `WeakReference` optimisations aim to reduce the overhead of the callback-less use case while preserving Java's existing API and object model: for applications that already choose the one-argument `WeakReference` constructor, the optimised pipeline makes the runtime cost of weak reachability closer to the lean, probe-only model that Go, C++, and .NET have long provided. The weak-field mechanism explores a more structural alternative, closer in spirit to Lua's embedded weak semantics than to Java's traditional wrapper-object approach.

8. Conclusion

Four mechanisms were designed and implemented within generational ZGC to reduce weak-reference overhead: a skip-enqueue separation routing queue-less weak references away from the pending-list machinery; a dynamic array replacing the intrusive linked list used during discovery; a specialised clear path eliminating unnecessary CAS operations for queue-less old-generation references; and a weak-field annotation allowing ordinary object fields to carry weak semantics without a `WeakReference` wrapper. The three pipeline optimisations were combined into eight variants alongside the separate `weak_fields` variant, benchmarked on two custom synthetic workloads under controlled conditions on a supercomputer cluster, and their GC timing and memory outcomes were analysed in Chapter 5 and discussed in depth in Chapter 6. Within this evaluated setting, queue-less synthetic workloads on a single hardware platform, the findings reveal a consistent pattern: weak-reference overhead behaves more like a representation problem than a pipeline problem. In these workloads, reducing overhead meaningfully appears to require reconsidering how weak semantics is encoded in the language, not merely how the resulting objects are processed once they have been allocated. Whether this pattern holds in production workloads with diverse reference usage and across different hardware platforms remains a direction for future work.

8.1 Weak Reference Pipeline Optimisations

Addressing Research Question 1, the pipeline optimisations demonstrate that the non-strong reference-processing phase of ZGC can be substantially accelerated. The most effective combinations, `clear_path_dyn` and `all`, achieved an 81 % reduction in median non-strong processing time in the single-object benchmark. This improvement is driven by a superadditive interaction between the dynamic array and the specialised clear path: the dynamic array enables contiguous, cache-friendly traversal and pre-loading of referent data during discovery, while the specialised clear path exploits those pre-loaded values to eliminate a load barrier and replace a CAS with a direct store. Each mechanism removes a bottleneck that persists when the other is applied in isolation, together they deliver a reduction far exceeding the sum of their individual contributions (Section 6.1).

This large improvement in the targeted metric did not translate into a significant reduction in total collection time. Even in a benchmark deliberately constructed to maximise the proportion of GC time spent on weak-reference processing, the `all` variant (the best-balanced pipeline configuration) reduced median major collection time by only 8 % against the baseline. That figure is an upper bound on what the pipeline optimisations can achieve: the benchmark was engineered to favour them, and real-world applications, whose heaps contain a far more diverse mix of object types and GC phases, would see proportionally smaller gains.

Two secondary findings complement the answer to Research Question 1. First, the skip-enqueue separation contributed least of the three optimisations in isolation: the `sep_only` variant reduced single-object median non-strong processing time by only 5 % and showed no measurable effect in the multi-object benchmark, indicating that the GC-side cost of pending-list construction is not a meaningful bottleneck under these workloads. The effect on the `ReferenceHandler` thread was not evaluated: since neither benchmark contains queue-backed references, the thread's CPU time was not measured and is not reflected in any of the collected GCr metrics. In a mixed workload containing both queue-backed and queue-less references, routing queue-less references away from the thread would be expected to reduce the volume of pending-list entries it must traverse, potentially lowering latency for queue-backed notifications. Whether that benefit is significant in practice remains an open question that the current evaluation cannot answer. Second, the dynamic-array variants incur substantially higher auxiliary GCr memory: `clear_path_dyn` consumed up to ten times the baseline auxiliary GCr memory in the single-object benchmark. Adding the skip-enqueue separation, as in `all`, eliminates the need to store the reference address in each array entry and reduces auxiliary GCr memory by approximately 30 % relative to `clear_path_dyn`, while achieving the same non-strong processing time reduction. Even so, `all` still consumes almost seven times the baseline auxiliary GCr memory in the single-object benchmark (884 MB versus 120 MB for `none`). This memory overhead must be weighed against the modest major-collection-time improvements when assessing whether these optimisations merit integration in practice.

8.2 Weak Fields as a Structural Alternative

Addressing Research Question 2, the `weak_fields` variant avoids the issue of weak reference processing not being a significant part of the total GC time in Section 8.1 by reducing the workload in phases that the optimisations do not. In the single-object benchmark, `weak_fields` reduced median major collection time by 41 % against the unmodified baseline, far exceeding the 8 % achieved by the `all` variant on the same metric, and reduced Java heap occupancy by 53 %. In the multi-object benchmark the gains were more modest (28 % and 5 % respectively), consistent with the smaller proportion of total heap activity attributable to the reference population. This advantage was unambiguous across every measured phase, even those in Appendix D.

The mechanism behind this advantage is structural rather than algorithmic. Eliminating `WeakReference` wrapper objects reduces the number of live objects the GCr must process during every phase of every collection cycle. The pipeline optimisations accelerate what happens to those objects in the reference processor, the weak-field approach removes the objects entirely. No pipeline improvement can recover this saving, because the wrapper objects and their associated GC costs persist regardless of how efficiently they are subsequently processed.

This finding aligns with the design choices prevalent across the broader language ecosystem. As documented in Chapter 7, Go's `weak.Pointer`, C++'s `std::weak_ptr`, .NET's `WeakReference<T>`, and Lua's weak table entries all provide callback-less, probe-based weak semantics, treating weak reachability and cleanup notification as entirely separate concerns. Java's `WeakReference` is notable for coupling weak semantics and queue-based lifecycle notification into a single object and processing pipeline, imposing their combined overhead on all callers regardless of whether the callback mechanism is ever used, Python's `weakref` takes a similar approach with direct callbacks rather than a queue. The `@weak` annotation explored here is closest in spirit to Lua's field-level weak semantics, embedding weakness directly into an existing field as a structural property of the containing object rather than an additional layer of indirection, and in doing so brings the runtime cost of callback-less weak semantics in Java closer to the leaner model other languages have long provided.

8.3 Recommendations and Implications

For the common case of probe-based weak semantics (where no queue-backed lifecycle callback is needed, and which Chapter 7 establishes as the standard model across most mainstream managed and systems languages), the weak-field approach is both more effective and more architecturally coherent than pipeline optimisation. This advantage does not come for free, however: as shown in Table 4.2, the `weak_fields` implementation touches 27 files and adds a net 683 lines across the compiler front-end, class-file parser, interpreter, JIT compilers, JVMTI, and GCr, compared to at most 10 files and 371 net lines for the most complex `WeakReference` pipeline variant. Whether that implementation cost is acceptable depends on the use case and the extent to which the mechanism is hardened and generalised beyond its current prototype state.

For use cases that require queue-based notification, the full `WeakReference` API remains necessary. Among the pipeline variants, `all` offers the best balance: it matches `clear_path_dyn`'s 81 % reduction in non-strong processing time while using approximately 30 % less auxiliary GCr memory, because the skip-enqueue separation eliminates the reference address field from each array entry. Whether the remaining memory overhead and implementation complexity are justified is a question best settled against a production workload rather than a synthetic benchmark.

The broader implication, at least within the evaluated synthetic setting, is that optimisation efforts directed at the reference-processing pipeline face a hard ceiling set by the weight of wrapper objects distributed throughout the heap. An 81 % reduction in a phase that accounts for just 14.7 % of total GC work (see **??**) yields, at best, an 8 % reduction in collection time that was measured under conditions constructed to maximise it. Shifting effort toward how weak semantics is represented, rather than how the resulting representation is processed, offers a potentially better path to meaningful improvement. Though this conclusion is drawn from queue-less, synthetic workloads and should be revisited against production benchmarks with more diverse weak-reference usage.

8.4 Future Work

Several directions emerge naturally from this work.

Production workload evaluation. The most immediate extension would be to evaluate the optimised variants on real-world applications with diverse weak-reference usage patterns. A production-based benchmark would reveal whether the 81 % non-strong processing improvement translates into meaningful end-to-end latency improvements in a multi-threaded workload where reference processing competes with other GC work. Java Flight Recorder (JFR) is well suited to such an evaluation: it captures per-event GC timing at low overhead without requiring custom benchmark instrumentation, enabling the same metrics used here to be collected from unmodified production applications. Monitoring system load throughout any such run would be essential to detect and exclude measurements affected by interference from co-located workloads.

ReferenceHandler thread load in mixed workloads. The benchmarks contain only queue-less references, so the benefit of routing those references away from the `ReferenceHandler` thread was never observable in the collected metrics. Measuring `ReferenceHandler` CPU time and queue-backed reference notification latency in a workload that mixes queue-backed and queue-less references would establish whether the skip-enqueue separation provides meaningful latency improvements for applications that use both patterns concurrently.

Extending the clear-path optimisation. The specialised clear path is currently limited to old-generation `WeakReference` objects without a `ReferenceQueue`. Extending it to cover `SoftReference` and `PhantomReference` objects and `WeakReference` objects with queues could yield additional reductions in non-strong processing time. The safety argument for omitting the CAS depends on the constrained update model described in Section 3.3, and it would need to be re-examined for each additional reference type.

Upstream integration. The skip-enqueue separation, the dynamic array, and the clear-path optimisation are all independent changes with well-defined correctness arguments. Submitting one or more of them to the OpenJDK project for review would allow the broader community to evaluate the trade-offs and potentially integrate the improvements that pass the upstream quality bar. The JDK-8029205 enhancement request provides a natural entry point for such a submission.

Fully testing and integrating the weak-field variant. The `weak_fields` variant is a proof-of-concept implementation that demonstrates the potential of using weak fields as an alternative to `WeakReference` objects. It is not fully tested or integrated into the OpenJDK codebase, and its correctness relies on the absence of crashes or assertion failures during the benchmark runs. A future direction would be to develop a comprehensive test suite for the `@weak` annotation, formally verify the implementation, and submit it for upstream review.

References

[1] Oracle, “Available collectors.” https://docs.oracle.com/en/java/javase/25/gctuning/available-collectors.html, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-02-24.

[2] Oracle, “Garbage collector implementation.” https://docs.oracle.com/en/java/javase/25/gctuning/garbage-collector-implementation.html, 2025. Java Platform, Standard Edition 25 Garbage Collection Tuning Guide. Accessed: 2026-04-07.

[3] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” https://openjdk.org/jeps/333, 2018. Accessed: 2026-03-12.

[4] OpenJDK, “Jep 439: Generational zgc.” https://openjdk.org/jeps/439, 2023. Accessed: 2026-03-12.

[5] E. Österlund, “JVMLS – generational ZGC and beyond.” https://inside.java/2023/08/31/generational-zgc-and-beyond/, 2023. Accessed: 2026-05-25.

[6] OpenJDK, “Jep 377: Zgc: A scalable low-latency garbage collector (production).” https://openjdk.org/jeps/377, 2020. Accessed: 2026-03-12.

[7] Oracle, “java.lang.ref (java se 25).” https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html, 2025. Accessed: 2026-03-12.

[8] OpenJDK, “Openjdk source: java.lang.ref.weakreference.java.” https://github.com/openjdk/jdk/blob/61b2508/src/java.base/share/classes/java/lang/ref/WeakReference.java, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[9] OpenJDK, “Openjdk source: java.lang.ref.reference.java.” https://github.com/openjdk/jdk/blob/61b2508/src/java.base/share/classes/java/lang/ref/Reference.java, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[10] OpenJDK, “Openjdk source: java.lang.ref.referencequeue.java.” https://github.com/openjdk/jdk/blob/61b2508/src/java.base/share/classes/java/lang/ref/ReferenceQueue.java, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[11] OpenJDK, “Openjdk source: java.util.weakhashmap.java.” https://github.com/openjdk/jdk/blob/61b2508/src/java.base/share/classes/java/util/WeakHashMap.java, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[12] OpenJDK, “Openjdk source: Zgc reference processor (zreferenceprocessor.cpp).” https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/z/zReferenceProcessor.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[13] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.” https://bugs.openjdk.org/browse/JDK-8029205, n.d. Accessed: 2026-02-24.

[14] OpenJDK, “Openjdk source: Shared reference processor (referenceprocessor.cpp).” https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/shared/referenceProcessor.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[15] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007. Accessed: 2026-04-07.

[16] Oracle, “Java hotspot vm.” https://www.oracle.com/java/technologies/javase/javase-core-technologies-apis.html, 2025. Accessed: 2026-05-12.

[17] OpenJDK, “Hotspot jvm architecture overview.” https://openjdk.org/groups/hotspot/, 2025. Accessed: 2026-05-12.

[18] OpenJDK, “Openjdk source: Zgc barrier implementation (zbarrier.cpp).” https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/z/zBarrier.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[19] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981. Accessed: 2026-02-24.

[20] OpenJDK, “Openjdk source: Zgc address representation (zaddress.hpp).” https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/z/zAddress.hpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[21] OpenJDK, "Openjdk source: Zgc worker infrastructure (zworkers.cpp)." https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/z/zWorkers.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[22] OpenJDK, "Openjdk source: Access decorators (accessdecorators.hpp)." https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/oops/accessDecorators.hpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[23] OpenJDK, "Openjdk source: Reference policy (referencepolicy.cpp)." https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/shared/referencePolicy.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[24] OpenJDK, "Openjdk source: Generational zgc implementation (zgeneration.cpp)." https://github.com/openjdk/jdk/blob/61b2508/src/hotspot/share/gc/z/zGeneration.cpp, 2026. GitHub permalink, commit 61b2508 (upstream OpenJDK, 2026-04-09).

[25] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.

[26] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd ed., 1997. Section 3.4.2, Algorithm P.

[27] F. Hammarberg, "Benchmark data: Optimising weak reference processing in the jvm z garbage collector." Zenodo, 2026. doi: 10.5281/zenodo.20445473.

[28] The Go Authors, "weak package – go standard library documentation." https://pkg.go.dev/weak, 2026. Accessed: 2026-04-22.

[29] The Go Authors, "runtime.setfinalizer – go standard library documentation." https://pkg.go.dev/runtime#SetFinalizer, 2026. Accessed: 2026-04-22.

[30] cppreference.com contributors, "std::weak_ptr – cppreference.com." https://en.cppreference.com/w/cpp/memory/weak_ptr, 2026. Accessed: 2026-04-22.

[31] Microsoft, "Weakreference class (.net)." https://learn.microsoft.com/en-us/dotnet/api/system.weakreference?view=net-9.0, 2026. Accessed: 2026-04-22.

[32] R. Ierusalimschy, L. H. de Figueiredo, and W. Celes, "Lua 5.4 reference manual." https://www.lua.org/manual/5.4/, 2025. Sections 2.5.3 and 2.5.4. Accessed: 2026-04-22.

[33] F. L. Drake, "Pep 205 – weak references." https://peps.python.org/pep-0205/, 2000. Python Enhancement Proposal. Accessed: 2026-04-22.

[34] Python Software Foundation, "weakref – weak references (python 3 documentation)." https://docs.python.org/3/library/weakref.html, 2026. Accessed: 2026-04-22.

Appendix A.
Usage of GenAI Tools

The Generative Artificial Intelligence (GenAI) tool GitHub Copilot was used as support during this thesis. It was used under a student licence via Uppsala University. The tool was used to accelerate practical work and improve writing quality. All final technical decisions, implementation choices, and evaluations remained under the author's control.

A.1 Use During Implementation

GenAI was used as support during coding of the implementation, primarily for discussing alternative implementation approaches, suggesting code edits through auto-complete, and implementing refactorings of already written code.

A.2 Use for Tests and Scripts

GenAI was used to completely generate several test files and automation scripts used during this thesis, including scripts for benchmarking workflows and result-processing utilities. These generated artefacts were reviewed and validated through repeated execution.

A.3 Use During Writing

GenAI was used as writing support in four main ways:

- Suggesting rephrasing for clarity, tone, and structure.
- Suggesting and correcting LaTeX formatting and fixing compilation or syntax errors.
- Generating first drafts of information-dense sections, especially where the text was based directly on OpenJDK source code and implementation details.
- Writing the first draft of the abstract with the context of the entire finished report.

All GenAI-generated text was manually edited and verified against primary sources before inclusion in the final report.

Appendix B.
Summary Statistics

This appendix provides numerical summary statistics (median, mean, and interquartile range) for all configuration variants and the key metrics discussed in Chapter 5. All tables are produced automatically by the `parse_gc_stats.py` script from the benchmark CSV data. The single-object benchmark values are per-run maxima, the multi-object benchmark values are per-run averages. Each cell is based on $n = 250$ independent benchmark runs.

B.1 Major Collection Time

Table B.1: *Summary statistics for **Major Collection** across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	6958.1	7088.1	1595.6	982.0	991.7	92.6
`clear_path_only`	6963.0	7131.0	1493.4	996.4	1002.8	112.5
`sep_only`	7044.1	7195.2	1486.3	985.1	990.2	116.2
`dyn_only`	6733.4	6909.2	1495.2	965.0	969.7	109.1
`clear_path_sep`	6902.6	7131.8	1353.9	1001.5	1010.8	134.4
`clear_path_dyn`	6399.0	6565.7	1393.5	971.6	976.7	115.5
`sep_dyn`	6912.6	7026.7	1458.7	987.8	995.2	117.7
`all`	6376.9	6506.5	1419.3	966.7	969.9	136.7
`weak_fields`	4135.9	4220.0	484.9	708.3	705.7	93.2

B.2 Old-Generation Collection Time

Table B.2: *Summary statistics for **Old Generation** across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	3630.8	3637.7	683.2	809.2	813.4	80.3
`clear_path_only`	3600.1	3650.8	626.7	821.8	820.7	110.7
`sep_only`	3675.5	3708.3	603.9	813.0	812.0	100.4
`dyn_only`	3377.0	3417.1	871.5	796.7	797.1	97.0
`clear_path_sep`	3631.1	3688.2	599.3	830.6	829.6	122.7
`clear_path_dyn`	3003.3	3084.6	719.5	796.3	794.7	117.3
`sep_dyn`	3392.5	3426.8	761.9	817.4	813.4	106.6
`all`	2992.7	3069.4	669.7	792.2	792.8	122.5
`weak_fields`	2295.0	2301.0	281.6	554.3	558.8	82.0

B.3 Old Phase: Concurrent Mark

Table B.3: *Summary statistics for **Old Phase: Concurrent Mark** across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	2575.3	2607.4	607.7	578.1	592.2	66.6
`clear_path_only`	2621.4	2687.4	584.9	588.0	599.3	89.4
`sep_only`	2662.3	2713.2	564.9	580.1	589.9	83.1
`dyn_only`	2644.8	2721.0	810.5	570.7	582.8	81.1
`clear_path_sep`	2660.1	2721.7	560.3	590.9	608.9	99.1
`clear_path_dyn`	2780.2	2834.9	661.4	588.1	598.0	96.2
`sep_dyn`	2731.7	2791.0	666.2	584.1	594.7	76.1
`all`	2750.2	2825.3	624.0	577.8	594.0	94.3
`weak_fields`	1782.4	1766.7	272.6	399.0	401.7	46.7

B.4 Non-Strong Processing Time

Table B.4: *Summary statistics for **Non-Strong Processing** across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	996.9	1000.4	75.9	44.1	44.1	9.4
`clear_path_only`	923.7	933.8	53.1	44.5	44.1	8.0
`sep_only`	943.8	965.4	73.8	44.1	44.2	9.6
`dyn_only`	639.1	666.0	100.7	36.3	36.4	6.4
`clear_path_sep`	923.9	936.9	52.6	40.5	39.9	8.0
`clear_path_dyn`	187.9	220.2	11.6	18.8	18.8	4.2
`sep_dyn`	576.9	606.1	75.9	35.5	35.9	7.1
`all`	184.4	214.6	22.8	18.7	18.9	4.5
`weak_fields`	484.6	531.8	8.4	35.8	35.5	8.2

B.5 Old Phase: Concurrent Relocate

Table B.5: *Summary statistics for **Old Phase: Concurrent Relocate** across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	0.0	0.0	0.0	155.6	164.6	36.3
`clear_path_only`	0.0	0.1	0.0	148.2	164.3	46.9
`sep_only`	0.0	0.1	0.0	148.6	165.0	45.5
`dyn_only`	0.0	0.1	0.0	155.3	165.0	34.3
`clear_path_sep`	0.0	0.1	0.0	149.6	168.0	46.7
`clear_path_dyn`	0.0	0.1	0.0	148.5	164.9	46.0
`sep_dyn`	0.0	0.1	0.0	160.8	169.7	46.0
`all`	0.0	0.0	0.0	148.8	166.7	46.3

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
weak_fields	1.6	1.2	1.9	103.7	116.5	33.8

B.6 Young Generation (Promote All)

Table B.6: *Summary statistics for* ***Young Generation (Promote All)*** *across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
none	3201.3	3446.1	1516.6	163.5	175.5	50.7
clear_path_only	3219.0	3475.7	1210.2	164.0	179.3	57.5
sep_only	3245.8	3482.6	1571.1	163.8	175.4	48.7
dyn_only	3233.1	3487.7	1688.8	155.7	169.8	45.0
clear_path_sep	3168.9	3439.2	1312.2	164.0	178.5	55.9
clear_path_dyn	3245.0	3476.6	1404.0	161.5	179.1	62.0
sep_dyn	3438.9	3595.6	1622.4	163.9	179.1	53.0
all	3185.2	3432.8	1395.7	161.2	174.3	47.9
weak_fields	1777.3	1915.7	241.2	137.0	144.6	16.4

B.7 Young Phase: Concurrent Mark

Table B.7: *Summary statistics for* ***Young Phase: Concurrent Mark*** *across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
none	2605.4	2851.6	1523.8	56.1	62.0	25.5
clear_path_only	2623.1	2874.9	1198.1	56.6	64.1	28.7
sep_only	2645.0	2883.8	1574.1	56.2	62.0	24.2
dyn_only	2631.1	2888.7	1691.1	52.5	59.2	22.2
clear_path_sep	2566.5	2837.2	1317.8	56.7	63.7	28.0
clear_path_dyn	2642.4	2873.8	1394.3	55.1	63.9	31.0
sep_dyn	2837.9	2997.3	1623.3	56.3	63.8	26.8
all	2583.7	2831.1	1409.3	54.9	61.4	24.1
weak_fields	1493.5	1631.2	238.6	47.9	51.4	8.3

B.8 Young Phase: Concurrent Relocate

Table B.8: *Summary statistics for* ***Young Phase: Concurrent Relocate*** *across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	395.7	396.8	1.6	13.9	13.9	0.1
`clear_path_only`	400.7	404.5	7.9	13.7	13.7	0.2
`sep_only`	394.5	396.2	2.3	13.8	13.8	0.2
`dyn_only`	399.6	402.0	4.3	13.9	13.9	0.2
`clear_path_sep`	397.8	399.0	5.6	13.6	13.6	0.2
`clear_path_dyn`	405.4	406.4	8.9	13.7	13.7	0.2
`sep_dyn`	394.4	396.2	4.3	13.8	13.8	0.2
`all`	404.2	405.2	10.2	13.9	13.9	0.1
`weak_fields`	198.2	199.0	1.2	11.4	11.4	0.1

B.9 Concurrent Mark Follow Time

Table B.9: *Summary statistics for* ***Concurrent Mark Follow*** *across all variants. Values in ms; single-object benchmark uses per-run maxima, multi-object uses per-run averages ($n = 250$ per variant).*

	Single (ms)			Multi (ms)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	2575.2	2607.3	607.7	578.1	592.1	66.6
`clear_path_only`	2621.3	2687.3	584.8	587.9	599.3	89.4
`sep_only`	2662.2	2713.1	564.9	580.1	589.9	83.1
`dyn_only`	2644.7	2721.0	810.5	570.7	582.8	81.1
`clear_path_sep`	2660.0	2721.6	560.3	590.8	608.8	99.1
`clear_path_dyn`	2780.1	2834.8	661.4	588.0	597.9	96.2
`sep_dyn`	2731.7	2790.9	666.2	584.1	594.7	76.2
`all`	2750.1	2825.2	624.0	577.7	593.9	94.3
`weak_fields`	1782.3	1766.7	272.6	398.9	401.6	46.7

B.10 auxiliary GCr memory

Table B.10: *Summary statistics for* ***Auxiliary GCr Memory*** *per-run maximum across all variants. Values in MB ($n = 250$ per variant).*

	Single (MB)			Multi (MB)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	120	120	2	292	294	4
`clear_path_only`	121	121	2	293	294	2
`sep_only`	120	120	2	293	294	3
`dyn_only`	308	310	0	305	305	5
`clear_path_sep`	121	121	2	292	294	2
`clear_path_dyn`	1268	1303	3	363	357	23
`sep_dyn`	308	310	0	304	305	6
`all`	884	901	3	340	337	14
`weak_fields`	446	448	5	240	239	6

B.11 Java Heap

Table B.11: *Summary statistics for **Java Heap** per-run maximum across all variants. Values in MB (n = 250 per variant).*

	Single (MB)			Multi (MB)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	1720	1721	2	1926	1927	2
`clear_path_only`	1720	1721	2	1928	1929	2
`sep_only`	1722	1721	2	1928	1928	2
`dyn_only`	1720	1721	2	1928	1928	2
`clear_path_sep`	1721	1721	2	1928	1929	2
`clear_path_dyn`	1720	1721	2	1928	1928	2
`sep_dyn`	1720	1721	2	1928	1928	2
`all`	1720	1720	2	1926	1927	2
`weak_fields`	806	805	2	1834	1834	2

B.12 Total Process RSS

Table B.12: *Summary statistics for **RSS** per-run maximum across all variants. Values in MB (n = 250 per variant).*

	Single (MB)			Multi (MB)		
Variant	Median	Mean	IQR	Median	Mean	IQR
`none`	1901	1903	5	2407	2408	6
`clear_path_only`	1903	1905	6	2409	2410	8
`sep_only`	1902	1904	5	2409	2409	8
`dyn_only`	2107	2108	4	2423	2424	11
`clear_path_sep`	1904	1905	7	2410	2410	8
`clear_path_dyn`	2856	2861	43	2487	2492	30
`sep_dyn`	2107	2109	3	2425	2425	10
`all`	2556	2552	16	2466	2468	27
`weak_fields`	1260	1260	3	2255	2256	13

Appendix C. Per-Variant Patch Details

The following tables list every modified or added file in each patch variant, together with its insertion and deletion counts relative to the upstream OpenJDK `master` branch. New files (not present in the upstream source) are listed with their total line count and no deletion count. For variants that use the shared base patch, its files are listed first under a "Base patch" heading within each table. The `dyn_only` and `weak_fields` variants do not include the base patch.

C.1 `all`

Table C.1: *Files changed in the* `all` *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
`gc/shared/collectedHeap.hpp`	+4	−0
`gc/z/zCollectedHeap.cpp`	+9	−0
`gc/z/zCollectedHeap.hpp`	+1	−0
`gc/z/zGeneration.hpp`	+1	−0
`gc/z/zGeneration.inline.hpp`	+4	−0
`gc/z/zStat.cpp`	+14	−8
`gc/z/zStat.hpp`	+2	−1
`runtime/threads.cpp`	+4	−8
Variant-specific patch		
`gc/z/zReferenceProcessor.cpp`	+223	−32
`gc/z/zReferenceProcessor.hpp`	+31	−16
`gc/z/zWeakRefArray.hpp` (new)	+143	—
Total	**+436**	**−65**

C.2 `clear_path_dyn`

Table C.2: *Files changed in the* `clear_path_dyn` *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
`gc/shared/collectedHeap.hpp`	+4	−0
`gc/z/zCollectedHeap.cpp`	+9	−0
`gc/z/zCollectedHeap.hpp`	+1	−0
`gc/z/zGeneration.hpp`	+1	−0
`gc/z/zGeneration.inline.hpp`	+4	−0
`gc/z/zStat.cpp`	+14	−8
`gc/z/zStat.hpp`	+2	−1

File	+	−
runtime/threads.cpp	+4	−8
Variant-specific patch		
gc/z/zReferenceProcessor.cpp	+224	−53
gc/z/zReferenceProcessor.hpp	+30	−17
gc/z/zWeakRefArray.hpp (new)	+155	—
Total	**+448**	**−87**

C.3 clear_path_only

Table C.3: *Files changed in the* clear_path_only *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
Variant-specific patch		
gc/z/zReferenceProcessor.cpp	+172	−22
gc/z/zReferenceProcessor.hpp	+22	−5
Total	**+233**	**−44**

C.4 clear_path_sep

Table C.4: *Files changed in the* clear_path_sep *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
gc/shared/collectedHeap.hpp	+4	−0
gc/z/zCollectedHeap.cpp	+9	−0
gc/z/zCollectedHeap.hpp	+1	−0
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zStat.cpp	+14	−8
gc/z/zStat.hpp	+2	−1
runtime/threads.cpp	+4	−8
Variant-specific patch		
gc/z/zReferenceProcessor.cpp	+193	−21
gc/z/zReferenceProcessor.hpp	+24	−5
Total	**+256**	**−43**

C.5 `dyn_only`

Table C.5: *Files changed in the* `dyn_only` *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
`gc/z/zReferenceProcessor.cpp`	+44	−28
`gc/z/zReferenceProcessor.hpp`	+17	−15
`gc/z/zWeakRefArray.hpp` (new)	+149	—
Total	**+210**	**−43**

C.6 `sep_dyn`

Table C.6: *Files changed in the* `sep_dyn` *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
`gc/shared/collectedHeap.hpp`	+4	−0
`gc/z/zCollectedHeap.cpp`	+9	−0
`gc/z/zCollectedHeap.hpp`	+1	−0
`gc/z/zGeneration.hpp`	+1	−0
`gc/z/zGeneration.inline.hpp`	+4	−0
`gc/z/zStat.cpp`	+14	−8
`gc/z/zStat.hpp`	+2	−1
`runtime/threads.cpp`	+4	−8
Variant-specific patch		
`gc/z/zReferenceProcessor.cpp`	+182	−27
`gc/z/zReferenceProcessor.hpp`	+28	−14
`gc/z/zWeakRefArray.hpp` (new)	+149	—
Total	**+398**	**−58**

C.7 `sep_only`

Table C.7: *Files changed in the* `sep_only` *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
Base patch		
`gc/shared/collectedHeap.hpp`	+4	−0
`gc/z/zCollectedHeap.cpp`	+9	−0
`gc/z/zCollectedHeap.hpp`	+1	−0
`gc/z/zGeneration.hpp`	+1	−0
`gc/z/zGeneration.inline.hpp`	+4	−0
`gc/z/zStat.cpp`	+14	−8
`gc/z/zStat.hpp`	+2	−1
`runtime/threads.cpp`	+4	−8
Variant-specific patch		
`gc/z/zReferenceProcessor.cpp`	+157	−20

File	+	−
gc/z/zReferenceProcessor.hpp	+17	−5
Total	**+213**	**−42**

C.8 weak_fields

Table C.8: *Files changed in the* weak_fields *variant, with insertion and deletion counts relative to upstream OpenJDK.*

File	+	−
cpu/x86/templateTable_x86.cpp	+30	−9
cpu/x86/templateTable_x86.hpp	+1	−1
c1/c1_LIRGenerator.cpp	+5	−0
c1/c1_Runtime1.cpp	+12	−3
ci/ciField.cpp	+1	−1
ci/ciField.hpp	+1	−0
ci/ciFlags.hpp	+5	−3
classfile/classFileParser.cpp	+36	−17
classfile/vmSymbols.hpp	+1	−1
gc/shared/accessBarrierSupport.cpp	+25	−6
gc/z/zBarrier.cpp	+55	−6
gc/z/zBarrier.hpp	+2	−1
gc/z/zGeneration.hpp	+1	−0
gc/z/zGeneration.inline.hpp	+4	−0
gc/z/zMark.cpp	+70	−20
gc/z/zReferenceProcessor.cpp	+172	−2
gc/z/zReferenceProcessor.hpp	+20	−9
gc/z/zStat.cpp	+13	−7
gc/z/zStat.hpp	+2	−1
gc/z/zWeakFieldArray.hpp (new)	+149	—
oops/fieldInfo.hpp	+3	−0
oops/resolvedFieldEntry.cpp	+2	−1
oops/resolvedFieldEntry.hpp	+10	−5
opto/library_call.cpp	+216	−146
opto/parse3.cpp	+12	−0
prims/jvmtiTagMap.cpp	+41	−10
runtime/fieldDescriptor.hpp	+2	−2
services/heapDumper.cpp	+14	−6
java.base/.../ref/weak.java (new)	+35	—
Total	**+940**	**−257**

Appendix D.
Additional Benchmark Plots

This appendix contains the remaining GC timing and memory plots from the benchmark evaluation. The metrics shown here (Old Phase: Concurrent Relocate, Young Phase: Concurrent Mark, and Young Phase: Concurrent Relocate) were not included in the main evaluation chapter as they are less central to the optimised reference processing pipeline. Memory maximum distributions are included here to complement the median percentage-difference histograms shown in the main chapter.

Additional GC timing plots are shown for Old Phase: Concurrent Relocate (Figures D.1 and D.2), Young Phase: Concurrent Mark (Figures D.3 and D.4), and Young Phase: Concurrent Relocate (Figures D.5 and D.6) for both benchmark families. Memory per-run maximum distributions are shown for auxiliary GCr memory, Java heap, and total process RSS for both the single-object and multi-object benchmarks (Figures D.7 and D.8).

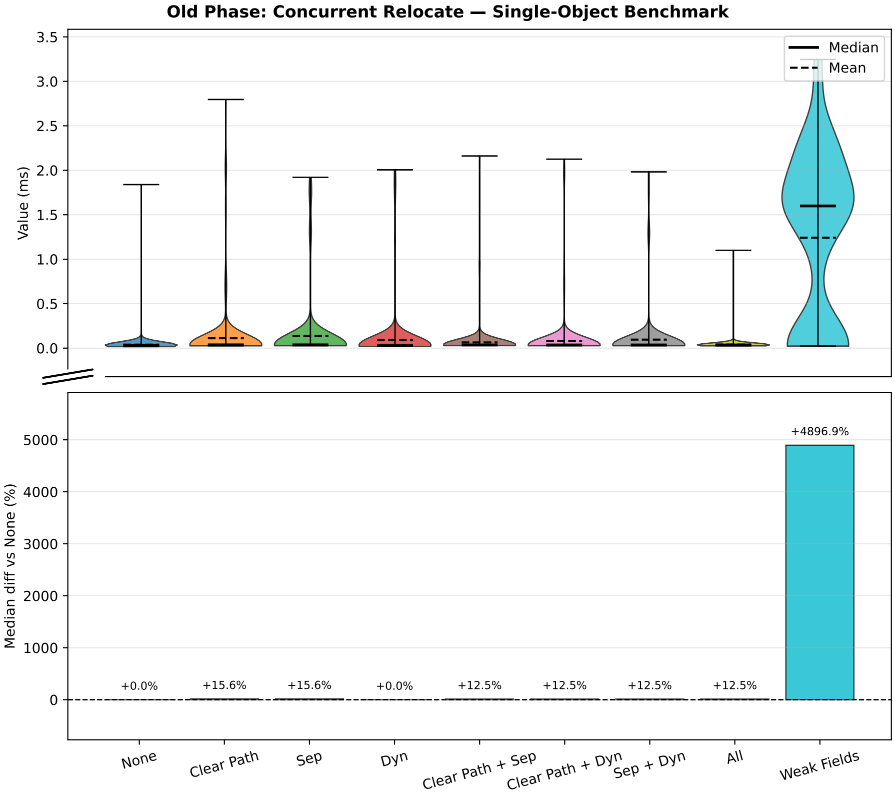

Figure D.1. Old Phase: Concurrent Relocate timing, **single-object benchmark** (per-run maxima). Top: distribution, bottom: median % difference vs baseline.

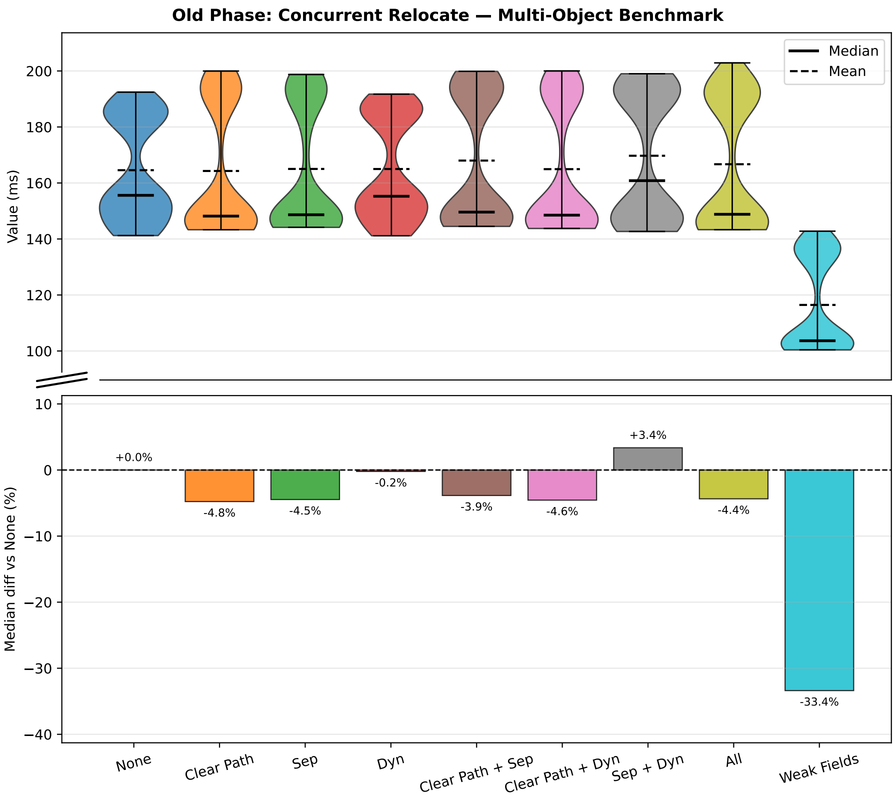

Figure D.2. Old Phase: Concurrent Relocate timing, **multi-object benchmark** (per-run averages). Top: distribution, bottom: median % difference vs baseline.

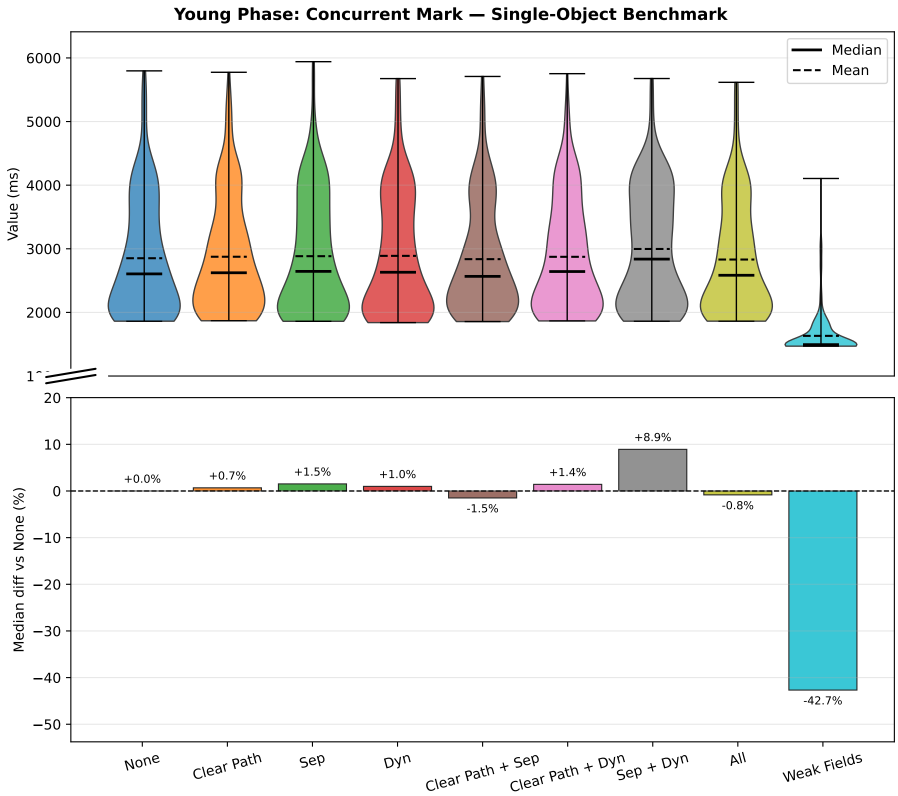

Figure D.3. Young Phase: Concurrent Mark timing, **single-object benchmark** (per-run maxima). Top: distribution, bottom: median % difference vs baseline.

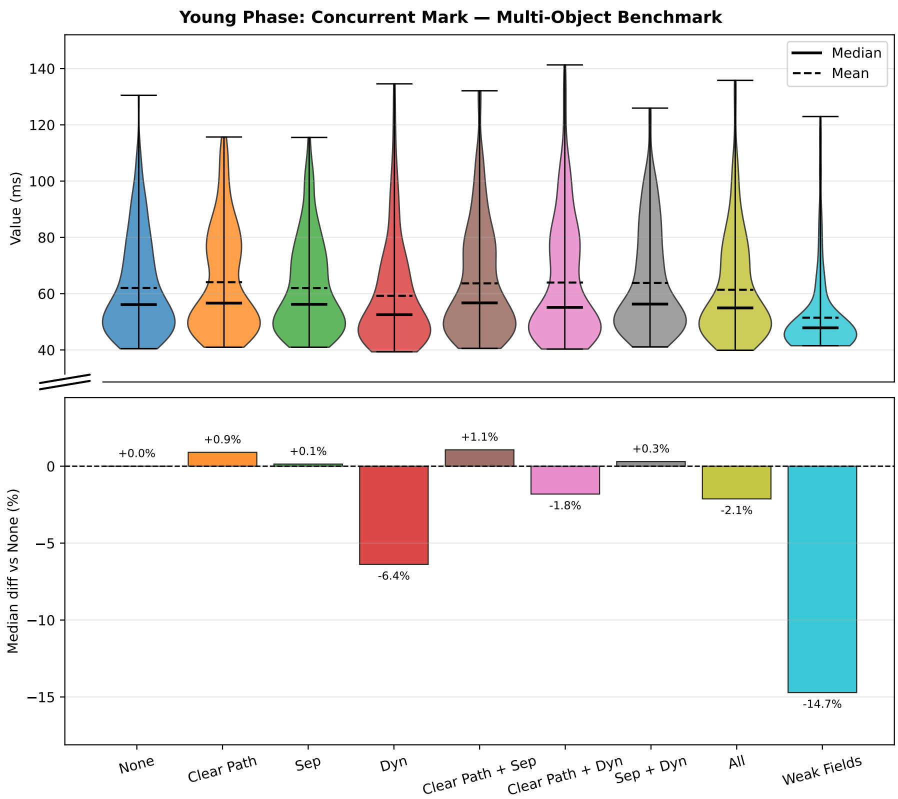

Figure D.4. Young Phase: Concurrent Mark timing, **multi-object benchmark** (per-run averages). Top: distribution, bottom: median % difference vs baseline.

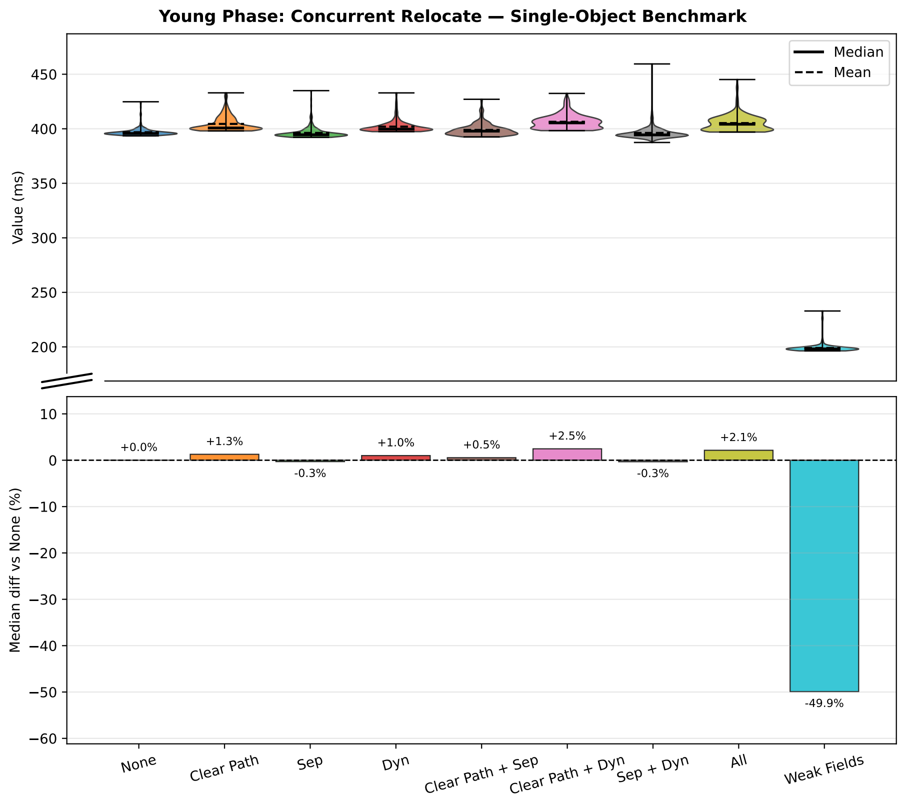

Figure D.5. Young Phase: Concurrent Relocate timing, **single-object benchmark** (per-run maxima). Top: distribution, bottom: median % difference vs baseline.

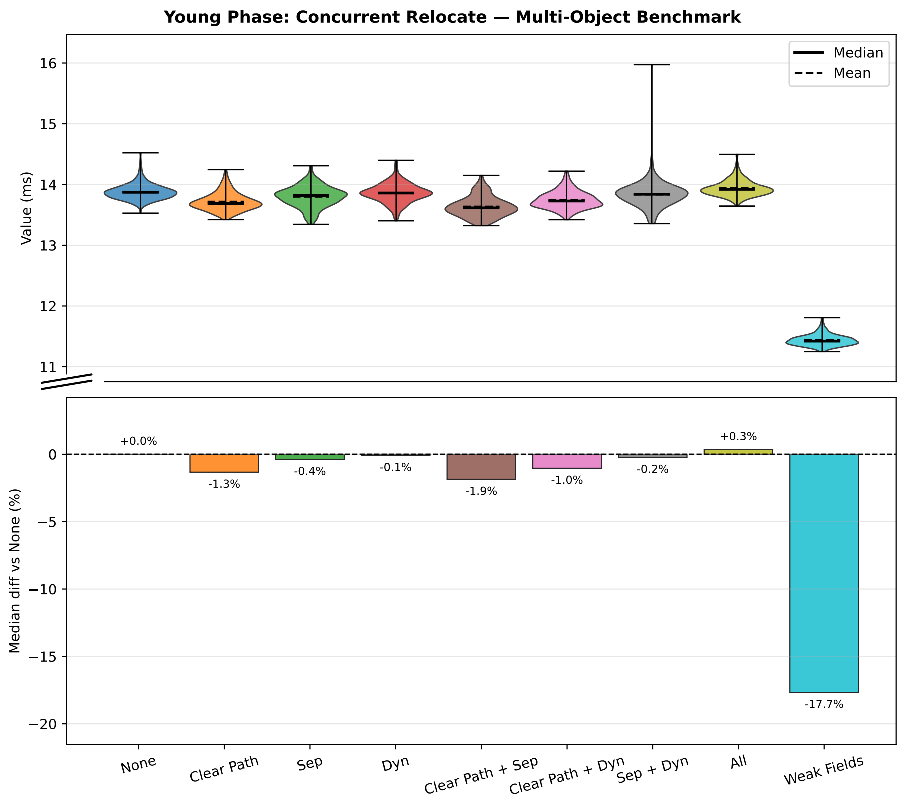

Figure D.6. Young Phase: Concurrent Relocate timing, **multi-object benchmark** (per-run averages). Top: distribution, bottom: median % difference vs baseline.

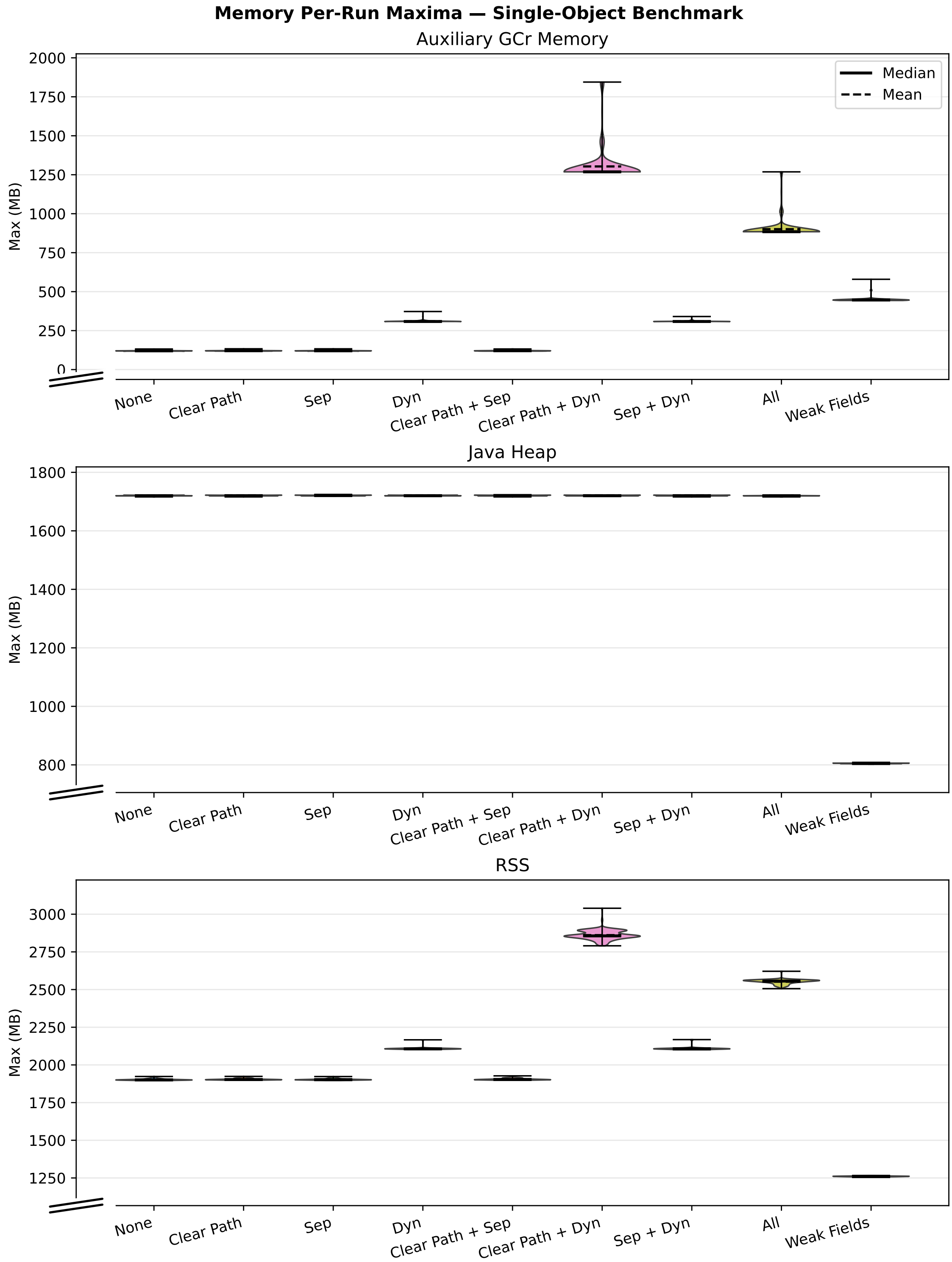

Figure D.7. Distribution of per-run maximum memory across variants, **single-object benchmark**. Top to bottom: auxiliary GCr memory, Java heap, total process RSS.

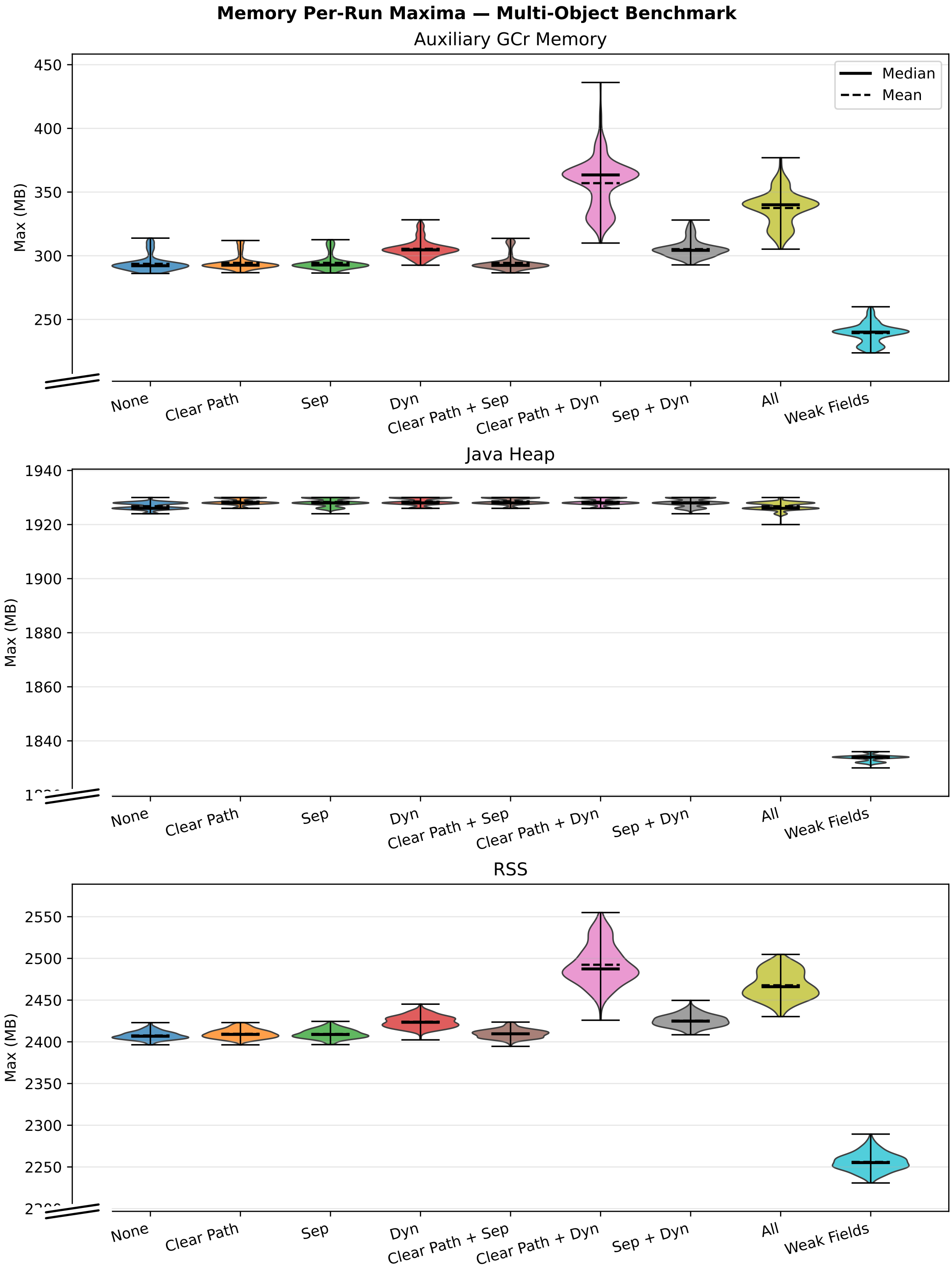

Figure D.8. Distribution of per-run maximum memory across variants, **multi-object benchmark**. Top to bottom: auxiliary GCr memory, Java heap, total process RSS.

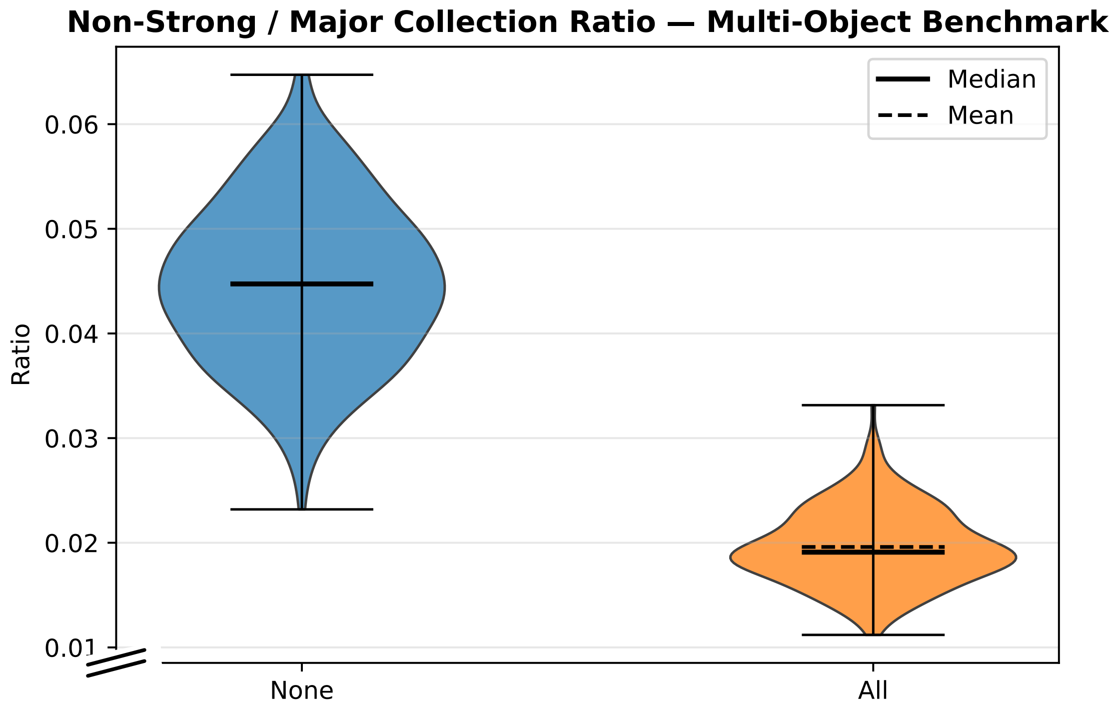

Figure D.9. Ratio of non-strong processing time to total major collection time, **multi-object benchmark** (per-run averages). Distribution across variants.

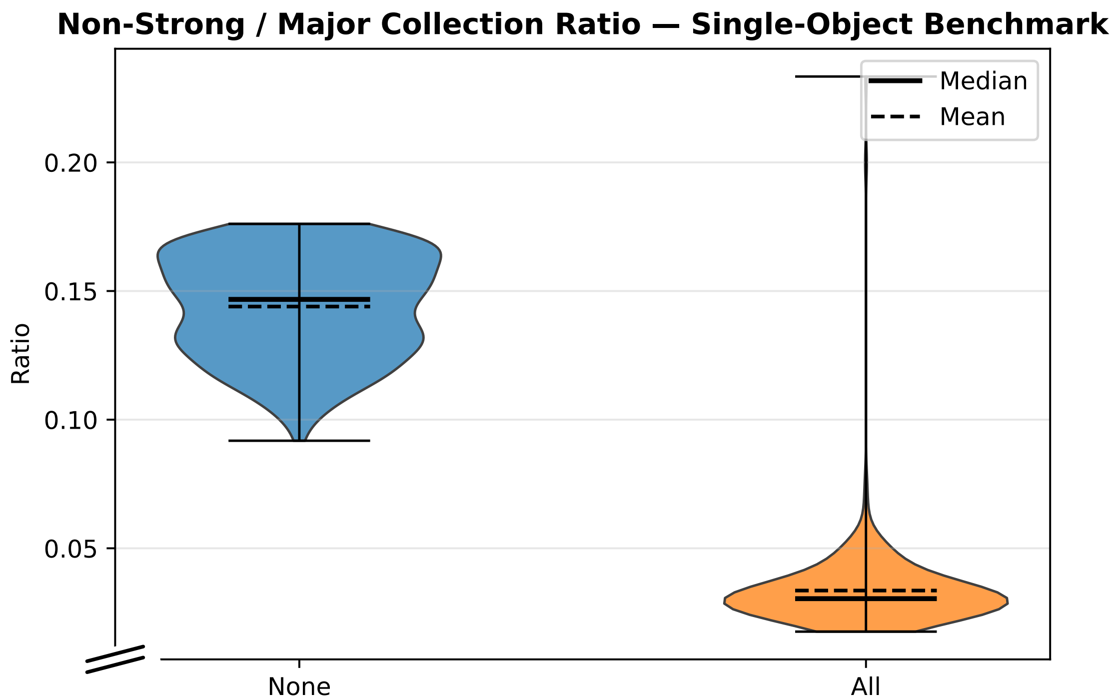

Figure D.10. Ratio of non-strong processing time to total major collection time, **single-object benchmark** (per-run maxima). Distribution across variants.

Appendix E. Source Code

This appendix provides the key source code for the implemented optimisations. The code for the three core optimisations is from the combined (`all`) configuration variant, which includes all three optimisations. The code for the weak-field mechanism is from the `weak_fields` configuration variant. Complete sources for all variants are available in the `patches/` directory of the project repository.

E.1 Dynamic-array Data Structure

The `ZWeakRefArray` template is the central data structure for the dynamic-array optimisation. It stores discovered weak references contiguously on the C heap, enabling sequential traversal without pointer chasing. Each entry in the extended variant (`ZWeakRefDataExt`) pre-loads the referent field address and value during discovery so that the clear path can operate without additional heap loads. Note in particular the `clear_and_reserve` method, which retains capacity between GC cycles to avoid repeated reallocation when the reference population is stable.
File: `zWeakRefArray.hpp`

```
/*
 * Copyright (c) 2025, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

#ifndef SHARE_GC_Z_ZWEAKREFARRAY_HPP
#define SHARE_GC_Z_ZWEAKREFARRAY_HPP

#include "gc/z/zAddress.hpp"
#include "memory/allocation.hpp"
#include "utilities/debug.hpp"
#include "utilities/powerOfTwo.hpp"
#include "utilities/globalDefinitions.hpp"
#include <string.h>

```

```
struct ZWeakRefData {
  zpointer* referent_field_addr;
  zaddress* discovered_field_addr;
  zaddress referent_addr;
  zpointer referent_field_value;
};


class ZWeakRefArray : public AnyObj {
private:
  ZWeakRefData* _data;
  const size_t _ZWeakRefData_size = sizeof(ZWeakRefData);
  size_t _length;
  size_t _capacity;

  void expand_to(size_t new_capacity) {
    assert(new_capacity > _capacity, "expected growth but %ld <= %ld",
        new_capacity, _capacity);

    ZWeakRefData* new_data = (ZWeakRefData*)AllocateHeap(new_capacity *
        _ZWeakRefData_size, mtGC);

    if (_length > 0) {
      // Use memcpy for trivially copyable types - much faster than element-by-
          element copy
      memcpy(new_data, _data, _length * _ZWeakRefData_size);
    }

    if (_data != nullptr) {
      FreeHeap(_data);
    }

    _data = new_data;
    _capacity = new_capacity;
  }

  void grow(size_t min_capacity) {
    size_t new_capacity = MAX2((size_t) 8, next_power_of_2(min_capacity - 1));
    expand_to(new_capacity);
  }

public:
  ZWeakRefArray() : _data(nullptr), _length(0), _capacity(0) {}

  ~ZWeakRefArray() {
    if (_data != nullptr) {
      FreeHeap(_data);
      _data = nullptr;
    }
  }

  void append(const ZWeakRefData& entry) {
    if (_length >= _capacity) {
      grow(_length + 1);
    }
    _data[_length] = entry;
    _length++;
  }

  // Get entry at index
  const ZWeakRefData& at(size_t index) const {
```

```
    assert(index < _length, "index out of bounds: %ld (length: %ld)", index, _length);
    return _data[index];
  }

  // Current length
  size_t length() const {
    return _length;
  }

  // Whether the array is empty
  bool is_empty() const {
    return _length == 0;
  }

  // Current capacity
  size_t capacity() const {
    return _capacity;
  }

  void clear_and_reserve(size_t new_capacity) {
    if (new_capacity == 0) {
      // Special case for clearing to empty - free memory and reset
      if (_data != nullptr) {
        FreeHeap(_data);
        _data = nullptr;
      }
      _length = 0;
      _capacity = 0;
      return;
    }
    _length = 0;
    if (_data != nullptr) {
      FreeHeap(_data);
    }
    _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity - 1));
    _data = (ZWeakRefData*)AllocateHeap(_capacity * _ZWeakRefData_size, mtGC);
  }

  // Clear without deallocating
  void clear() {
    _length = 0;
  }

  // Reserve capacity without clearing
  void reserve(size_t new_capacity) {
    if (new_capacity > _capacity) {
      grow(new_capacity);
    }
  }
};

#endif // SHARE_GC_Z_ZWEAKREFARRAY_HPP
```

E.2 Reference Processor

E.2.1 Interface

The header file shows the per-worker state added by the `all` variant: a `ZWeakRefArray` for queue-less discovered references (bypassing the pending list), a separate `ZWeakRefArray` used by the standard pipeline, and a counter for cleared weak references without a queue. The key type change to notice is `ZWeakRefDataExt` in the queue-less path – this is the extended entry type that pre-loads the referent for the barrier-free clear path.

File: `zReferenceProcessor.hpp`

```
/*
 * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

#ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
#define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

#include "gc/shared/referenceDiscoverer.hpp"
#include "gc/z/zAddress.hpp"
#include "gc/z/zWeakRefArray.hpp"
#include "gc/z/zValue.hpp"

class ConcurrentGCTimer;
class ReferencePolicy;
class ZWorkers;

class ZReferenceProcessor : public ReferenceDiscoverer {
  friend class ZReferenceProcessorTask;

private:
  static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
  typedef size_t Counters[ReferenceTypeCount];

  ZWorkers* const      _workers;
  ReferencePolicy*     _soft_reference_policy;
  bool                 _uses_clear_all_soft_reference_policy;
  ZPerWorker<Counters> _encountered_count;
  ZPerWorker<Counters> _discovered_count;
  ZPerWorker<Counters> _enqueued_count;
  ZPerWorker<size_t>  _encountered_weak_refs_without_queue_count;
  ZPerWorker<size_t>  _discovered_weak_refs_without_queue_count;
  ZPerWorker<size_t>  _cleared_weak_refs_without_queue_count;
  ZPerWorker<zaddress> _discovered_list;
  ZContended<zaddress> _pending_list;
  zaddress             _pending_list_tail;
  ZPerWorker<ZWeakRefArray> _discovered_weak_refs_without_queue_arr;
  ZPerWorker<bool>     _array_empty;
  OopHandle            _null_queue_handle;
```

```
  bool is_inactive(zaddress reference, oop referent, ReferenceType type) const;
  bool is_strongly_live(oop referent) const;
  bool is_softly_live(zaddress reference, ReferenceType type) const;

  bool should_discover(zaddress reference, ReferenceType type, oop referent)
      const;
  bool try_make_inactive(zaddress reference, ReferenceType type) const;
  bool try_make_inactive_fast(const ZWeakRefData& data);

  void discover(zaddress reference, ReferenceType type, zaddress referent);

  void verify_empty() const;

  void process_worker_discovered_list(zaddress discovered_list);
  void process_worker_discovered_weak_refs_without_queue(ZWeakRefArray&
      weak_refs_without_queue);
  void work();
  void collect_statistics();

  zaddress swap_pending_list(zaddress pending_list);

  inline bool has_reference_queue(zaddress reference);

  void initialize_null_queue_handle();

public:
  ZReferenceProcessor(ZWorkers* workers);

  void set_soft_reference_policy(bool clear_all_soft_references);
  bool uses_clear_all_soft_reference_policy() const;

  void reset_statistics();

  virtual bool discover_reference(oop reference, ReferenceType type);
  void process_references();
  void enqueue_references();

  void verify_pending_references();

  void initializeResources();
};

#endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
```

E.2.2 Implementation Excerpts

The following excerpts from `zReferenceProcessor.cpp` show the constructor and per-worker storage setup, the discovery path with queue-less weak-reference routing, the processing paths, and the queue sentinel initialisation.

Constructor and per-worker storage:

```
ZReferenceProcessor::ZReferenceProcessor(ZWorkers* workers)
  : _workers(workers),
    _soft_reference_policy(nullptr),
    _uses_clear_all_soft_reference_policy(false),
    _encountered_count(),
    _discovered_count(),
    _enqueued_count(),
    _encountered_weak_refs_without_queue_count(),
```

```
  _discovered_weak_refs_without_queue_count(),
  _cleared_weak_refs_without_queue_count(),
  _discovered_list(zaddress::null),
  _pending_list(zaddress::null),
  _pending_list_tail(zaddress::null),
  _discovered_weak_refs_without_queue_arr(),
  _array_empty(),
  _null_queue_handle() {

 _array_empty.set_all(true);
 log_info(gc, ref)("Reference Processor created, Config: All optimisations");
}
```

Discovery path with queue-less routing:

```
void ZReferenceProcessor::discover(zaddress reference, ReferenceType type, zaddress referent) {
 log_trace(gc, ref)("Discovered Reference: " PTR_FORMAT " (%s)", untype(reference), reference_type_name(type));

 assert(ZHeap::heap()->is_old(reference), "Must be old");
 assert(is_null(reference_discovered(reference)), "Already discovered");

 // Update statistics
 if (type == REF_WEAK && !has_reference_queue(reference)) {
  _discovered_weak_refs_without_queue_count.get()++;
 }
 else {
  _discovered_count.get()[type]++;
 }


 if (type == REF_WEAK && !has_reference_queue(reference)) {
  zpointer* const referent_addr = reference_referent_addr_non_vol(reference);
  zaddress* const discovered_addr = reference_discovered_addr(reference);
  const zpointer referent_value = *referent_addr;

  // WeakReference with null ReferenceQueue - remember for special processing
  ZWeakRefArray& weak_refs_without_queue = _discovered_weak_refs_without_queue_arr.get();
  ZWeakRefData data = {referent_addr, discovered_addr, referent, referent_value};
  weak_refs_without_queue.append(data);
  _array_empty.set(false);
  reference_set_discovered(reference, reference); // mark as discovered
 } else {
  if (type == REF_FINAL) {
   // Mark referent (and its reachable subgraph) finalizable. This avoids
   // the problem of later having to mark those objects if the referent is
   // still final reachable during processing.
   volatile zpointer* const referent_addr = reference_referent_addr(reference);
   ZBarrier::mark_barrier_on_old_oop_field(referent_addr, true /* finalizable */);
  }

  // Add reference to discovered list
  zaddress* const list = _discovered_list.addr();
  reference_set_discovered(reference, *list);
  *list = reference;
 }
}

```

```
bool ZReferenceProcessor::discover_reference(oop reference_obj, ReferenceType
    type) {
  if (!RegisterReferences) {
    // Reference processing disabled
    return false;
  }

  log_trace(gc, ref)("Encountered Reference: " PTR_FORMAT " (%s)", p2i(
      reference_obj), reference_type_name(type));

  const zaddress reference = to_zaddress(reference_obj);

  // Update statistics
  if (type == REF_WEAK && !has_reference_queue(reference)) {
    _encountered_weak_refs_without_queue_count.get()++;
  }
  else {
    _encountered_count.get()[type]++;
  }

  if (ZHeap::heap()->is_young(reference)) {
    // Don't discover young references. Young gen reference processing is scary
        and therefore not supported.
    return false;
  }

  volatile zpointer* const referent_addr = reference_referent_addr(reference);
  const zaddress ref_raw_addr = ZBarrier::load_barrier_on_oop_field(
      referent_addr);
  const oop referent = to_oop(ref_raw_addr);

  if (!should_discover(reference, type, referent)) {
    // Not discovered
    return false;
  }

  discover(reference, type, ref_raw_addr);

  // Discovered
  return true;
}
```

Processing paths for separate discovered lists and dynamic arrays:

```
void ZReferenceProcessor::process_worker_discovered_weak_refs_without_queue(
    ZWeakRefArray& weak_refs_without_queue) {
  size_t dropped = 0;
  for (size_t i = 0; i < weak_refs_without_queue.length(); i++) {
    const ZWeakRefData& data = weak_refs_without_queue.at(i);
    // discovered is an oop field and must use a coloured null representation.
    *((zpointer*)data.discovered_field_addr) = color_null(); // Mark as dropped
    if (try_make_inactive_fast(data)) {
      log_trace(gc, ref)("\"Enqueued\" Weak Reference without Queue");
      // Update statistics
      _cleared_weak_refs_without_queue_count.get()++;
    } else {
      log_trace(gc, ref)("Dropped Weak Reference without Queue");
      dropped++;
    }
    SuspendibleThreadSet::yield();
  }
  weak_refs_without_queue.clear_and_reserve(dropped);
}
```

```

void ZReferenceProcessor::work() {
  SuspendibleThreadSetJoiner sts_joiner;

  ZPerWorkerIterator<zaddress> iter(&_discovered_list);
  ZPerWorkerIterator<ZWeakRefArray> iter_weak_refs(&_discovered_weak_refs_without_queue_arr);
  ZPerWorkerIterator<bool> iter_array_empty(&_array_empty);

  zaddress* list_addr = nullptr;
  ZWeakRefArray* array_addr = nullptr;
  bool* array_empty = nullptr;

  for (; iter.next(&list_addr) && iter_weak_refs.next(&array_addr) && iter_array_empty.next(&array_empty);) {

    const zaddress discovered_list = AtomicAccess::xchg(list_addr, zaddress::null);
    const bool has_array = !AtomicAccess::xchg(array_empty, true);
    const bool has_discovered = discovered_list != zaddress::null;

    if (has_array) {
      // Process discovered weak references without queue
      process_worker_discovered_weak_refs_without_queue(*array_addr);
    }
    if (has_discovered) {
      // Process discovered references
      process_worker_discovered_list(discovered_list);
    }
  }
}
```

Queue sentinel initialisation and queue checks:

```
inline bool ZReferenceProcessor::has_reference_queue(zaddress reference) {
  if (_null_queue_handle.is_empty()) {
    // If NULL_QUEUE is not available yet, conservatively treat all references
    // as having a queue and avoid the weak-without-queue fast path.
    log_debug(gc, ref)("ReferenceQueue.NULL_QUEUE not available, treating a reference as having a queue");
    return true;
  }

  const zaddress ref_queue_addr_value = ZBarrier::load_barrier_on_oop_field(reference_queue_addr(reference));
  const oop ref_queue = to_oop(ref_queue_addr_value);
  bool result = ref_queue != _null_queue_handle.resolve();
  return result;
}

void ZReferenceProcessor::initialize_null_queue_handle() {
  EXCEPTION_MARK;
  TempNewSymbol class_name = SymbolTable::new_symbol("java/lang/ref/ReferenceQueue");
  Klass* k = SystemDictionary::resolve_or_fail(class_name, true, CHECK);
  InstanceKlass* ik = InstanceKlass::cast(k);
  ik->initialize(CHECK);
  fieldDescriptor fd;
  bool found = ik->find_local_field(SymbolTable::new_symbol("NULL_QUEUE"),
                                    vmSymbols::referencequeue_signature(), &fd);
  assert(found && fd.is_static(), "ReferenceQueue.NULL_QUEUE missing");
  oop null_q = ik->java_mirror()->obj_field(fd.offset());
  _null_queue_handle = OopHandle(Universe::vm_global(), null_q);
```

```
}

void ZReferenceProcessor::initializeResources() {
  Thread* const current = Thread::current();
  guarantee(current->is_Java_thread(), "Must be called by a JavaThread");
  guarantee(!current->is_ConcurrentGC_thread(), "Must not be called by a GC
      thread");

  initialize_null_queue_handle();
}
```

E.3 Weak Fields

E.3.1 Annotation

The `@weak` annotation is the user-facing entry point for the weak-field mechanism. It is intentionally minimal – a marker annotation with `@Target(ElementType.FIELD)` and `@Retention(RetentionPolicy.RUNTIME)`. The runtime retention is required because the class-file parser must be able to detect the annotation on field descriptors when loading the class; it is not consumed by the JIT compilers, which instead query the resolved field entry.

File: `java/lang/ref/weak.java`

```
/*
 * Copyright (c) 2026, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation. Oracle designates this
 * particular file as subject to the "Classpath" exception as provided
 * by Oracle in the LICENSE file that accompanied this code.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

package java.lang.ref;

import java.lang.annotation.*;

/**
 * A field annotation for references that should be treated as weak by the VM.
 */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface weak {}
```

E.3.2 Data Structure

The `ZWeakFieldArray` is the weak-fields counterpart of `ZWeakRefArray`. The key structural difference is that each entry records the *field address* (a pointer into the containing object's fields) rather than the address of a `WeakReference` wrapper object. This is the detail that allows the GC worker to clear the field in-place without involving the reference pipeline at all. The entry size is smaller than `ZWeakRefDataExt` because there is no queue or discovered field to track.

File: `zWeakFieldArray.hpp`

```cpp
/*
 * Copyright (c) 2025, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation.
 *
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

#ifndef SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
#define SHARE_GC_Z_ZWEAKFIELDARRAY_HPP

#include "gc/z/zAddress.hpp"
#include "memory/allocation.hpp"
#include "utilities/debug.hpp"
#include "utilities/powerOfTwo.hpp"
#include "utilities/globalDefinitions.hpp"
#include <string.h>

struct ZWeakFieldData {
  volatile zpointer* field_addr;
  zpointer field_value;
};

// High-performance growable array specifically for storing discovered reference data.
// Uses array-of-structs (AoS) layout for better cache locality when processing sequentially.
// The stored entry type is configured by the template parameter.
//
// Performance optimizations:
// - Uses memcpy for bulk copying instead of element-by-element loops
// - Does not zero newly allocated memory (caller responsible for initialization)
// - Single clear_and_reserve operation to avoid separate clear/reserve calls
// - Array-of-structs layout improves cache efficiency for sequential processing
class ZWeakFieldArray : public AnyObj {
private:
  ZWeakFieldData* _data;
```

```cpp
  const size_t _ZWeakFieldData_size = sizeof(ZWeakFieldData);
  size_t _length;
  size_t _capacity;

  void expand_to(size_t new_capacity) {
    assert(new_capacity > _capacity, "expected growth but %ld <= %ld", new_capacity, _capacity);

    ZWeakFieldData* new_data = (ZWeakFieldData*)AllocateHeap(new_capacity * _ZWeakFieldData_size, mtGC);

    if (_length > 0) {
      // Use memcpy for trivially copyable types - much faster than element-by-element copy
      memcpy(new_data, _data, _length * _ZWeakFieldData_size);
    }

    if (_data != nullptr) {
      FreeHeap(_data);
    }

    _data = new_data;
    _capacity = new_capacity;
  }

  void grow(size_t min_capacity) {
    size_t new_capacity = MAX2((size_t) 8, next_power_of_2(min_capacity - 1));
    expand_to(new_capacity);
  }

public:
  ZWeakFieldArray() : _data(nullptr), _length(0), _capacity(0) {}

  ~ZWeakFieldArray() {
    if (_data != nullptr) {
      FreeHeap(_data);
      _data = nullptr;
    }
  }

  void append(const ZWeakFieldData& entry) {
    if (_length >= _capacity) {
      grow(_length + 1);
    }
    _data[_length] = entry;
    _length++;
  }

  // Get entry at index
  const ZWeakFieldData& at(size_t index) const {
    assert(index < _length, "index out of bounds: %ld (length: %ld)", index, _length);
    return _data[index];
  }

  // Current length
  size_t length() const {
    return _length;
  }

  // Whether the array is empty
  bool is_empty() const {
```

```
    return _length == 0;
  }

  // Current capacity
  size_t capacity() const {
    return _capacity;
  }

  void clear_and_reserve(size_t new_capacity) {
    if (new_capacity == 0) {
      // Special case for clearing to empty - free memory and reset
      if (_data != nullptr) {
        FreeHeap(_data);
        _data = nullptr;
      }
      _length = 0;
      _capacity = 0;
      return;
    }
    _length = 0;
    if (_data != nullptr) {
      FreeHeap(_data);
    }
    _capacity = MAX2((size_t) 8, next_power_of_2(new_capacity - 1));
    _data = (ZWeakFieldData*)AllocateHeap(_capacity * _ZWeakFieldData_size, mtGC);
  }

  // Clear without deallocating
  void clear() {
    _length = 0;
  }

  // Reserve capacity without clearing
  void reserve(size_t new_capacity) {
    if (new_capacity > _capacity) {
      grow(new_capacity);
    }
  }
};

#endif // SHARE_GC_Z_ZWEAKFIELDARRAY_HPP
```

E.3.3 Reference Processor

The weak-fields variant of the reference processor header shows the additional per-worker `ZWeak-FieldArray` alongside the standard weak-reference discovered lists. The implementation excerpts below illustrate the three key changes: how the discovery filtering recognises weak fields and diverts them away from the standard reference pipeline; how fields are appended to the per-worker array; and how the processing function iterates the array and clears each field entry.

File: `zReferenceProcessor.hpp`

```
/*
 * Copyright (c) 2015, 2024, Oracle and/or its affiliates. All rights reserved.
 * DO NOT ALTER OR REMOVE COPYRIGHT NOTICES OR THIS FILE HEADER.
 *
 * This code is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 only, as
 * published by the Free Software Foundation.
 *
```

```
 * This code is distributed in the hope that it will be useful, but WITHOUT
 * ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
 * FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License
 * version 2 for more details (a copy is included in the LICENSE file that
 * accompanied this code).
 *
 * You should have received a copy of the GNU General Public License version
 * 2 along with this work; if not, write to the Free Software Foundation,
 * Inc., 51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA.
 *
 * Please contact Oracle, 500 Oracle Parkway, Redwood Shores, CA 94065 USA
 * or visit www.oracle.com if you need additional information or have any
 * questions.
 */

#ifndef SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
#define SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP

#include "gc/shared/referenceDiscoverer.hpp"
#include "gc/z/zAddress.hpp"
#include "gc/z/zWeakFieldArray.hpp"
#include "gc/z/zValue.hpp"

class ConcurrentGCTimer;
class ReferencePolicy;
class ZWorkers;

class ZReferenceProcessor : public ReferenceDiscoverer {
  friend class ZReferenceProcessorTask;

private:
  static const size_t ReferenceTypeCount = REF_PHANTOM + 1;
  typedef size_t Counters[ReferenceTypeCount];

  ZWorkers* const        _workers;
  ReferencePolicy*       _soft_reference_policy;
  bool                   _uses_clear_all_soft_reference_policy;
  ZPerWorker<Counters>   _encountered_count;
  ZPerWorker<Counters>   _discovered_count;
  ZPerWorker<Counters>   _enqueued_count;
  ZPerWorker<size_t>     _encountered_weak_fields_count;
  ZPerWorker<size_t>     _discovered_weak_fields_count;
  ZPerWorker<size_t>     _cleared_weak_fields_count;
  ZPerWorker<zaddress>   _discovered_list;
  ZContended<zaddress>   _pending_list;
  zaddress               _pending_list_tail;
  ZPerWorker<ZWeakFieldArray> _discovered_weak_fields;
  ZPerWorker<bool>       _discovered_weak_fields_empty;

  bool is_inactive(zaddress reference, oop referent, ReferenceType type) const;
  bool is_strongly_live(oop referent) const;
  bool is_softly_live(zaddress reference, ReferenceType type) const;

  bool should_discover(zaddress reference, ReferenceType type) const;
  bool should_discover_weak_field(zpointer field_value, volatile zpointer*
      field_addr) const;
  bool try_make_inactive(zaddress reference, ReferenceType type) const;
  bool try_make_inactive_fast(const ZWeakFieldData& data);

  void discover(zaddress reference, ReferenceType type);

  void verify_empty() const;
```

```cpp

  void process_worker_discovered_list(zaddress discovered_list);
  void process_worker_discovered_weak_fields(ZWeakFieldArray& weak_fields);
  void work();
  void collect_statistics();

  zaddress swap_pending_list(zaddress pending_list);

public:
  ZReferenceProcessor(ZWorkers* workers);

  bool discover_weak_field(volatile zpointer* field_addr);

  void set_soft_reference_policy(bool clear_all_soft_references);
  bool uses_clear_all_soft_reference_policy() const;

  void reset_statistics();

  virtual bool discover_reference(oop reference, ReferenceType type);
  void process_references();
  void enqueue_references();

  void verify_pending_references();
};

#endif // SHARE_GC_Z_ZREFERENCEPROCESSOR_HPP
```

The following excerpts from `zReferenceProcessor.cpp` show the discovery filtering, the discovery function, and the processing function.

Discovery filtering:

```cpp
bool ZReferenceProcessor::should_discover_weak_field(zpointer referent_ptr,
    volatile zpointer* field_addr) const {
  if (ZPointer::is_mark_good(referent_ptr)) {
    // Field points to a strongly live object, no need to discover
    return false;
  }

  if (is_null_any(referent_ptr)) {
    // Field is null but not yet marked
    AtomicAccess::cmpxchg(field_addr, referent_ptr, color_null(),
        memory_order_relaxed);
    return false;
  }

  const zaddress referent_addr = ZBarrier::make_load_good(referent_ptr);

  if (ZHeap::heap()->is_young(referent_addr) || ZHeap::heap()->
      is_object_strongly_live(referent_addr)) {
    // Field points to a strongly live object, no need to discover
    return false;
  }

  // Field points to an object that is not strongly live, we should discover
  return true;
}

bool ZReferenceProcessor::try_make_inactive(zaddress reference, ReferenceType
    type) const {
  const zpointer referent = reference_referent(reference);

  if (is_null_any(referent)) {
```

Weak-field discovery:

```
bool ZReferenceProcessor::discover_weak_field(volatile zpointer* field_addr) {
  if (!RegisterReferences) {
    // Reference processing disabled
    return false;
  }

  log_trace(gc, ref)("Encountered Weak Reference Field: " PTR_FORMAT, p2i(
      field_addr));

  // Update statistics
  _encountered_weak_fields_count.get()++;

  if (ZHeap::heap()->is_young(to_zaddress(p2i(field_addr)))) {
    // Don't discover young references. Young gen reference processing is scary
        and therefore not supported.
    return false;
  }

  const zpointer field_value = ZBarrier::load_atomic(field_addr);

  if (!should_discover_weak_field(field_value, field_addr)) {
    // Not discovered
    return false;
  }

  // Discovered
  log_trace(gc, ref)("Discovered Weak Reference Field: " PTR_FORMAT, p2i(
      field_addr));

  // Update statistics
  _discovered_weak_fields_count.get()++;

  ZWeakFieldArray& weak_fields = _discovered_weak_fields.get();
  ZWeakFieldData data = {field_addr, field_value};
  weak_fields.append(data);
  _discovered_weak_fields_empty.set(false);

  return true;
}
```

Weak-field processing:

```
void ZReferenceProcessor::process_worker_discovered_weak_fields(ZWeakFieldArray&
     weak_fields) {
  size_t dropped = 0;
  for (size_t i = 0; i < weak_fields.length(); i++) {
    const ZWeakFieldData& data = weak_fields.at(i);
    if (try_make_inactive_fast(data)) {
      log_trace(gc, ref)("\"Cleared\" Weak Reference Field: " PTR_FORMAT, p2i(
          data.field_addr));
      // Update statistics
      _cleared_weak_fields_count.get()++;
    } else {
      log_trace(gc, ref)("Dropped Weak Reference Field: " PTR_FORMAT, p2i(data.
          field_addr));
      dropped++;
    }
    SuspendibleThreadSet::yield();
  }
  weak_fields.clear_and_reserve(dropped);
```

```
}
```

E.3.4 Marking Closure

The following excerpt from `zMark.cpp` shows how weak-annotated fields are detected and diverted to the weak-field discovery path during object marking.

Field annotation check and discovery diversion:

```
  bool is_weak_annotated_field(oop* p) const {
    if (_current_instance_klass == nullptr) {
      return false;
    }

    const uintptr_t addr = (uintptr_t)p;
    if (addr < _current_obj_start || addr >= _current_obj_end) {
      return false;
    }

    const int offset = (int)(addr - _current_obj_start);
    fieldDescriptor fd;
    if (!_current_instance_klass->find_field_from_offset(offset, false /* is_static
        */, &fd)) {
      return false;
    }

    return fd.is_weak();
  }

  const bool _visit_metadata;

public:
  ZMarkBarrierFollowOopClosure()
    : OopIterateClosure(discoverer()),
      _current_instance_klass(nullptr),
      _current_obj_start(0),
      _current_obj_end(0),
      _visit_metadata(visit_metadata()) {}

  void set_iteration_object(oop obj) {
    if (obj->klass()->is_instance_klass()) {
      _current_instance_klass = InstanceKlass::cast(obj->klass());
      _current_obj_start = cast_from_oop<uintptr_t>(obj);
      _current_obj_end = _current_obj_start + (obj->size() * HeapWordSize);
    } else {
      _current_instance_klass = nullptr;
      _current_obj_start = 0;
      _current_obj_end = 0;
    }
  }

  void clear_iteration_object() {
    _current_instance_klass = nullptr;
    _current_obj_start = 0;
    _current_obj_end = 0;
  }

  virtual void do_oop(oop* p) {
    if (!finalizable && is_weak_annotated_field(p)) {
      if (ZGeneration::old()->discover_weak_field((volatile zpointer*)p)) {
        return;
```

```
    }
  }

  switch (generation) {
  case ZGenerationIdOptional::young:
    ZBarrier::mark_barrier_on_young_oop_field((volatile zpointer*)p);
    break;
  case ZGenerationIdOptional::old:
    ZBarrier::mark_barrier_on_old_oop_field((volatile zpointer*)p, finalizable);
    break;
  case ZGenerationIdOptional::none:
    ZBarrier::mark_barrier_on_oop_field((volatile zpointer*)p, finalizable);
    break;
  }
}
```

E.3.5 Compiler and Interpreter Support

The following excerpts show how the `@weak` decorator is applied in each execution tier. In all three tiers, the pattern is the same: if the field carries the `weak` flag and ZGC is active, `ON_WEAK_OOP_REF` is added to the decorator set before the access is performed.

C2 (opto/parse3.cpp) – field load and store:

```
void Parse::do_get_xxx(Node* obj, ciField* field, bool is_field) {
  BasicType bt = field->layout_type();

  // Does this field have a constant value? If so, just push the value.
  if (field->is_constant() &&
      // Keep consistent with types found by ciTypeFlow: for an
      // unloaded field type, ciTypeFlow::StateVector::do_getstatic()
      // speculates the field is null. The code in the rest of this
      // method does the same. We must not bypass it and use a non
      // null constant here.
      (bt != T_OBJECT || field->type()->is_loaded())) {
    // final or stable field
    Node* con = make_constant_from_field(field, obj);
    if (con != nullptr) {
      push_node(field->layout_type(), con);
      return;
    }
  }

  ciType* field_klass = field->type();
  bool is_vol = field->is_volatile();

  // Compute address and memory type.
  int offset = field->offset_in_bytes();
  const TypePtr* adr_type = C->alias_type(field)->adr_type();
  Node *adr = basic_plus_adr(obj, obj, offset);
  assert(C->get_alias_index(adr_type) == C->get_alias_index(_gvn.type(adr)->
      isa_ptr()),
    "slice of address and input slice don't match");

  // Build the resultant type of the load
  const Type *type;

  bool must_assert_null = false;

  DecoratorSet decorators = IN_HEAP;
  decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;
```

```

bool is_obj = is_reference_type(bt);

if (is_obj) {
  if (UseZGC && field->is_weak()) {
    decorators |= ON_WEAK_OOP_REF;
  }
  if (!field->type()->is_loaded()) {
    type = TypeInstPtr::BOTTOM;
    must_assert_null = true;
  } else if (field->is_static_constant()) {
    // This can happen if the constant oop is non-perm.
    ciObject* con = field->constant_value().as_object();
    // Do not "join" in the previous type; it doesn't add value,
    // and may yield a vacuous result if the field is of interface type.
    if (con->is_null_object()) {
      type = TypePtr::NULL_PTR;
    } else {
      type = TypeOopPtr::make_from_constant(con)->isa_oopptr();
    }
    assert(type != nullptr, "field singleton type must be consistent");
  } else {
    type = TypeOopPtr::make_from_klass(field_klass->as_klass());
  }
} else {
  type = Type::get_const_basic_type(bt);
}

Node* ld = access_load_at(obj, adr, adr_type, type, bt, decorators);
if (is_obj && UseZGC && field->is_weak()) {
  insert_mem_bar(Op_MemBarCPUOrder);
}

// Adjust Java stack
if (type2size[bt] == 1)
  push(ld);
else
  push_pair(ld);

if (must_assert_null) {
  // Do not take a trap here. It's possible that the program
  // will never load the field's class, and will happily see
  // null values in this field forever. Don't stumble into a
  // trap for such a program, or we might get a long series
  // of useless recompilations. (Or, we might load a class
  // which should not be loaded.) If we ever see a non-null
  // value, we will then trap and recompile. (The trap will
  // not need to mention the class index, since the class will
  // already have been loaded if we ever see a non-null value.)
  // uncommon_trap(iter().get_field_signature_index());
  if (PrintOpto && (Verbose || WizardMode)) {
    method()->print_name(); tty->print_cr(" asserting nullness of field at bci:
        %d", bci());
  }
  if (C->log() != nullptr) {
    C->log()->elem("assert_null reason='field' klass='%d'",
                   C->log()->identify(field->type()));
  }
  // If there is going to be a trap, put it at the next bytecode:
  set_bci(iter().next_bci());
  null_assert(peek());
  set_bci(iter().cur_bci()); // put it back
```

```
  }
}

void Parse::do_put_xxx(Node* obj, ciField* field, bool is_field) {
  bool is_vol = field->is_volatile();

  // Compute address and memory type.
  int offset = field->offset_in_bytes();
  const TypePtr* adr_type = C->alias_type(field)->adr_type();
  Node* adr = basic_plus_adr(obj, obj, offset);
  assert(C->get_alias_index(adr_type) == C->get_alias_index(_gvn.type(adr)->
      isa_ptr()),
    "slice of address and input slice don't match");
  BasicType bt = field->layout_type();
  // Value to be stored
  Node* val = type2size[bt] == 1 ? pop() : pop_pair();

  DecoratorSet decorators = IN_HEAP;
  decorators |= is_vol ? MO_SEQ_CST : MO_UNORDERED;

  bool is_obj = is_reference_type(bt);

  if (is_obj) {
    if (UseZGC && field->is_weak()) {
      decorators |= ON_WEAK_OOP_REF;
    }
  }

  // Store the value.
  const Type* field_type;
  if (!field->type()->is_loaded()) {
    field_type = TypeInstPtr::BOTTOM;
  } else {
    if (is_obj) {
      field_type = TypeOopPtr::make_from_klass(field->type()->as_klass());
    } else {
      field_type = Type::BOTTOM;
    }
  }
  access_store_at(obj, adr, adr_type, val, field_type, bt, decorators);

  if (is_field) {
    // Remember we wrote a volatile field.
    // For not multiple copy atomic cpu (ppc64) a barrier should be issued
    // in constructors which have such stores. See do_exits() in parse1.cpp.
    if (is_vol) {
      set_wrote_volatile(true);
    }
    set_wrote_fields(true);

    // If the field is final, the rules of Java say we are in <init> or <clinit>.
    // If the field is @Stable, we can be in any method, but we only care about
    // constructors at this point.
    //
    // Note the presence of writes to final/@Stable non-static fields, so that we
    // can insert a memory barrier later on to keep the writes from floating
    // out of the constructor.
    if (field->is_final() || field->is_stable()) {
```

C1 (c1/c1_LIRGenerator.cpp) – field load decorator:

```
  }
```

```

  DecoratorSet decorators = IN_HEAP;
  if (is_volatile) {
    decorators |= MO_SEQ_CST;
  }
  if (needs_patching) {
    decorators |= C1_NEEDS_PATCHING;
  } else if (UseZGC && field_type == T_OBJECT && x->field()->is_weak()) {
    decorators |= ON_WEAK_OOP_REF;
  }

  LIR_Opr result = rlock_result(x, field_type);
  access_load_at(decorators, field_type,
                 object, LIR_OprFact::intConst(x->offset()), result,
                 info ? new CodeEmitInfo(info) : nullptr, info);
}
```

C1 (c1/c1_LIRGenerator.cpp) – field store decorator:

```

  DecoratorSet decorators = IN_HEAP;
  if (is_volatile) {
    decorators |= MO_SEQ_CST;
  }
  if (needs_patching) {
    decorators |= C1_NEEDS_PATCHING;
  } else if (UseZGC && field_type == T_OBJECT && x->field()->is_weak()) {
    decorators |= ON_WEAK_OOP_REF;
  }

  access_store_at(decorators, field_type, object, LIR_OprFact::intConst(x->offset()),
                  value.result(), info != nullptr ? new CodeEmitInfo(info) : nullptr, info);
}

void LIRGenerator::do_StoreIndexed(StoreIndexed* x) {
  assert(x->is_pinned(),"");
  bool needs_range_check = x->compute_needs_range_check();
```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – getfield weak path:

```
  __ bind(notBool);
  __ cmpl(tos_state, atos);
  __ jcc(Assembler::notEqual, notObj);
  // atos
  if (UseZGC) {
    __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
    __ jcc(Assembler::zero, strongOopLoad);
    // Weak load path
    do_oop_load(_masm, field, rax, ON_WEAK_OOP_REF);
    __ push(atos);
    // Don't rewrite bytecode for weak references, the fast path is only for strong loads
    __ jmp(Done);
    __ bind(strongOopLoad);
  }
  do_oop_load(_masm, field, rax);
  __ push(atos);
  if (!is_static && rc == may_rewrite) {
    patch_bytecode(Bytecodes::_fast_agetfield, bc, rbx);
  }
```

```
  __ jmp(Done);
```

Template Interpreter (cpu/x86/templateTable_x86.cpp) – putfield weak path:

```
// atos
{
  if (UseZGC) {
    // Test weak flag before pop(atos) overwrites flags (rax)
    __ testl(flags, (1 << ResolvedFieldEntry::is_weak_shift));
    __ jcc(Assembler::notZero, strongOopStore);
    // Weak store path
    __ pop(atos);
    if (!is_static) pop_and_check_object(obj);
    do_oop_store(_masm, field, rax, ON_WEAK_OOP_REF);
    // Don't rewrite bytecode for weak references, the fast path is only for
        strong stores
    __ jmp(Done);
    // Strong store path
    __ bind(strongOopStore);
  }

  __ pop(atos);
  if (!is_static) pop_and_check_object(obj);
  do_oop_store(_masm, field, rax);
  if (!is_static && rc == may_rewrite) {
    patch_bytecode(Bytecodes::_fast_aputfield, bc, rbx, true, byte_no);
  }
  __ jmp(Done);
}
```